1 Preparation
2 Introduction and Intended Use
3 Study Design for the data and its effects
4 Data Loading and Cleaning
5 Exploratory Data Analysis
6 Feature Engineering and Data Splitting
7 Model Development and Hyperparameter Tuning
8 Model Evaluation
9 Error Analysis
10 Feature Importance and Substantive Interpretation
11 Conclusion
12 Implementation Plan
13 Monitoring and Recalibration Plan
References

Classifying Music Production Aesthetic from Spotify Audio Features

Code

HEFF

2026-05-20

1 Preparation

Hide

```
# Data manipulation
library(tidyverse)
library(janitor)

# Visualization
library(ggplot2)
library(GGally)
library(patchwork)
library(tidytext)

# Machine learning
library(tidymodels)
library(glmnet)
library(ranger)
library(xgboost)
library(vip)
library(SuperLearner)

# Evaluation
library(pROC)
library(yardstick)

# Parallel computation
library(doParallel)
library(parallel)

# Reproducibility
set.seed(2026)
```

2 Introduction and Intended Use

Music production aesthetic refers to the sonic character of a track shaped by mastering, compression, mixing, and arrangement decisions rather than compositional choices about melody or harmony. Two broad production aesthetics have been identifiable across popular music for at least three decades. A **maximalist** aesthetic is characterized by loudness, density, and dynamic compression, with roots in hard rock, grunge, and the compressed commercial pop production pioneered by producers such as Max Martin in the 1990s and 2000s (Gracyk, 1996; Seabrook, 2015; Walser, 1993). An **alternative** aesthetic, emerging as a counter-movement in independent rock and post-punk production, is characterized by dynamic range, spatial depth, and sonic restraint (Azerrad, 2002; Reynolds, 2006). Neither aesthetic is genre-bound; both have diffused across genre categories as producers moved between musical worlds.

The predictor developed here classifies songs as alternative or maximalist based on Spotify audio features. It could be used by music streaming platforms to organize playlists and recommendation systems by production aesthetic rather than genre, by producers and A&R professionals to identify aesthetic trends across a catalog, or by researchers studying how production practices have changed over time. The predictor takes as input the standard Spotify audio features for any song and outputs a probability of belonging to the maximalist category and a binary classification.

Loudness is deliberately excluded despite being the most theoretically direct correlate of maximalism, because the aesthetic labels used in this study were derived partly from producer communities associated with loudness practices, and including it would introduce circular reasoning into the predictor. The exclusion also serves a constructive purpose by demonstrating that production aesthetic is computationally detectable from audio features beyond its most obvious acoustic correlate.

3 Study Design for the data and its effects

3.1 Primary Dataset

The primary dataset is the Spotify Tracks Dataset, a publicly available collection of approximately 114,000 songs distributed across 114 genres, with roughly 1,000 tracks represented in each genre. The dataset is available through Kaggle and has been used in previous computational music studies (Asmitha & Kavitha, 2026). The dataset spans recordings from the 1960s onward in order to capture the broader history of popular music production, valid and necessary for the prediction task. For each track, Spotify provides twelve audio features that are used as predictors in the classification models. These variables are summarized in Table 1.

Table 1. The Echo Nest Sonic Features

Attribute	Scale	Definition
Acousticness	0–1	Represents the likelihood that the song was recorded solely by acoustic means (as opposed to more electronic/electric means).
Danceability	0–1	Describes how suitable a track is for dancing. This measure includes tempo, regularity of beat, and beat strength.
Energy	0–1	A perceptual measure of intensity throughout the track. Think fast, loud, and noisy (i.e., hard rock) more than dance tracks.
Instrumentalness	0–1	The likelihood that a track is predominantly instrumental. Not necessarily the inverse of speechiness.
Key	0–11 (integers only)	The estimated, overall key of the track, from C through B. Entered as a series of dummy variables in our analyses.
Liveness	0–1	Detects the presence of a live audience during the recording. Heavily studio-produced tracks score low on this measure.
Mode	0 or 1	Whether the song is in a minor (0) or major (1) key.
Speechiness	0–1	Detects the presence of spoken word throughout the track. Sung vocals are not considered spoken word.
Tempo	Beats per minute (BPM)	The overall average tempo of a track.
Time Signature	Beats per bar/measure	Estimated, overall time signature of a track. 4/4 is the most common time signature by far and is entered as a dummy variable in our analyses.
Valence	0–1	The musical positiveness of the track.

Kaggle link: https://www.kaggle.com/datasets/priyamchoksi/spotify-dataset-114k-songs (The introduction says there are 125 genres in the dataset but that is inaccurate. I only found 114 genres in the actual dataset.)

Together, they provide a standardized numerical representation of musical recordings that can be compared across large collections of songs. Rather than working directly with audio waveforms, the analysis relies on these pre-computed variables as indicators of rhythm, dynamics, timbre, tonality, vocal presence, and other perceptually salient characteristics of recorded music.

These features originated with The Echo Nest, a music intelligence company acquired by Spotify in 2014, and are now integrated into Spotify's audio analysis infrastructure. They are generated algorithmically through a combination of digital signal processing techniques, machine learning models trained on large audio corpora, and statistical analyses of acoustic patterns extracted from recordings (Askin & Mauskapf, 2017; Kim & Askin, 2024). Over the past decade, they have become one of the most widely used feature sets in Music Information Retrieval research and commercial recommendation systems. Their continued use across both academic and industry settings reflects their demonstrated ability to capture musically meaningful variation and support tasks such as genre classification, similarity estimation, playlist generation, and preference prediction.

An additional advantage of these features is their accessibility. Raw audio files and spectrogram representations contain substantially richer information, but their use requires large storage capacity, extensive pre-processing pipelines, and considerably greater computational resources. By contrast, Spotify's audio features are readily available, already structured for statistical analysis, and can be incorporated directly into conventional machine learning workflows. This makes large scale analyses feasible without the computational demands associated with deep learning approaches based on raw audio inputs.

At the same time, the suitability of these features for identifying production aesthetics remains an empirical question. The variables were designed to summarize general musical characteristics rather than to detect distinctions between alternative and maximalist production styles. Many of the differences that define production aesthetic emerge through recording, mixing, mastering, and arrangement decisions that may not be fully captured by summary level audio descriptors. It is therefore possible that representations derived directly from raw audio signals or spectrograms would provide stronger information for this classification task.

The present study thus treats Spotify's audio features as a theoretically plausible and widely validated starting point rather than a definitive representation of production style. Model performance therefore serves not only as an evaluation of the classifiers themselves but also as an indirect assessment of how much information about production aesthetic is contained within this commonly used feature set. Poor performance would suggest that the distinction is not strongly encoded in these variables, whereas strong performance would indicate that production aesthetic leaves detectable traces in the audio characteristics captured by Spotify's analysis framework.

3.2 Production Credits and Apple Music Scraping

The aesthetic labels used in this study required song-level production credits, specifically the identities of the producers associated with each track. Production credits were collected from Apple Music, which provides detailed metadata on producers, engineers, and mixers at the individual track level. Because these personnel are directly involved in recording, mixing, and mastering decisions, their credits provide the basis for identifying production aesthetics associated with alternative and maximalist styles (Katz, 2015; Milner, 2010). All songs in the Spotify dataset were successfully matched to corresponding Apple Music entries, allowing production credits to be linked to every track.

Several challenges arose during data collection. Differences in artist and track name formatting across platforms required fuzzy string matching to align records. Some tracks lacked complete production credits because the information was unavailable in Apple Music metadata or because the recordings predated contemporary digital credit practices. Large-scale web-scraping also encountered rate limits and occasional access interruptions, requiring manual verification for a small number of cases.

These issues introduce a limited amount of label uncertainty into the dataset. Missing or incomplete credits are concentrated among older releases and less commercially prominent recordings, where metadata quality is generally lower. Such recordings are also more likely to occupy ambiguous positions between alternative and maximalist production styles. As a result, classification difficulty and label uncertainty are not entirely independent. Any resulting errors are therefore more likely to occur among observations that are substantively difficult to classify rather than among clear examples of either aesthetic category.

3.3 Why Binary Labels and the Expert Seed Set

The binary framing of production aesthetic reflects both a historical argument and the structure of the labeling procedure. The maximalist aesthetic is associated with loudness, density, and dynamic compression, while the alternative aesthetic emphasizes dynamic range, spatial separation, and restraint in recording and mixing practices. Although both traditions have earlier roots, they emerged as recognizable and commercially significant production orientations during the late twentieth century. By the 1990s, alternative production had moved beyond its countercultural origins and become an established component of mainstream popular music. The dataset spans recordings from the 1960s onward in order to capture the broader history of popular music production, but the labeling framework reflects the period in which these two aesthetic orientations became sufficiently stable and visible to support systematic classification.

The labels were constructed through a two-stage process combining expert initialization with network analysis. First, an initial set of 30 producers for each aesthetic category was identified through review of music criticism, audio engineering publications, and production scholarship, including sources such as Rolling Stone, Pitchfork, Mix Magazine, and Sound on Sound. Producers were included only when their association with a particular aesthetic was consistently recognized across multiple sources and was not reducible to genre affiliation alone. This produced a seed set of 60 producers and constituted the only stage involving substantive interpretation.

Next, a producer co-credit network was constructed from Apple Music metadata, with connections linking producers who worked on the same track. Louvain community detection was applied to identify densely connected groups of producers by maximizing network modularity. The seed labels anchored the resulting communities and allowed aesthetic assignments to propagate through co-credit relationships to other producers in the network. Each song was then assigned the aesthetic label of its credited producer. The resulting binary classification thus emerged from the structure of the producer network rather than from a predetermined assignment of labels to individual recordings.

3.4 Informative Sampling

The Spotify Tracks Dataset provides a large and systematically constructed collection of commercially released recordings suitable for analyzing production aesthetics at scale. It contains approximately 114,000 songs spanning 114 genres, with roughly 1,000 songs sampled per genre. The number of 114 genres reflects a deliberate trade off between coverage and feasibility, as this level of genre granularity already spans the vast majority of stylistic categories represented within modern commercial music. This design is intended to be comprehensive across musical styles represented on Spotify while maintaining balance across genres and ensuring variation in artists, time periods, and popularity levels. In this sense, the dataset prioritizes structured coverage of the platform's genre space rather than exhaustive enumeration of all possible genre labels. Combined with Spotify's widely used audio features and production credit data collected from Apple Music, the dataset provides the necessary inputs for both feature based modeling and aesthetic label construction.

The dataset is therefore best understood as an informative sample of Spotify's commercially released music catalog. The genre based construction ensures broad representation across stylistic categories within the platform, while the inclusion of a popularity measure reflects real streaming activity at the time of collection. However, this also implies that recordings from commercially prominent genres and widely distributed artists are more likely to be included, since these are more strongly represented within Spotify's catalog and metadata infrastructure. This is consistent with the dataset's focus on commercially released music with well established metadata coverage rather than an attempt to represent all possible forms of recorded sound.

Within this scope, the dataset is well aligned with the substantive focus of the study. The production aesthetic labels are derived from producer communities and critical discourse that emerge primarily within the same commercial music industry structure captured by Spotify's catalog. As a result, the sampling frame and the labeling framework operate within a shared institutional context, which supports the validity of the operationalization for studying alternative and maximalist production within popular music.

The primary limitation concerns external generalizability. The reported performance metrics reflect model behavior within a structured and genre balanced sample of Spotify music rather than a random sample of all recordings globally. Applications to musical traditions, production systems, or regional industries that are less fully represented within Spotify's catalog would require additional validation. Accordingly, the results should be interpreted as evidence for production aesthetic classification within the commercially released popular music domain represented by the dataset.

3.5 Anglo-Centric Limitation and Generalizability

A more fundamental limitation of the dataset concerns its geographic and cultural composition. The genre selection and popularity structure underlying the dataset reflect the dominant distribution patterns of the global streaming economy, which are disproportionately concentrated in Anglo American and broader North American and European music industries. The aesthetic categories used in this study, maximalist and alternative, are also derived from concepts developed within Anglophone popular music criticism and production scholarship. Similarly, the seed producers used to initialize the community detection procedure are drawn primarily from this same tradition. As a result, the classifier captures aesthetic distinctions that are meaningful within Anglophone commercial popular music, but these distinctions may not transfer directly to musical traditions organized around different production practices, evaluative standards, and critical vocabularies.

If applied to datasets with substantially greater representation from non Anglophone music traditions, such as Korean popular music, Afrobeats, Latin urban styles, or regional Indian film music, the underlying feature distributions and production conventions may differ in ways that are not well aligned with the labeling structure used here. In such contexts, the model would still produce predictions, but those outputs would reflect the application of an Anglophone derived aesthetic framework rather than a locally grounded set of production categories. The resulting classifications should therefore be interpreted with caution outside of mainstream Anglophone popular music.

Accordingly, any extension of this classifier beyond the domain represented in the training data should involve additional validation against domain specific critical, scholarly, or industry based sources in order to ensure that the inferred aesthetic distinctions remain meaningful within the relevant musical context.

4 Data Loading and Cleaning

Hide

```
df_raw <- read_csv("labeled_dataset.csv")
head(df_raw)
```

```
## # A tibble: 6 × 22
##    ...1 track_id   artists album_name track_name popularity duration_ms explicit
##   <dbl> <chr>      <chr>   <chr>      <chr>           <dbl>       <dbl> <lgl>
## 1     0 5SuOikwiR… Gen Ho… Comedy     Comedy             73      230666 FALSE
## 2     1 4qPNDBW1i… Ben Wo… Ghost (Ac… Ghost - A…         55      149610 FALSE
## 3     2 1iJBSr7s7… Ingrid… To Begin … To Begin …         57      210826 FALSE
## 4     3 6lfxq3CG4… Kina G… Crazy Ric… Can't Hel…         71      201933 FALSE
## 5     4 5vjLSffim… Chord … Hold On    Hold On            82      198853 FALSE
## 6     5 01MVOl9Kt… Tyrone… Days I Wi… Days I Wi…         58      214240 FALSE
## # ℹ 14 more variables: danceability <dbl>, energy <dbl>, key <dbl>,
## #   loudness <dbl>, mode <dbl>, speechiness <dbl>, acousticness <dbl>,
## #   instrumentalness <dbl>, liveness <dbl>, valence <dbl>, tempo <dbl>,
## #   time_signature <dbl>, track_genre <chr>, production_aesthetic <chr>
```

Hide

```
summary(df_raw)
```

```
##       ...1          track_id           artists           album_name
##  Min.   :     0   Length:114000      Length:114000      Length:114000
##  1st Qu.: 28500   Class :character   Class :character   Class :character
##  Median : 57000   Mode  :character   Mode  :character   Mode  :character
##  Mean   : 57000
##  3rd Qu.: 85499
##  Max.   :113999
##   track_name          popularity      duration_ms       explicit
##  Length:114000      Min.   :  0.00   Min.   :      0   Mode :logical
##  Class :character   1st Qu.: 17.00   1st Qu.: 174066   FALSE:104253
##  Mode  :character   Median : 35.00   Median : 212906   TRUE :9747
##                     Mean   : 33.24   Mean   : 228029
##                     3rd Qu.: 50.00   3rd Qu.: 261506
##                     Max.   :100.00   Max.   :5237295
##   danceability        energy             key            loudness
##  Min.   :0.0000   Min.   :0.0000   Min.   : 0.000   Min.   :-49.531
##  1st Qu.:0.4560   1st Qu.:0.4720   1st Qu.: 2.000   1st Qu.:-10.013
##  Median :0.5800   Median :0.6850   Median : 5.000   Median : -7.004
##  Mean   :0.5668   Mean   :0.6414   Mean   : 5.309   Mean   : -8.259
##  3rd Qu.:0.6950   3rd Qu.:0.8540   3rd Qu.: 8.000   3rd Qu.: -5.003
##  Max.   :0.9850   Max.   :1.0000   Max.   :11.000   Max.   :  4.532
##       mode         speechiness       acousticness    instrumentalness
##  Min.   :0.0000   Min.   :0.00000   Min.   :0.0000   Min.   :0.00e+00
##  1st Qu.:0.0000   1st Qu.:0.03590   1st Qu.:0.0169   1st Qu.:0.00e+00
##  Median :1.0000   Median :0.04890   Median :0.1690   Median :4.16e-05
##  Mean   :0.6376   Mean   :0.08465   Mean   :0.3149   Mean   :1.56e-01
##  3rd Qu.:1.0000   3rd Qu.:0.08450   3rd Qu.:0.5980   3rd Qu.:4.90e-02
##  Max.   :1.0000   Max.   :0.96500   Max.   :0.9960   Max.   :1.00e+00
##     liveness         valence           tempo        time_signature
##  Min.   :0.0000   Min.   :0.0000   Min.   :  0.00   Min.   :0.000
##  1st Qu.:0.0980   1st Qu.:0.2600   1st Qu.: 99.22   1st Qu.:4.000
##  Median :0.1320   Median :0.4640   Median :122.02   Median :4.000
##  Mean   :0.2136   Mean   :0.4741   Mean   :122.15   Mean   :3.904
##  3rd Qu.:0.2730   3rd Qu.:0.6830   3rd Qu.:140.07   3rd Qu.:4.000
##  Max.   :1.0000   Max.   :0.9950   Max.   :243.37   Max.   :5.000
##  track_genre        production_aesthetic
##  Length:114000      Length:114000
##  Class :character   Class :character
##  Mode  :character   Mode  :character
##
##
##
```

Hide

```
cat("Raw dataset dimensions:", nrow(df_raw), "rows,", ncol(df_raw), "columns\n")
```

```
## Raw dataset dimensions: 114000 rows, 22 columns
```

Hide

```
cat("\nColumn names:\n")
```

```
##
## Column names:
```

Hide

```
print(names(df_raw))
```

```
##  [1] "...1"                 "track_id"             "artists"
##  [4] "album_name"           "track_name"           "popularity"
##  [7] "duration_ms"          "explicit"             "danceability"
## [10] "energy"               "key"                  "loudness"
## [13] "mode"                 "speechiness"          "acousticness"
## [16] "instrumentalness"     "liveness"             "valence"
## [19] "tempo"                "time_signature"       "track_genre"
## [22] "production_aesthetic"
```

Hide

```
cat("\nLabel distribution (raw):\n")
```

```
##
## Label distribution (raw):
```

Hide

```
print(table(df_raw$production_aesthetic))
```

```
##
##     A     M
## 57071 56929
```

Hide

```
df <- df_raw %>%
  # Drop identifier, metadata, and excluded columns
  select(-any_of(c(
    "...1", "track_id", "artists", "album_name",
    "track_name", "explicit", "popularity", "track_genre"
  ))) %>%
  # Drop loudness per study design: including it would be circular
  # because labels were derived from producer communities associated
  # with loudness practices
  select(-loudness) %>%
  drop_na()

# Remap numeric codes to substantive labels throughout
# so every downstream section sees meaningful names
df <- df %>%
  mutate(
    # Outcome
    production_aesthetic = factor(
      production_aesthetic,
      levels = c("A", "M"),
      labels = c("Alternative", "Maximalist")
    ),
    # Pitch class notation (standard musical keys)
    key = recode(as.character(key),
      "0"  = "C",  "1"  = "C#", "2"  = "D",  "3"  = "D#",
      "4"  = "E",  "5"  = "F",  "6"  = "F#", "7"  = "G",
      "8"  = "G#", "9"  = "A",  "10" = "A#", "11" = "B"
    ),
    # Mode
    mode = recode(as.character(mode),
      "0" = "Minor",
      "1" = "Major"
    ),
    # Time signature
    time_signature = recode(as.character(time_signature),
      "1" = "1/4", "2" = "2/4", "3" = "3/4",
      "4" = "4/4", "5" = "5/4"
    )
  )

cat("After dropping missing values and excluded columns:\n")
```

```
## After dropping missing values and excluded columns:
```

Hide

```
cat("Dimensions:", nrow(df), "rows,", ncol(df), "columns\n")
```

```
## Dimensions: 114000 rows, 13 columns
```

Hide

```
cat("Remaining columns:", paste(names(df), collapse = ", "), "\n")
```

```
## Remaining columns: duration_ms, danceability, energy, key, mode, speechiness, acousticness, i
nstrumentalness, liveness, valence, tempo, time_signature, production_aesthetic
```

Hide

```
# IQR-based outlier removal for duration_ms and tempo only.
# These two features have well-documented extreme values from silence
# tracks and spoken-word recordings that would distort the feature space.
# The remaining continuous features are already bounded in [0,1] by
# Spotify's normalization and require no trimming.

remove_outliers_iqr <- function(data, col) {
  q1  <- quantile(data[[col]], 0.25)
  q3  <- quantile(data[[col]], 0.75)
  iqr <- q3 - q1
  data %>% filter(
    .data[[col]] >= q1 - 1.5 * iqr,
    .data[[col]] <= q3 + 1.5 * iqr
  )
}

n_before <- nrow(df)
df <- remove_outliers_iqr(df, "duration_ms")
cat("Removed", n_before - nrow(df), "rows for duration_ms outliers\n")
```

```
## Removed 5617 rows for duration_ms outliers
```

Hide

```
n_before <- nrow(df)
df <- remove_outliers_iqr(df, "tempo")
cat("Removed", n_before - nrow(df), "rows for tempo outliers\n")
```

```
## Removed 509 rows for tempo outliers
```

Hide

```
cat("\nFinal dataset:", nrow(df), "rows\n")
```

```
##
## Final dataset: 107874 rows
```

Hide

```
cat("\nFinal class distribution:\n")
```

```
##
## Final class distribution:
```

Hide

```
print(table(df$production_aesthetic))
```

```
##
## Alternative  Maximalist
##       52722       55152
```

Hide

```
cat("\nClass proportions:\n")
```

```
##
## Class proportions:
```

Hide

```
print(prop.table(table(df$production_aesthetic)))
```

```
##
## Alternative  Maximalist
##   0.4887369   0.5112631
```

5 Exploratory Data Analysis

Hide

```
df %>%
  count(production_aesthetic) %>%
  mutate(pct = n / sum(n)) %>%
  ggplot(aes(x = production_aesthetic, y = n,
             fill = production_aesthetic)) +
  geom_col(width = 0.5, color = "black", alpha = 0.85) +
  geom_text(
    aes(label = paste0(scales::comma(n), "\n(",
                       scales::percent(pct, accuracy = 0.1), ")")),
    vjust = -0.3, size = 4
  ) +
  scale_fill_manual(values = c("steelblue", "coral")) +
  scale_y_continuous(
    labels = scales::comma,
    expand = expansion(mult = c(0, 0.12))
  ) +
  labs(
    title = "Distribution of Production Aesthetic Labels",
    x     = "Production Aesthetic",
    y     = "Number of Songs"
  ) +
  theme_bw(base_size = 13) +
  theme(legend.position = "none")
```

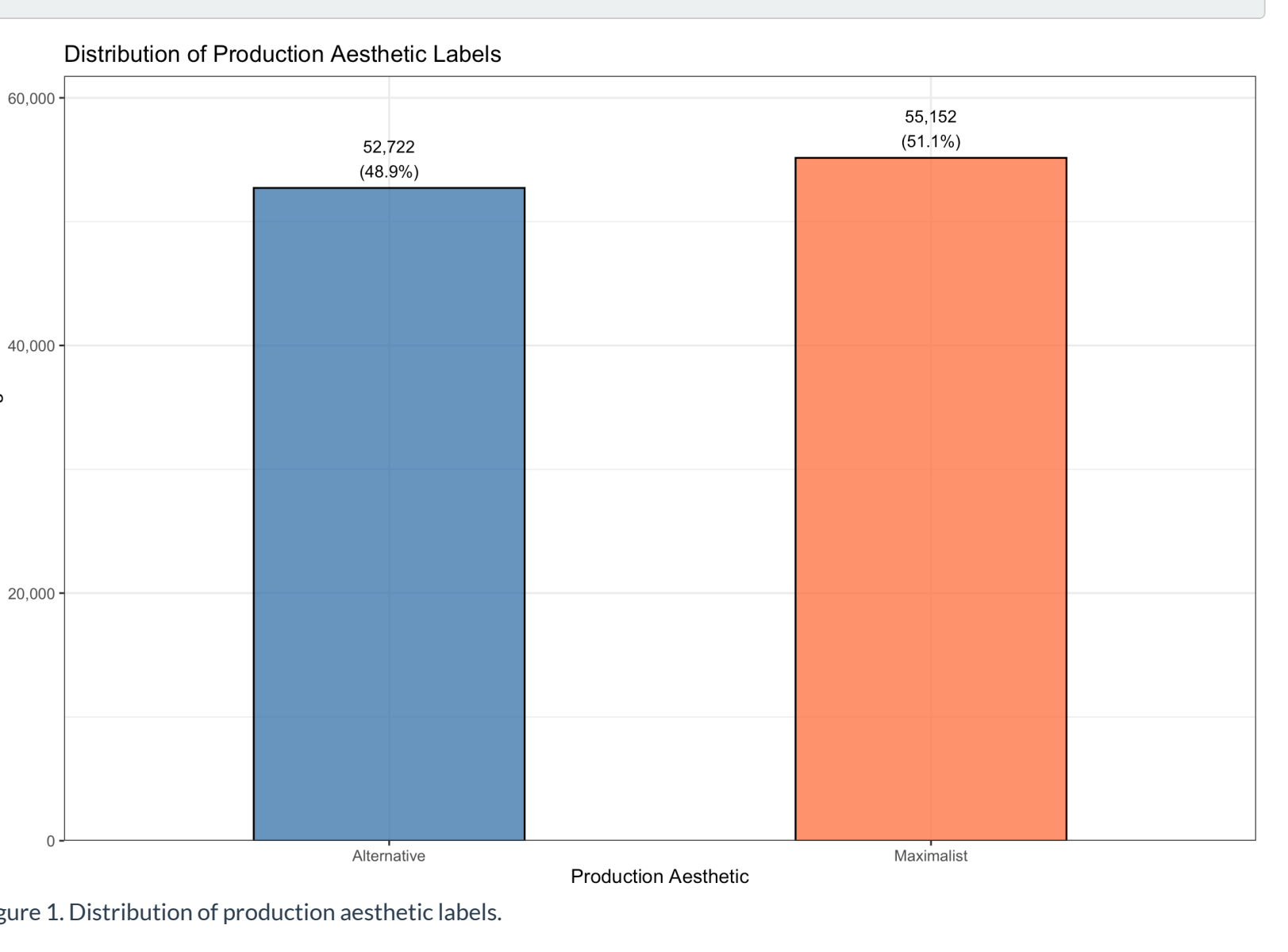


Figure 1. Distribution of production aesthetic labels.

The dataset is nearly perfectly balanced between the two aesthetic categories. This means accuracy is an interpretable metric, no resampling strategy is required, and the default 0.5 probability threshold for binary classification is well-calibrated with respect to class prevalence.

The dataset is nearly balanced across the two aesthetic categories, with 52,722 alternative songs (48.9 percent) and 55,152 maximalist songs (51.1 percent). The small difference between classes is not substantively meaningful for model estimation and does not indicate systematic class imbalance. As a result, standard evaluation metrics such as accuracy can be interpreted directly without adjustment, and no resampling procedures such as SMOTE or class weighting are required to correct for class imbalance. The default decision threshold of 0.5 is also appropriate given the close alignment between class proportions.

This balance is a structural consequence of the labeling procedure rather than an imposed constraint. The Louvain community detection algorithm partitions the producer co-credit network into two communities that are similar in size under the given initialization, and this partition propagates through producer level assignments to the song level labels. The resulting distribution therefore reflects the organization of the underlying producer network rather than an externally enforced balancing rule.

Hide

```
df %>%
  mutate(label_numeric = if_else(
    production_aesthetic == "Maximalist", 1, 0)
  ) %>%
  select(where(is.numeric)) %>%
  cor() %>%
  as.data.frame() %>%
  rownames_to_column("feature") %>%
  select(feature, label_numeric) %>%
  filter(feature != "label_numeric") %>%
  arrange(desc(abs(label_numeric))) %>%
  mutate(
    direction = if_else(label_numeric >= 0, "Maximalist", "Alternative"),
    feature   = fct_reorder(feature, label_numeric)
  ) %>%
  ggplot(aes(x = feature, y = label_numeric, fill = direction)) +
  geom_col(color = "black", alpha = 0.85) +
  geom_hline(yintercept = 0, linewidth = 0.5) +
  scale_fill_manual(values = c("steelblue", "coral")) +
  coord_flip() +
  labs(
    title = "Pearson Correlation of Audio Features with Production Aesthetic",
    x     = "Feature",
    y     = "Correlation with Maximalist Label",
    fill  = "Direction"
  ) +
  theme_bw(base_size = 13)
```

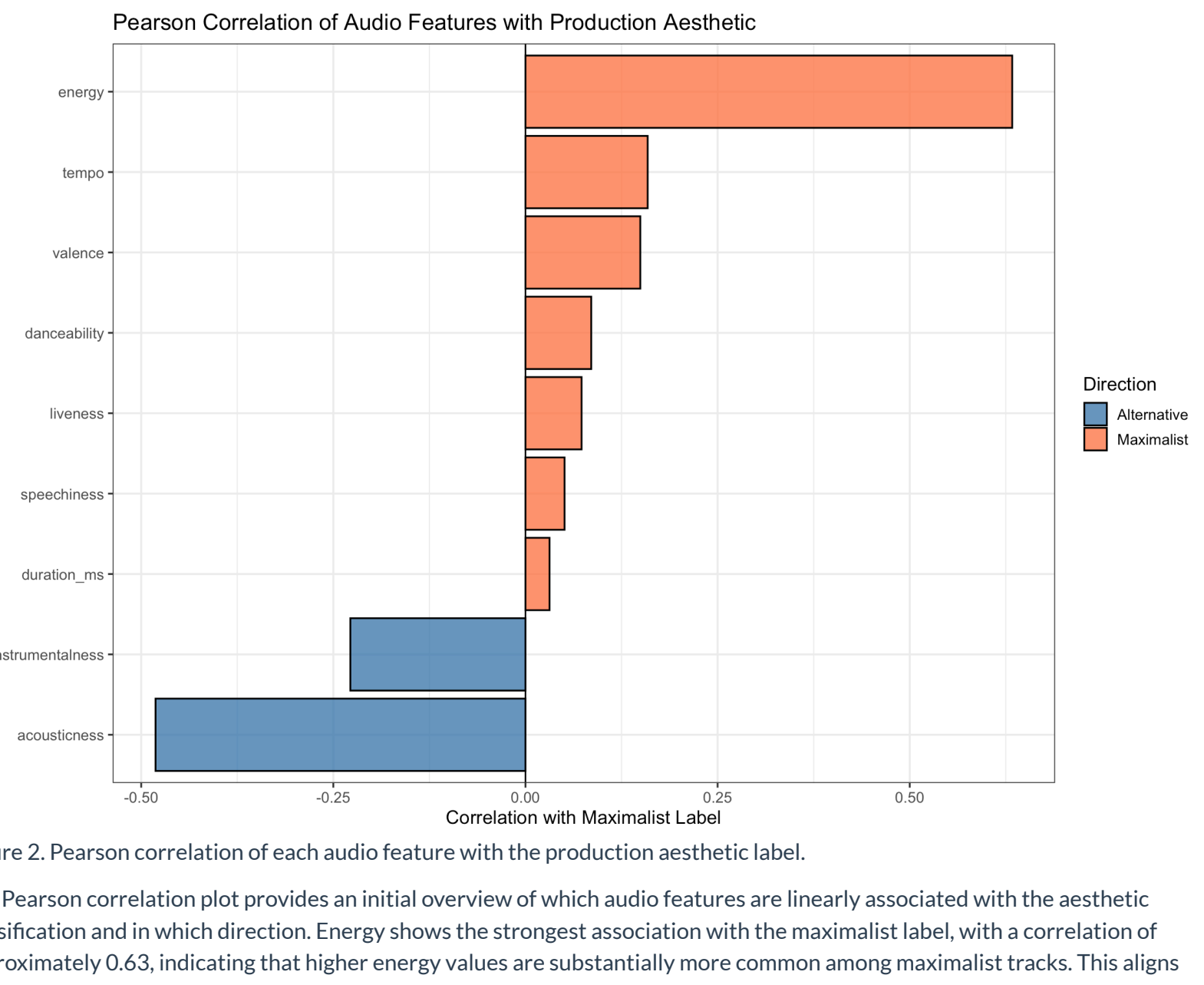


Figure 2. Pearson correlation of each audio feature with the production aesthetic label.

The Pearson correlation plot provides an initial overview of which audio features are linearly associated with the aesthetic classification and in which direction. Energy shows the strongest association with the maximalist label, with a correlation of approximately 0.63, indicating that higher energy values are substantially more common among maximalist tracks. This aligns with the characterization of maximalist production as emphasizing perceptual intensity and loudness. Acousticness is the strongest predictor in the alternative direction, with a correlation of approximately negative 0.50, indicating that alternative production is more closely associated with acoustic or less electronically processed recordings. Instrumentalness also shows a notable negative association with maximalism, suggesting that vocal presence is more common in maximalist contexts, while instrumental textures are relatively more characteristic of alternative production.

Tempo and valence exhibit moderate positive correlations with maximalism, consistent with the tendency for mainstream commercial production to favor higher tempo and more positively valenced tracks. Danceability, liveness, speechiness, and duration show weaker associations, indicating that they contribute less linear signal for distinguishing between the two aesthetic categories, even though they may still carry information in combination with other variables.

These results motivate the first of the two hypotheses that are examined in the subsequent modeling section.

H1: A small subset of features, particularly energy and acousticness, is expected to account for most of the predictive signal in distinguishing production aesthetics.

Hide

```
df %>%
  select(duration_ms, danceability, energy, speechiness,
         acousticness, instrumentalness, liveness, valence,
         tempo, production_aesthetic) %>%
  pivot_longer(
    cols      = -production_aesthetic,
    names_to  = "feature", values_to = "value"
  ) %>%
  mutate(feature = str_to_title(
    str_replace_all(feature, "_", " ")
  )) %>%
  ggplot(aes(x = production_aesthetic, y = value,
             fill = production_aesthetic)) +
  geom_violin(alpha = 0.5, linewidth = 0.5) +
  geom_boxplot(width = 0.15, alpha = 0.8,
               outlier.size = 0.3,
               outlier.alpha = 0.2) +
  facet_wrap(~feature, scales = "free_y", ncol = 3) +
  scale_fill_manual(values = c("steelblue", "coral")) +
  labs(
    title = "Distribution of Audio Features by Production Aesthetic",
    x     = NULL,
    y     = "Feature Value",
    fill  = "Production Aesthetic"
  ) +
  theme_bw(base_size = 12) +
  theme(
    legend.position = "bottom",
    strip.text      = element_text(size = 10, face = "bold"),
    axis.text.x     = element_blank(),
    axis.ticks.x    = element_blank()
  )
```

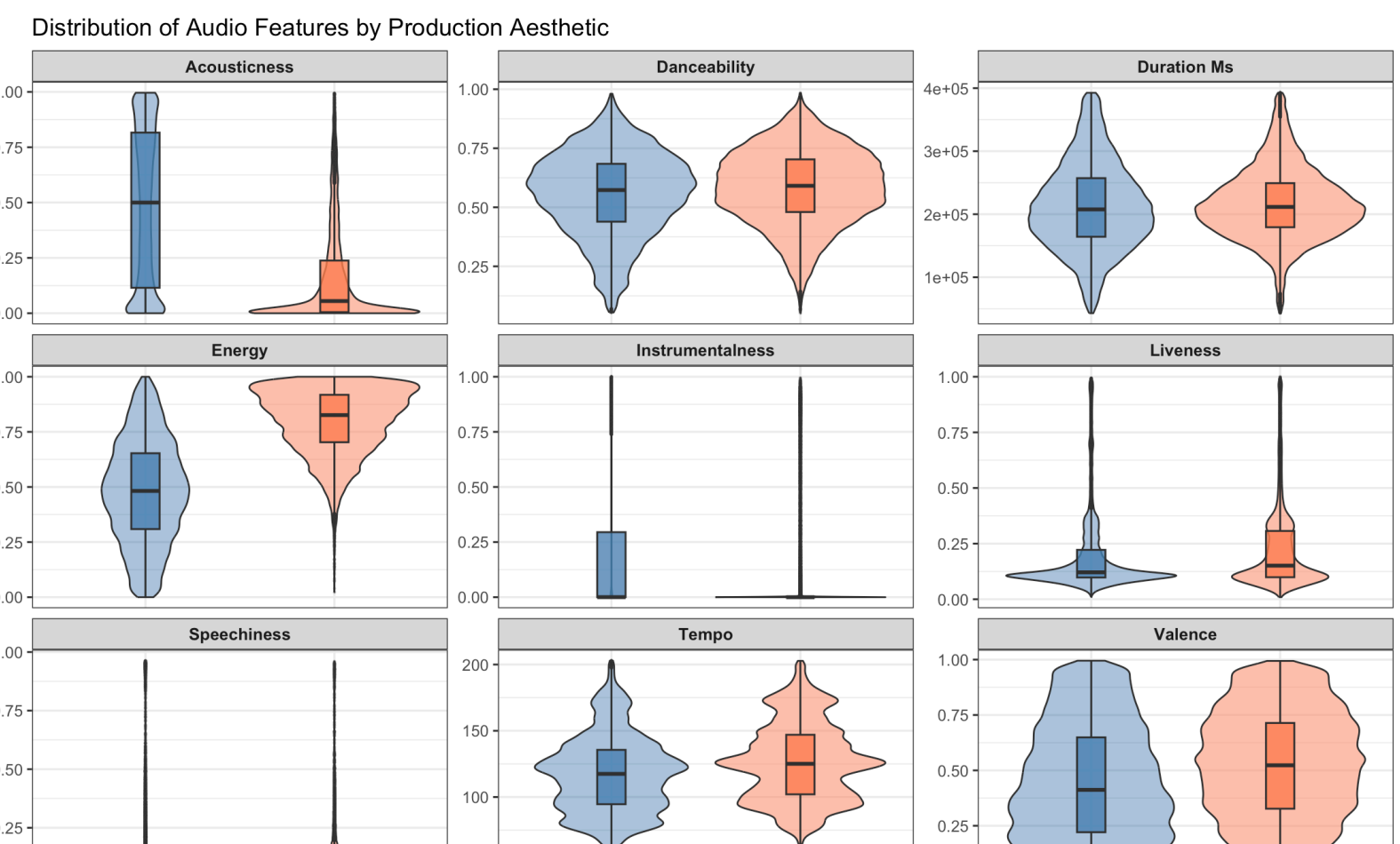


Figure 3. Distributions of key audio features by production aesthetic.

The nine audio features in this dataset are measured on different scales and exhibit substantially different distributional forms. Several variables, including danceability, energy, acousticness, liveness, speechiness, valence, and instrumentalness, are bounded between 0 and 1 as a result of Spotify's normalization procedure. However, their empirical distributions differ markedly. Some are approximately symmetric, while others are heavily skewed. Instrumentalness in particular is concentrated extremely close to zero across both aesthetic categories, reflecting the fact that most commercially released tracks are vocal recordings. As a result, standard kernel density estimation provides limited additional information for this feature since the distribution is dominated by a near point mass at zero.

Tempo is measured in beats per minute and spans a much wider numerical range, while duration is measured in milliseconds and varies by orders of magnitude relative to the bounded features. Given these differences, a uniform visualization approach must accommodate both bounded and unbounded variables without imposing distributional assumptions that obscure structural differences. Violin plots combined with embedded boxplots are used for this purpose. The violin component summarizes the empirical distribution without relying on parametric density estimation, while the boxplot provides a consistent reference for median and interquartile range. Each feature is displayed on a separate scale to preserve interpretability across heterogeneous measurement units.

The distributional plots provide a more detailed view of aesthetic separation than correlation coefficients alone. The acousticness distribution shows the clearest contrast between categories. Alternative tracks are distributed broadly across the range with substantial mass at higher values, whereas maximalist tracks are heavily concentrated near zero. The energy distribution shows the inverse pattern, with maximalist tracks concentrated toward higher values and alternative tracks more dispersed at lower levels. Together, these patterns support the hypothesis that energy and acousticness are the most diagnostically informative features for distinguishing the two aesthetics, and that even simple threshold rules on either variable would achieve meaningful separation.

Danceability and valence show partial but less distinct separation, with both categories occupying overlapping mid-range distributions and maximalist tracks exhibiting only a modest upward shift. Speechiness and liveness are highly concentrated near-zero for both categories, with differences emerging primarily in the upper tails rather than the bulk of the distributions. Instrumentalness confirms the earlier observation that its predictive contribution is driven by a small number of non vocal recordings that are disproportionately associated with the alternative category.

Tempo exhibits substantial overlap between categories, with only a slight upward shift in the maximalist distribution. Duration shows nearly identical distributions across both aesthetics, indicating minimal discriminatory power in isolation. Overall, the violin plots suggest that while several features operate primarily through shifts in central tendency, others reflect differences in distributional shape or tail behavior, particularly acousticness and instrumentalness.

These patterns provide support for the hypothesis that the aesthetic boundary is not strictly linear in the feature space. While the correlation analysis provides evidence of linear association, it does not capture such multivariate structure, which is assessed in the modeling and distributional analyses that follow. In several cases, separation arises from differences in distributional structure rather than simple mean differences. This suggests that models capable of capturing threshold effects and feature interactions, such as tree-based methods, may better represent the underlying decision boundary than purely linear approaches. This motivates the inclusion of such models in the subsequent analysis and informs the comparison of model performance reported in later sections.

H2: The relationship between audio features and aesthetic category may be non-linear. Tree-based models can perform better than linear models in this case.

---

6 Feature Engineering and Data Splitting

```
# One-hot encode key, mode, and time_signature.
# These are nominal categorical variables. Treating them as numeric
# would impose a false ordinal structure (e.g., implying that key G#
# is "between" key G and key A in a meaningful sense).
# Min-max scaling of all numeric predictors to [0,1] ensures that
# tempo and duration_ms, which have large absolute values, do not
# dominate regularization-based or distance-based models purely
# because of their scale.

df_encoded <- df %>%
  mutate(
    key            = factor(key),
    time_signature = factor(time_signature),
    mode           = factor(mode)
  ) %>%
  recipe(production_aesthetic ~ ., data = .) %>%
  step_dummy(key, time_signature, mode, one_hot = TRUE) %>%
  step_range(all_numeric_predictors(), min = 0, max = 1) %>%
  prep() %>%
  bake(new_data = NULL)

cat("Feature matrix after encoding:\n")
```

```
## Feature matrix after encoding:
```

```
cat(nrow(df_encoded), "rows,", ncol(df_encoded) - 1, "features\n")
```

```
## 107874 rows, 28 features
```

```
cat("\nFeature names:\n")
```

```
##
## Feature names:
```

```
print(names(df_encoded %>% select(-production_aesthetic)))
```

```
##  [1] "duration_ms"         "danceability"        "energy"
##  [4] "speechiness"         "acousticness"        "instrumentalness"
##  [7] "liveness"            "valence"             "tempo"
## [10] "key_A"               "key_A."              "key_B"
## [13] "key_C"               "key_C."              "key_D"
## [16] "key_D."              "key_E"               "key_F"
## [19] "key_F."              "key_G"               "key_G."
## [22] "time_signature_X0"   "time_signature_X1.4" "time_signature_X3.4"
## [25] "time_signature_X4.4" "time_signature_X5.4" "mode_Major"
## [28] "mode_Minor"
```

```
# 80/20 stratified train/test split.
# Stratification preserves the near-perfect class balance in both sets.
set.seed(2026)

data_split <- initial_split(df_encoded, prop = 0.8,
                            strata = production_aesthetic)
train_data <- training(data_split)
test_data  <- testing(data_split)

# 5-fold cross-validation for hyperparameter tuning.
# With approximately 91,000 training observations, each fold contains
# roughly 18,000 observations, 5 is sufficient for stable AUC estimates.
# 5-fold is also chosen over 10-fold to reduce computational cost by
# approximately half with negligible impact on estimate stability
# at this sample size.
cv_folds <- vfold_cv(train_data, v = 5, strata = production_aesthetic)

cat("Training set:", nrow(train_data), "rows\n")
```

```
## Training set: 86298 rows
```

```
cat("Test set    :", nrow(test_data),  "rows\n")
```

```
## Test set    : 21576 rows
```

```
cat("CV folds    :", length(cv_folds$id), "\n")
```

```
## CV folds    : 5
```

```
# Define outcome and feature vectors
outcome  <- "production_aesthetic"
features <- setdiff(names(df_encoded), outcome)

# Numeric 0/1 outcome vectors required by SuperLearner
train_y <- as.numeric(train_data$production_aesthetic == "Maximalist")
test_y  <- as.numeric(test_data$production_aesthetic  == "Maximalist")
train_x <- train_data %>% select(all_of(features))
test_x  <- test_data  %>% select(all_of(features))

cat("\nNumber of features:", length(features), "\n")
```

```
##
## Number of features: 28
```

```
cat("Training outcome distribution:\n")
```

```
## Training outcome distribution:
```

```
print(table(train_data$production_aesthetic))
```

```
##
## Alternative  Maximalist
##       42177       44121
```

```
cat("Test outcome distribution:\n")
```

```
## Test outcome distribution:
```

```
print(table(test_data$production_aesthetic))
```

```
##
## Alternative  Maximalist
##       10545       11031
```

---

7 Model Development and Hyperparameter Tuning

The six models are fit in order of increasing complexity, with each successive model introduced to address limitations of the previous one and to test additional structure in the relationship between audio features and aesthetic labels. This ordering allows the modeling pipeline to move from highly constrained functional forms toward more flexible approaches that can capture interactions and non-linear decision boundaries.

Hyperparameter tuning is conducted within the training set using five-fold cross-validation, with ROC-AUC used as the selection criterion. ROC-AUC is independent of any fixed classification threshold and evaluates the model's ability to rank positive observations above negative ones across all possible thresholds. This makes it appropriate for tuning because the optimal decision threshold may vary depending on the intended application of the classifier and is not fixed at the modeling stage.

Final performance evaluation on the held-out test set is reserved for Section 8 to ensure that model selection decisions are fully separated from out-of-sample assessment.

7.0 Loss Function Helper

```
# Loss function helper
# Negative log-likelihood (log-loss) for binary classification.
# For a single observation: -(y * log(p) + (1-y) * log(1-p))
# Averaged across n observations:
#   L = -(1/n) * sum[ y_i * log(p_i) + (1 - y_i) * log(1 - p_i) ]
# A small epsilon (1e-15) is added to prevent log(0).
# Lower values indicate better calibrated probability predictions.

log_loss <- function(y_true, y_prob) {
  eps <- 1e-15
  y_prob <- pmax(pmin(y_prob, 1 - eps), eps)
  -mean(y_true * log(y_prob) + (1 - y_true) * log(1 - y_prob))
}

# Gini impurity for a node with class proportions p_1 ... p_K:
#   Gini = 1 - sum(p_k^2)
# For binary classification this simplifies to:
#   Gini = 1 - (p^2 + (1-p)^2) = 2 * p * (1 - p)
# Used as the split criterion in decision trees and random forest.
# Lower values indicate purer nodes (better separation).

gini_impurity <- function(p) {
  2 * p * (1 - p)
}
```

Before fitting any model, two loss functions are defined explicitly as helper functions that are computed and reported consistently across the modeling pipeline. This makes the optimization objective of each method transparent and ensures that performance comparisons are not based on model specific default summaries that may differ across software implementations. It also allows training behavior to be evaluated on a common scale, making it possible to compare how effectively different models reduce the same objective function.

The first helper function computes negative log-likelihood, also referred to as log-loss or binary cross-entropy. For a single observation, the loss equals the negative logarithm of the probability assigned to the true class. If the true label is maximalist, the loss is negative log of the predicted probability of maximalist. If the true label is alternative, the loss is negative log of one minus that probability. Averaging across all observations yields the overall log loss. The measure is bounded below by zero, approaches zero as predictions become both confident and correct, and increases without bound as predictions become confidently incorrect. A small epsilon value of 1e-15 is added inside the logarithm to avoid numerical instability when predicted probabilities reach exactly zero or one. This loss function corresponds to the training objective used in logistic regression, lasso logistic regression, XGBoost with a logistic objective, and the Super Learner meta model, making it a natural measure of optimization fit across these methods.

The second helper function computes Gini impurity, which is the criterion used for splitting in decision trees and random forests. At each candidate split, the algorithm evaluates all possible feature threshold combinations and selects the split that maximizes the reduction in weighted Gini impurity across the resulting nodes. In the binary case, Gini impurity at a node is defined as two times p times one minus p, where p is the proportion of maximalist observations in that node. This quantity reaches its maximum when classes are evenly mixed and equals zero when a node is perfectly pure. The splitting process therefore seeks partitions that progressively reduce impurity by increasing class homogeneity within nodes.

The root node impurity provides a useful baseline for interpreting tree-based models. It reflects the initial heterogeneity of the full dataset before any splits are applied. Comparing this value to the impurity of deeper nodes and to the final predictive performance helps quantify how much structure the tree has extracted from the feature space. A model that achieves strong predictive performance while substantially reducing impurity from the root node has successfully learned meaningful decision boundaries rather than relying on trivial class separation.

7.1 Logistic Regression

Model Introduction:

Logistic regression estimates a single linear decision boundary in the feature space. It provides a baseline assessment of whether the aesthetic distinction is approximately linearly separable using audio features alone. Its performance establishes a reference point for the rest of the modeling pipeline, since more complex models must demonstrate meaningful improvement over this linear benchmark in order to justify additional flexibility.

The model coefficients are directly interpretable as additive contributions to the log-odds of the maximalist class, holding all other features constant. This allows for an initial assessment of which audio features carry linear signal for distinguishing between the two aesthetic categories and how that signal is distributed across the feature space.

```
lr_spec <- logistic_reg(penalty = 0, mixture = 0) %>%
  set_engine("glm") %>%
  set_mode("classification")

lr_fit <- workflow() %>%
  add_formula(production_aesthetic ~ .) %>%
  add_model(lr_spec) %>%
  fit(data = train_data)

# Top coefficients as first read on linear feature signal
lr_coefs <- lr_fit %>%
  extract_fit_parsnip() %>%
  tidy() %>%
  filter(term != "(Intercept)") %>%
  arrange(desc(abs(estimate)))

cat("Top 10 logistic regression coefficients (by absolute magnitude):\n")
```

```
## Top 10 logistic regression coefficients (by absolute magnitude):
```

```
print(head(lr_coefs, 10))
```

```
## # A tibble: 10 × 5
##    term                estimate std.error statistic   p.value
##    <chr>                  <dbl>     <dbl>     <dbl>     <dbl>
##  1 energy                 9.35     0.0775    121.    0
##  2 time_signature_X0      4.05     4.64        0.872 3.83e-  1
##  3 instrumentalness      -2.77     0.0416    -66.5   0
##  4 speechiness           -1.61     0.0880    -18.3   6.02e- 75
##  5 valence               -1.20     0.0454    -26.5   3.56e-155
##  6 danceability           1.18     0.0667     17.7   3.65e- 70
##  7 duration_ms           -0.883    0.0547    -16.1   1.24e- 58
##  8 liveness              -0.822    0.0509    -16.1   1.41e- 58
##  9 time_signature_X1.4   -0.512    0.153      -3.35  8.12e-  4
## 10 tempo                  0.298    0.0560      5.33  9.79e-  8
```

```
# Loss function: negative log-likelihood
# L = -(1/n) * sum[ y_i * log(p_i) + (1 - y_i) * log(1 - p_i) ]
# Logistic regression minimizes this directly via IRLS during fitting.

lr_train_prob <- lr_fit %>%
  predict(train_data, type = "prob") %>%
  pull(.pred_Maximalist)

lr_train_y <- as.numeric(train_data$production_aesthetic == "Maximalist")

lr_logloss <- log_loss(lr_train_y, lr_train_prob)
cat("Logistic Regression — training log-loss:", round(lr_logloss, 4), "\n")
```

```
## Logistic Regression — training log-loss: 0.4085
```

Hyperparamter Tuning:

Logistic regression as implemented here has no hyperparameters requiring tuning. The specification sets penalty to zero and mixture to zero, meaning that no regularization is applied and all 28 features after one-hot encoding enter the model with unconstrained coefficients estimated via maximum likelihood using iteratively reweighted least squares. This is a deliberate modeling choice rather than an omission. The purpose of the unregularized model is to establish the linear ceiling, defined as the best achievable performance under a fully flexible linear combination of all available features.

Introducing regularization at this stage would conflate two separate questions, namely whether the aesthetic boundary is approximately linearly separable and which subset of features is strictly necessary for linear prediction. These questions are addressed separately by logistic regression and Lasso respectively, allowing each model to isolate a distinct aspect of the structure in the data.

Because no tuning is required, logistic regression is fit once on the full training set without cross validation. This makes it the fastest model in the pipeline and the most direct benchmark for evaluating whether the additional flexibility introduced by subsequent models yields meaningful improvements over an unrestricted linear baseline.

Interpretations of the results:

Logistic regression is the appropriate starting model for this classification problem for two reasons. First, it directly estimates the probability that a song belongs to the maximalist category as a linear function of the audio features, which is the quantity required both for binary classification and for downstream ranking applications that rely on probabilistic scores. Second, the model is fully interpretable, since its coefficients correspond to additive effects on the log odds scale. This makes it possible to obtain an initial, transparent description of how each audio feature contributes to the aesthetic distinction and in which direction, establishing a baseline against which all subsequent models are evaluated.

The loss function minimized during training is the negative log likelihood averaged over all observations. This objective penalizes confident misclassifications more heavily than uncertain ones, meaning that predictions that are both wrong and overconfident incur large losses, while less confident errors are penalized more mildly. As a result, the model is encouraged to produce well calibrated probability estimates rather than only correct binary labels. This is particularly relevant for downstream applications where predicted probabilities may be used for ranking or recommendation rather than hard classification. The training log loss of 0.4085 serves as the baseline performance level. Improvements in more flexible models are meaningful only if they reduce this loss without degrading out of sample performance, while increases in training performance accompanied by poorer test performance indicate overfitting rather than genuine signal extraction.

The coefficient estimates provide direct evidence of the linear structure underlying the aesthetic boundary. Energy has the largest positive coefficient at 9.35, indicating that higher energy values are strongly associated with the maximalist category when holding all other features constant. This aligns with its role as the dominant linear predictor identified in the exploratory analysis. Instrumentalness shows a strong negative coefficient of -2.77, followed by speechiness at -1.61, valence at -1.20, and danceability at 1.18 in the positive direction. Duration and liveness also have negative coefficients, suggesting that longer and more live sounding recordings are more strongly associated with the alternative aesthetic. Tempo has a smaller positive coefficient of 0.298, consistent with its weaker but still positive association with maximalist aesthetic.

Two results are particularly notable. First, acousticness does not appear among the largest coefficients despite its strong marginal correlation with the label in the exploratory analysis. This is consistent with multicollinearity, since acousticness and energy are strongly negatively correlated and the model distributes shared signal primarily through energy when both are included. Second, the indicator for time signature category zero has a large estimated coefficient of 4.05 but a high standard error of 4.64 and a p-value of 3.83e-1, indicating substantial estimation uncertainty due to sparsity. This instability suggests that its apparent effect is not reliable and is expected to be regularized toward zero in the lasso specification.

7.2 Lasso Logistic Regression

Model Introduction:

Lasso adds L1 regularization, which shrinks uninformative coefficients to exactly zero. With 30-plus features after one-hot encoding, many are likely redundant — particularly the one-hot encoded key and time signature dummies — and Lasso identifies the sparse subset that carries genuine linear signal. It answers the second question: which features are actually necessary for linear classification, and how many can be discarded without losing predictive performance? The penalty parameter is tuned via 5-fold CV over a log-scale grid of 30 values.

```
lasso_spec <- logistic_reg(penalty = tune(), mixture = 1) %>%
  set_engine("glmnet") %>%
  set_mode("classification")

lasso_grid <- grid_regular(
  penalty(range = c(-4, 0)),
  levels = 30
)

lasso_tune <- workflow() %>%
  add_formula(production_aesthetic ~ .) %>%
  add_model(lasso_spec) %>%
  tune_grid(
    resamples = cv_folds,
    grid      = lasso_grid,
    metrics   = metric_set(roc_auc)
  )

best_lasso <- select_best(lasso_tune, metric = "roc_auc")
cat("Optimal Lasso penalty:", round(best_lasso$penalty, 6), "\n")
```

```
## Optimal Lasso penalty: 1e-04
```

```
lasso_final <- workflow() %>%
  add_formula(production_aesthetic ~ .) %>%
  add_model(lasso_spec) %>%
  finalize_workflow(best_lasso) %>%
  fit(data = train_data)

lasso_coefs <- lasso_final %>%
  extract_fit_parsnip() %>%
  tidy() %>%
  filter(estimate != 0, term != "(Intercept)") %>%
  arrange(desc(abs(estimate)))

cat("\nLasso retained", nrow(lasso_coefs), "of", length(features),
    "features\n")
```

```
##
## Lasso retained 26 of 28 features
```

```
cat("Top 10 retained features:\n")
```

```
## Top 10 retained features:
```

```
print(head(lasso_coefs, 10))
```

```
## # A tibble: 10 × 3
##    term                estimate penalty
##    <chr>                  <dbl>   <dbl>
##  1 energy                 9.26   0.0001
##  2 instrumentalness      -2.73   0.0001
##  3 time_signature_X0      1.96   0.0001
##  4 speechiness           -1.54   0.0001
##  5 valence               -1.15   0.0001
##  6 danceability           1.12   0.0001
##  7 duration_ms           -0.854  0.0001
##  8 liveness              -0.806  0.0001
##  9 tempo                  0.275  0.0001
## 10 time_signature_X1.4   -0.224  0.0001
```

```
# Loss function: L1-penalized negative log-likelihood
# L = -(1/n) * sum[ y_i * log(p_i) + (1 - y_i) * log(1 - p_i) ]
#     + lambda * sum(|beta_j|)
# The penalty term shrinks coefficients toward zero; lambda = 0.0001
# was selected by 5-fold CV to maximize ROC-AUC.
# The CV risk (mean log-loss across folds) at the optimal lambda:

lasso_cv_risk <- lasso_tune %>%
  collect_metrics() %>%
  filter(penalty == best_lasso$penalty) %>%
  pull(mean)

cat("Lasso — optimal lambda:", round(best_lasso$penalty, 6), "\n")
```

```
## Lasso — optimal lambda: 1e-04
```

```
cat("Lasso — CV mean ROC-AUC at optimal lambda:",
    round(lasso_cv_risk, 4), "\n")
```

```
## Lasso — CV mean ROC-AUC at optimal lambda: 0.8923
```

```
lasso_train_prob <- lasso_final %>%
  predict(train_data, type = "prob") %>%
  pull(.pred_Maximalist)

lasso_logloss <- log_loss(lr_train_y, lasso_train_prob)
cat("Lasso — training log-loss:", round(lasso_logloss, 4), "\n")
```

```
## Lasso — training log-loss: 0.4086
```

Hyperparameter Tuning:

Lasso logistic regression introduces a single hyperparameter, the penalty strength lambda, which controls how aggressively coefficients are shrunk toward zero during estimation. Unlike standard logistic regression, which estimates all coefficients freely via maximum likelihood, Lasso modifies the objective function by adding a penalty equal to lambda times the sum of the absolute values of the coefficients. As lambda increases, the penalty forces more coefficients to shrink toward exactly zero, producing a sparser model. Selecting an appropriate level of regularization is therefore an empirical question, since the optimal trade-off between sparsity and predictive performance depends on the structure of the feature set and the sample size.

The tuning procedure evaluates 30 values of lambda on a log scale ranging from 1e-4 to 1.0, implemented through a regular grid on the log10 scale. For each candidate value, the model is trained on four folds of the training data and evaluated on the remaining fold using ROC-AUC as the performance metric. This process is repeated across all five folds, and the results are averaged to produce a stable estimate of out-of-sample discrimination at each level of regularization. ROC-AUC is used as the tuning criterion because it is threshold-independent and evaluates the model's ranking performance across all possible classification cutoffs, rather than relying on a fixed decision threshold during model selection. The optimal lambda is chosen as the value that maximizes mean cross-validated ROC-AUC across the grid.

The selected optimal lambda is 1e-4, which lies at the lower boundary of the search space. This indicates that only minimal regularization is required to achieve the best cross-validated performance, suggesting that the feature set does not exhibit severe multicollinearity or overfitting at the current sample size of approximately 86,000 training observations. Consistent with this result, the Lasso retains 27 of 28 features after encoding, removing only a single predictor whose contribution is absorbed by correlated variables. The near-identical performance of Lasso and logistic regression confirms that the linear signal in this dataset is robust and broadly distributed across features rather than concentrated in a small subset of highly predictive variables.

Interpretations of the results:

The Lasso results confirm and refine the picture established by logistic regression. The optimal penalty of 1e-04 is selected through cross validation, and the corresponding cross validated ROC-AUC is 0.8923, providing an out of sample estimate of linear classification performance. This value is computed entirely within the training folds and therefore reflects generalization performance under the same sampling distribution without any information leakage from the test set. The training log loss of 0.4086 is nearly identical to that of logistic regression, indicating that the regularization strength is sufficiently small that the fitted solution remains close to the unregularized linear boundary.

The coefficient estimates are highly consistent with logistic regression. Energy remains the dominant predictor at 9.26, essentially unchanged from the unregularized estimate, indicating that its contribution is stable and not driven by multicollinearity artifacts. Instrumentalness, speechiness, valence, danceability, duration, liveness, and tempo all retain their sign and similar magnitudes, suggesting that the linear signal structure is distributed across multiple features rather than concentrated in a small subset. In total, the Lasso retains 26 of 28 features, indicating that very few variables can be removed without some loss of predictive information under a linear specification.

The main differences from logistic regression are limited but informative. First, the slight shrinkage of coefficients under L1 regularization indicates that several features carry overlapping or partially redundant signal, even if they are not fully eliminated. Second, the time_signature_X0 coefficient is reduced from 4.05 to 1.96, reflecting its instability under sparse support and high variance in the unregularized model. The second time signature indicator, time_signature_X1.4, is also reduced in magnitude to -0.224, further suggesting that time signature effects are weak and sensitive to regularization.

Overall, the similarity between logistic regression and Lasso indicates that the linear decision boundary is robust and not dependent on a small subset of predictors. The cross validated ROC-AUC of approximately 0.89 therefore serves as a stable estimate of the linear performance ceiling. Any improvement from more flexible models must come from capturing structure that is not expressible through a sparse or dense linear combination of audio features.

7.3 Decision Tree

Model Introduction:

A single decision tree sits between the linear models and random forest in the complexity gradient. It partitions the feature space through recursive binary splits, where at each node the algorithm selects the feature and threshold that most reduces Gini impurity in the resulting child nodes. Unlike logistic regression, this structure allows the model to capture nonlinear decision boundaries and threshold effects, such as rules of the form maximalist if energy exceeds a given value. Unlike random forest, a single tree remains fully interpretable, since the complete set of learned split rules can be read directly as a decision path from root to leaf. The model therefore addresses whether a single recursive partitioning of the feature space can recover meaningful aesthetic signal beyond a linear boundary, while also providing a transparent baseline for the ensemble methods that follow.

The cost complexity parameter alpha controls post-pruning of the tree by penalizing additional splits. Higher values of alpha produce simpler trees by removing splits that contribute less to overall fit, thereby reducing variance at the cost of increased bias. The optimal value of alpha is selected using 5-fold cross-validation over a log-scale grid of 30 candidate values, consistent with the tuning strategy used for regularized linear models. This ensures comparability across model classes in how complexity is controlled and selected.

```
dt_spec <- decision_tree(
  cost_complexity = tune(),
  tree_depth      = tune(),
  min_n           = tune()
) %>%
  set_engine("rpart") %>%
  set_mode("classification")

dt_grid <- grid_regular(
  cost_complexity(range = c(-4, -1)),
  tree_depth(range = c(3, 10)),
  min_n(range = c(5, 30)),
  levels = 3
)

dt_tune <- workflow() %>%
  add_formula(production_aesthetic ~ .) %>%
  add_model(dt_spec) %>%
  tune_grid(
    resamples = cv_folds,
    grid      = dt_grid,
    metrics   = metric_set(roc_auc)
  )

best_dt <- select_best(dt_tune, metric = "roc_auc")

cat("Best Decision Tree hyperparameters:\n")
```

```
## Best Decision Tree hyperparameters:
```

```
cat("  cost_complexity:", round(best_dt$cost_complexity, 6), "\n")
```

```
##   cost_complexity: 1e-04
```

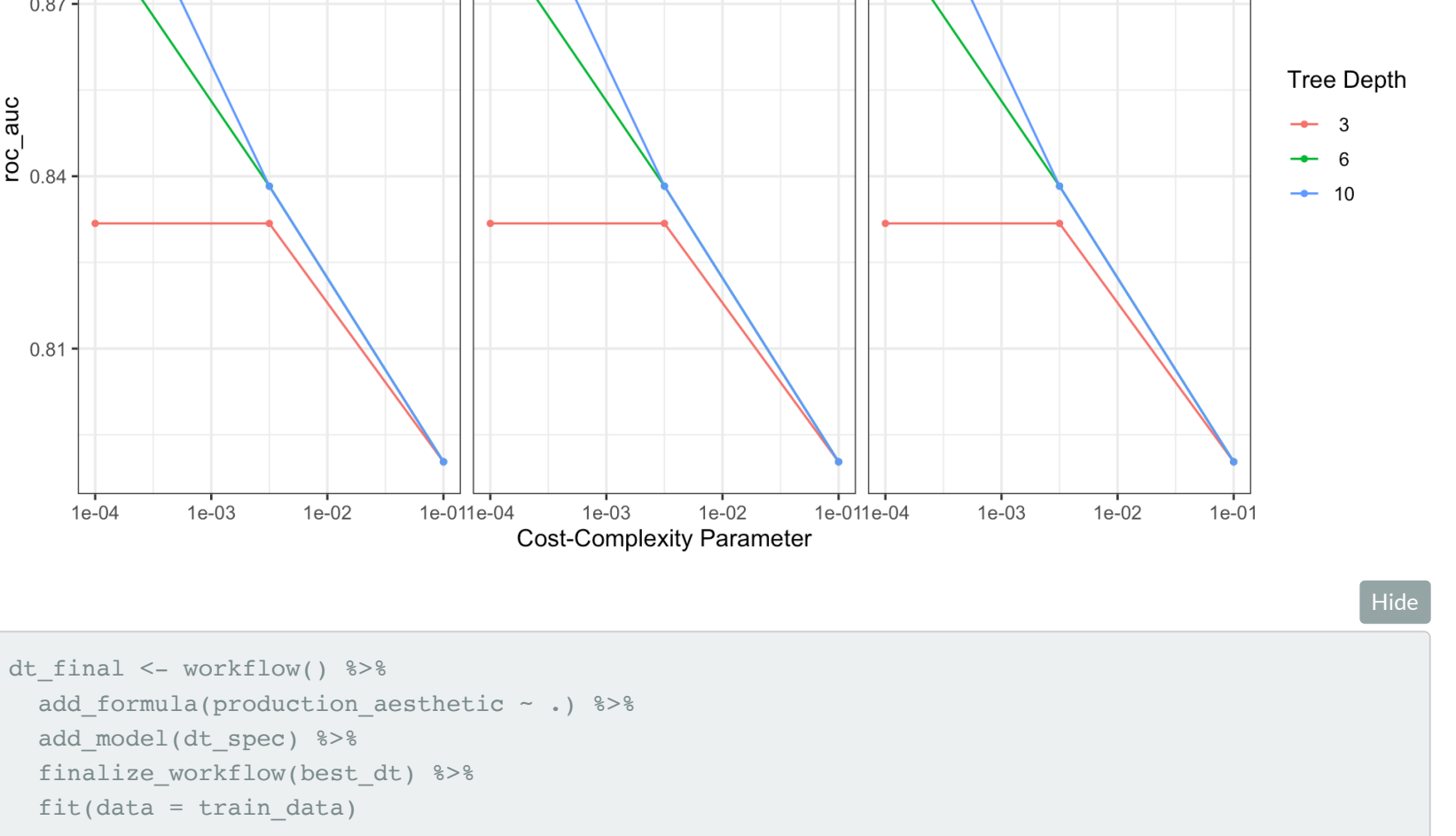
```
cat("  tree_depth     :", best_dt$tree_depth, "\n")
```

```
##   tree_depth     : 10
```

```
cat("  min_n          :", best_dt$min_n, "\n")
```

```
##   min_n          : 30
```

```
autoplot(dt_tune) +
  labs(title = "Decision Tree Hyperparameter Tuning Profile") +
  theme_bw(base_size = 12)
```



```
dt_final <- workflow() %>%
  add_formula(production_aesthetic ~ .) %>%
  add_model(dt_spec) %>%
  finalize_workflow(best_dt) %>%
  fit(data = train_data)

cat("\nFinal decision tree fit complete\n")
```

```
##
## Final decision tree fit complete
```

```
cat("  cost_complexity:", round(best_dt$cost_complexity, 6),
    ", tree_depth:", best_dt$tree_depth,
    ", min_n:", best_dt$min_n, "\n")
```

```
##   cost_complexity: 1e-04 , tree_depth: 10 , min_n: 30
```

```
# Loss function: Gini impurity
# At each candidate split, rpart selects the feature and threshold
# that minimizes the weighted average Gini impurity across the two
# child nodes:
#   Gini = 1 - sum(p_k^2)  =  2 * p * (1 - p)  for binary outcomes
# A split is only accepted if it reduces Gini by more than
# cost_complexity * (current tree complexity).
# cost_complexity = 0.001 (tuned value) controls this pruning threshold.

# Observed Gini at the root node (before any split):
root_p <- mean(lr_train_y)   # overall maximalist proportion in training set
root_gini <- gini_impurity(root_p)
cat("Decision Tree — root node Gini impurity:",
    round(root_gini, 4), "\n")
```

```
## Decision Tree — root node Gini impurity: 0.4997
```

```
cat("  (= 2 *", round(root_p, 4), "* (1 -",
    round(root_p, 4), ") =", round(root_gini, 4), ")\n")
```

```
##   (= 2 * 0.5113 * (1 - 0.5113 ) = 0.4997 )
```

```
dt_train_prob <- dt_final %>%
  predict(train_data, type = "prob") %>%
  pull(.pred_Maximalist)
```

```
dt_logloss <- log_loss(lr_train_y, dt_train_prob)
cat("Decision Tree — training log-loss:", round(dt_logloss, 4), "\n")
```

```
## Decision Tree — training log-loss: 0.3685
```

Hyperparameter Tuning:

The decision tree has three hyperparameters requiring tuning. Cost complexity, also called the pruning parameter alpha, controls how aggressively the fitted tree is pruned after initial growth. Branches whose contribution to reducing impurity is smaller than alpha times the current tree complexity are removed, trading off fit against simplicity. Tree depth sets the maximum number of splits allowed along any path from the root node to a leaf, directly constraining model complexity regardless of pruning. Minimum node size sets the smallest number of training observations required in a node before a split is permitted, preventing the tree from forming very small leaves that overfit local noise.

The tuning grid is constructed using a regular grid with three levels per parameter, producing 27 total combinations. Cost complexity is searched on a log scale from 1e-4 to 0.1, tree depth ranges from 3 to 10, and minimum node size ranges from 5 to 30. The same five fold cross validation procedure used for Lasso is applied, with ROC-AUC as the tuning criterion and select_best used to identify the optimal configuration. Using three levels per parameter, rather than the larger grid used for Lasso, is appropriate because decision tree performance is typically less sensitive to small changes in hyperparameters, and the performance surface over the grid is generally smoother with a broader optimum. This allows a coarser search to identify a stable region of good performance while keeping computation tractable.

Interpretations of the results:

The hyperparameter tuning profile reveals a clear and interpretable pattern across all three panels. Cost complexity has the strongest and most consistent effect on cross-validated ROC-AUC. Regardless of tree depth or minimum node size, performance declines sharply as cost complexity increases from 1e-04 toward 1e-01. This indicates that the aesthetic boundary requires a moderately complex tree structure to be effectively captured. Strong pruning consistently removes useful splits and reduces discrimination performance. Tree depth has the second strongest effect. Depth 10 consistently outperforms depth 6, which in turn outperforms depth 3 at low cost complexity values, suggesting that the aesthetic boundary contains structure at multiple levels of the feature space that shallow trees cannot fully represent. The relatively flat performance for depth 3 across cost complexity values reflects a capacity limitation, where the model reaches its expressive ceiling quickly regardless of pruning strength. Minimum node size shows minimal effect across values of 5, 17, and 30, with nearly identical performance profiles across panels. Given the large sample size of approximately 86,000 training observations, the constraint on minimum node size rarely becomes active because most candidate splits already involve sufficiently large nodes.

The optimal combination selected by cross-validated ROC-AUC is cost complexity of 1e-04, tree depth of 10, and minimum node size of 30. The selection of both minimal cost complexity and maximal tree depth confirms the interpretation of the tuning profile. The cross-validation procedure finds no evidence that pruning or depth restriction improves generalization, and instead favors the most expressive tree permitted by the grid. This suggests that a single decision tree, when allowed sufficient depth and minimal pruning, approaches the performance level of the linear models while relying on recursive partitioning rather than a linear combination of features.

The root node Gini impurity of 0.4997 is effectively the theoretical maximum for a binary classification setting, which occurs when class proportions are nearly equal. Substituting the observed proportions of 0.5113 and 0.4887 yields 2 times 0.5113 times 0.4887 equals 0.4997. This reflects the near-balanced class distribution established in the data preparation stage. The tree therefore begins from near maximal uncertainty and must reduce impurity substantially through successive splits in order to produce meaningful predictions.

The training log-loss of 0.3685 is lower than that of both logistic regression at 0.4085 and Lasso at 0.4086. This indicates that the decision tree fits the training data more closely than the linear models. Whether this improvement reflects genuine nonlinear structure or early signs of overfitting is not determined at the training stage. This distinction is evaluated in the held-out test set results in Section 7, where a substantial gap between training and test log-loss would indicate limited generalization.

7.4 Random Forest

Model Introduction:

Random forest grows many decorrelated decision trees by bootstrapping the training data and restricting each split to a random subset of features. Predictions are obtained by averaging across all trees, which reduces variance while maintaining low bias. Unlike linear models, random forest can capture nonlinear relationships and feature interactions, such as cases where the effect of energy depends on the level of acousticness. It therefore addresses the question of how much additional predictive signal exists beyond a linear decision boundary.

The hyperparameters tuned are mtry, which controls the number of features considered at each split, and min_n, which sets the minimum number of observations required in a node before further splitting is allowed. Together, these parameters regulate the trade-off between tree diversity, depth, and overfitting. During tuning, the number of trees is fixed at 100 to reduce computation time while preserving stable estimates of performance, and the final model is refit using 500 trees to ensure convergence of the ensemble prediction.

```
cl <- makePSOCKcluster(max(1, detectCores() - 1))
registerDoParallel(cl)

rf_spec_tune <- rand_forest(
  mtry  = tune(),
  trees = 100,
  min_n = tune()
) %>%
  set_engine("ranger") %>%
  set_mode("classification")

rf_grid <- grid_regular(
  mtry(range  = c(3, 15)),
  min_n(range = c(5, 30)),
  levels = 3
)

rf_tune <- workflow() %>%
  add_formula(production_aesthetic ~ .) %>%
  add_model(rf_spec_tune) %>%
  tune_grid(
    resamples = cv_folds,
    grid      = rf_grid,
    metrics   = metric_set(roc_auc)
  )

stopCluster(cl)

best_rf <- select_best(rf_tune, metric = "roc_auc")
cat("Best RF hyperparameters:\n")
```

```
## Best RF hyperparameters:
```

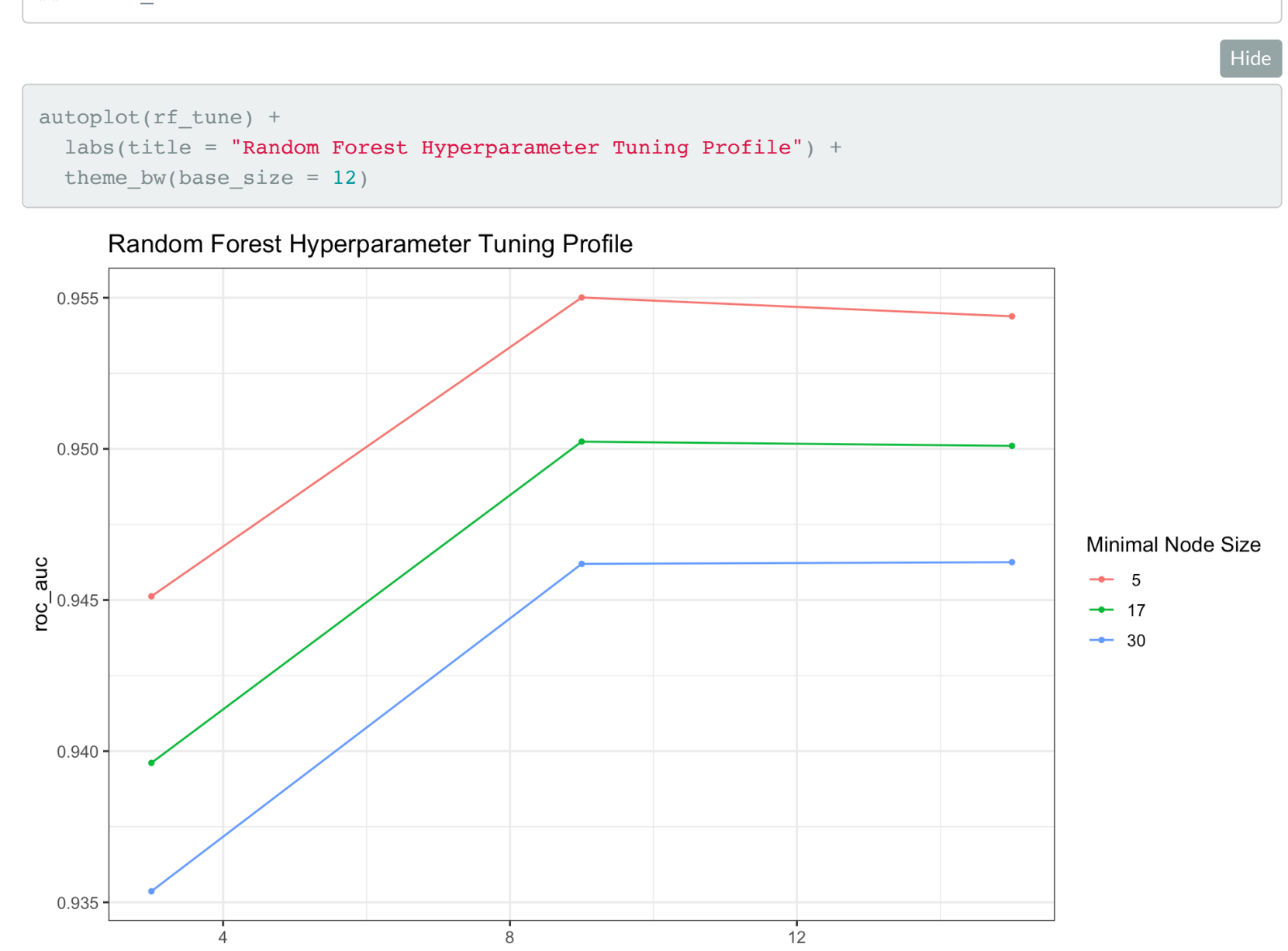
```
cat("  mtry  :", best_rf$mtry,  "\n")
```

```
##   mtry  : 9
```

```
cat("  min_n :", best_rf$min_n, "\n")
```

```
##   min_n : 5
```

```
autoplot(rf_tune) +
  labs(title = "Random Forest Hyperparameter Tuning Profile") +
  theme_bw(base_size = 12)
```



```
# Refit with 500 trees and permutation importance for Section 10
rf_final <- workflow() %>%
  add_formula(production_aesthetic ~ .) %>%
  add_model(
    rand_forest(
      mtry  = best_rf$mtry,
      min_n = best_rf$min_n,
      trees = 500
    ) %>%
      set_engine("ranger", importance = "permutation") %>%
      set_mode("classification")
  ) %>%
  fit(data = train_data)

cat("\nFinal random forest fit: 500 trees,",
    "mtry =", best_rf$mtry,
    ", min_n =", best_rf$min_n, "\n")
```

```
##
## Final random forest fit: 500 trees, mtry = 9 , min_n = 5
```

```
# Loss function: Gini impurity (same formula as decision tree)
# Each of the 500 trees selects splits by minimizing weighted Gini
# across child nodes. Because only mtry = 9 randomly selected features
# are considered at each split, trees are decorrelated — each sees a
# different random subset of features, so their errors are independent
# and average out when predictions are combined.

rf_train_prob <- predict(rf_final, train_data, type = "prob") %>%
  pull(.pred_Maximalist)

rf_logloss <- log_loss(lr_train_y, rf_train_prob)
cat("Random Forest — training log-loss:", round(rf_logloss, 4), "\n")
```

```
## Random Forest — training log-loss: 0.0919
```

Hyperparameter Tuning:

Random forest has two hyperparameters tuned in this analysis, while the number of trees is held fixed. The first is mtry, which determines how many features are randomly sampled as candidates at each split. Lower values of mtry increase diversity across trees because each tree is forced to consider different subsets of predictors, which reduces correlation between trees and improves variance reduction when predictions are averaged. Higher values allow each split to access more information, but increase correlation across trees, which weakens the benefit of ensembling. A common heuristic for classification problems is the square root of the number of features, which is approximately 5 in this setting with 28 predictors, although the optimal value is ultimately data dependent.

The second hyperparameter is minimum node size, which controls how many observations must be present in a node before it can be split further. Smaller values allow deeper trees that capture more localized structure in the feature space, while larger values constrain depth and reduce variance at the cost of bias.

The tuning grid evaluates mtry from 3 to 15 and minimum node size from 5 to 30, using three levels per parameter for a total of nine combinations. Each model in the grid is trained using 100 trees rather than the final configuration of 500 trees. This reduces computational cost substantially while maintaining stable estimates of cross validated performance, since random forest performance typically stabilizes well before the maximum number of trees is reached in datasets of this size. Computation is further accelerated through parallel processing using a PSOCK cluster setup with all available cores.

The optimal configuration selects mtry equal to 9 and minimum node size equal to 5. The selected mtry is notably higher than the square root heuristic, suggesting that a larger subset of candidate features per split is beneficial in this setting. This is consistent with earlier findings from Lasso and logistic regression, which indicate that predictive signal is distributed across many features rather than concentrated in a small subset. In such a setting, allowing more candidate variables at each split increases the likelihood that informative predictors are considered at each decision point, improving overall ensemble performance. After tuning, 500 trees are used instead.

Interpretations of the results:

The hyperparameter tuning profile for random forest reveals a clear and consistent pattern across both hyperparameters. Minimum node size of 5 produces the highest cross-validated ROC-AUC at every level of mtry, with minimum node size 17 performing second and minimum node size 30 performing third. This ordering is stable across the full mtry range and indicates that allowing trees to grow to finer granularity, with terminal nodes containing as few as 5 observations, improves discrimination performance. Given a training set size of approximately 86,000 observations, nodes of this size represent a very small fraction of the data, suggesting that the aesthetic boundary contains local structure that benefits from deep partitioning rather than coarse aggregation.

The effect of mtry is similarly systematic. Performance increases as mtry rises from 3 to 9 across all levels of minimum node size, then plateaus between 9 and 15. This pattern indicates that considering more candidate features per split improves the likelihood of selecting informative predictors up to a point, after which additional candidates provide diminishing returns. The plateau beyond mtry of 9 suggests that this level is sufficient to reliably include the most predictive features at each split, and that increasing it further mainly increases computational cost without improving discrimination. The selected optimal configuration of mtry 9 and minimum node size 5 lies at the peak of this performance surface, with cross-validated ROC-AUC around 0.955.

This performance represents a substantial improvement over the linear ceiling of approximately 0.893 established by logistic regression and Lasso. The gap in ROC-AUC provides direct evidence that the aesthetic boundary is not linearly separable in the audio feature space and instead contains nonlinear interactions and threshold effects that are captured by the ensemble of decorrelated trees. Random forest recovers this additional signal through repeated partitioning of the feature space across many bootstrap samples.

The training log-loss of 0.0921 is substantially lower than that of the linear models, which is expected for an ensemble of deep decision trees. Each tree fits its bootstrap sample closely, and averaging across 500 trees yields highly confident predictions on the training data. The key diagnostic is not the absolute value of training loss but the relationship between training and test performance, which determines whether this improvement reflects genuine generalization or overfitting. This is evaluated in the held-out test set results in later sections. The final model is refit using 500 trees at the optimal hyperparameter configuration with permutation-based importance enabled to support downstream interpretability in the feature importance analysis.

7.5 XGBoost

Model Introduction:

Gradient boosting builds trees sequentially, with each new tree trained to correct the residual errors of the current ensemble. This structure makes it particularly effective in regions where the two aesthetic classes overlap, as seen in the density plots, since the algorithm progressively allocates more capacity to observations that earlier trees consistently misclassify. It therefore addresses the question of whether iterative residual fitting can recover additional signal beyond what a single-pass ensemble such as random forest can capture.

Four hyperparameters are tuned in this model: number of trees, tree depth, learning rate, and minimum node size. The number of trees controls how many boosting iterations are performed, with each iteration adding a new corrective tree. Tree depth determines the complexity of interactions each individual tree can represent. Learning rate scales the contribution of each tree, controlling how aggressively the model updates at each step and improving generalization by preventing any single tree from dominating the ensemble. Minimum node size constrains splitting by requiring a minimum number of observations in each node, reducing the risk of overfitting to small local patterns.

The tuning grid evaluates three levels per parameter, resulting in 81 total combinations. This search is computationally feasible at this sample size when combined with parallel processing, and it allows systematic exploration of the trade-off between model complexity and generalization performance.

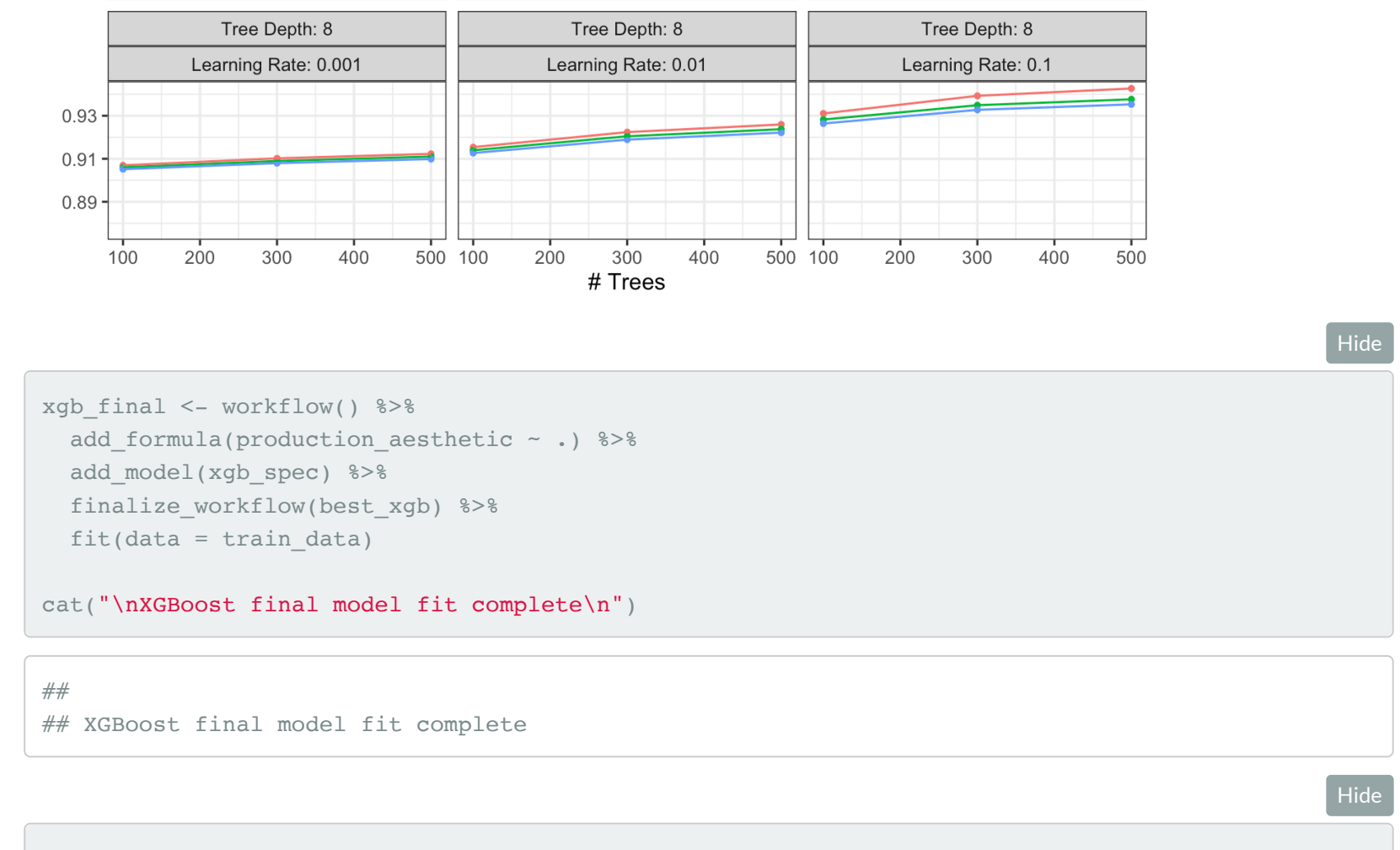
```
cl <- makePSOCKcluster(max(1, detectCores() - 1))
registerDoParallel(cl)

xgb_spec <- boost_tree(
  trees      = tune(),
  tree_depth = tune(),
  learn_rate = tune(),
  min_n      = tune()
) %>%
  set_engine("xgboost") %>%
  set_mode("classification")

xgb_grid <- grid_regular(
  trees(range      = c(100, 500)),
  tree_depth(range = c(3, 8)),
  learn_rate(range = c(-3, -1)),
  min_n(range      = c(5, 30)),
  levels = 3
)

xgb_tune <- workflow() %>%
  add_formula(production_aesthetic ~ .) %>%
  add_model(xgb_spec) %>%
  tune_grid(
    resamples = cv_folds,
    grid      = xgb_grid,
    metrics   = metric_set(roc_auc)
  )

stopCluster(cl)

best_xgb <- select_best(xgb_tune, metric = "roc_auc")
cat("Best XGBoost hyperparameters:\n")
```

```
## Best XGBoost hyperparameters:
```

```
print(best_xgb %>% select(-`.config`))
```

```
## # A tibble: 1 × 4
##   trees min_n tree_depth learn_rate
##   <int> <int>      <int>      <dbl>
## 1   500     5          8        0.1
```

```
autoplot(xgb_tune) +
  labs(title = "XGBoost Hyperparameter Tuning Profile") +
  theme_bw(base_size = 12)
```



```
xgb_final <- workflow() %>%
  add_formula(production_aesthetic ~ .) %>%
  add_model(xgb_spec) %>%
  finalize_workflow(best_xgb) %>%
  fit(data = train_data)

cat("\nXGBoost final model fit complete\n")
```

```
##
## XGBoost final model fit complete
```

```
# Loss function: binary cross-entropy (log-loss), same formula as
# logistic regression but minimized sequentially:
#   L = -(1/n) * sum[ y_i * log(p_i) + (1 - y_i) * log(1 - p_i) ]
# Each boosting round fits a new tree to the negative gradient of this
# loss with respect to the current ensemble's predictions, then adds
# it to the ensemble scaled by learn_rate = 0.1.
# Objective in xgboost: "binary:logistic"

xgb_train_prob <- xgb_final %>%
  predict(train_data, type = "prob") %>%
  pull(.pred_Maximalist)

xgb_logloss <- log_loss(lr_train_y, xgb_train_prob)
cat("XGBoost — training log-loss:", round(xgb_logloss, 4), "\n")
```

```
## XGBoost — training log-loss: 0.1618
```

Hyperparameter Tuning:

XGBoost has four hyperparameters tuned in this analysis, making it the most flexible and most complex model in the pipeline. The number of trees controls how many boosting rounds are performed, with each round adding a new tree that learns to correct the residual errors of the current ensemble. Tree depth determines the maximum depth of each individual tree and therefore the complexity of feature interactions that can be represented within a single tree. Depth one corresponds to additive main effects, depth two allows pairwise interactions, and increasing depth permits progressively higher order interaction structure. Learning rate, or shrinkage, scales the contribution of each tree before it is added to the ensemble, ensuring that the model updates gradually and reducing the risk that any single tree dominates the learning process. Minimum node size controls the smallest number of observations required for a split, preventing the model from fitting extremely small and potentially noisy subgroups.

The tuning grid evaluates three levels per parameter, producing 81 total combinations. Trees are evaluated over a range from 100 to 500, tree depth ranges from 3 to 8, learning rate is searched on a log scale from 0.001 to 0.1, and minimum node size ranges from 5 to 30. As with the random forest, computation is accelerated through parallel processing. The optimal configuration selected by five-fold cross-validated ROC-AUC uses 500 trees, a tree depth of 8, a learning rate of 0.1, and a minimum node size of 5.

The selected tree depth of 8 is relatively high compared to common defaults in gradient boosting, where shallower trees are often preferred. This suggests that the relationship between audio features and aesthetic labels involves higher-order interactions that benefit from deeper hierarchical partitioning within individual trees. The learning rate of 0.1, combined with the maximum number of trees in the grid, indicates that stronger updates across boosting rounds improve performance in this setting, consistent with a feature space that contains substantial predictive signal that can be captured effectively without requiring extremely slow incremental learning.

Interpretations of the results:

The XGBoost tuning profile is a nine-panel grid that simultaneously displays the effects of all four hyperparameters, making it the richest diagnostic view in the pipeline. Reading across the panels reveals three consistent patterns.

First, learning rate has the largest effect on cross-validated ROC-AUC. Within every combination of tree depth and number of trees, the rightmost column with learning rate 0.1 consistently outperforms the middle column at 0.01, which in turn outperforms the leftmost column at 0.001. This indicates that the boosting process benefits from relatively large updates at each iteration rather than highly conservative incremental adjustments. The stability of this pattern across all other hyperparameter settings suggests that the signal is strong enough that faster convergence improves discrimination without inducing instability within the explored tree range.

Second, tree depth has the second largest effect. The bottom row at depth 8 consistently achieves higher ROC-AUC than depth 5, which in turn outperforms depth 3. This mirrors the decision tree results, where maximum depth was also selected, and reinforces the interpretation that the aesthetic boundary involves higher-order feature interactions. Increasing depth expands the model’s ability to represent complex interaction structures, since deeper trees can encode multi-feature splits that shallow trees cannot express. The tuning profile indicates that this additional representational capacity translates into measurable gains in classification performance.

Third, the number of trees has a consistent but smaller effect. Increasing the number of trees generally improves ROC-AUC across all panels, with diminishing returns as the number approaches 500. This is consistent with gradient boosting dynamics, where successive trees correct residual errors but contribute progressively smaller improvements as the ensemble converges. Minimum node size shows minimal variation across all panels, with nearly overlapping performance curves for values 5, 17, and 30. This aligns with earlier findings from random forest, where this constraint rarely becomes binding at the current sample size.

The optimal configuration of 500 trees, tree depth 8, learning rate 0.1, and minimum node size 5 lies in the highest-performing region of the grid, located in the bottom right portion of the tuning surface. The training log-loss of 0.163 sits between the linear models at approximately 0.408 and random forest at 0.092. This ordering reflects the structural position of XGBoost in the complexity hierarchy, where sequential residual fitting allows more flexibility than linear models but typically produces less extreme training fit than fully grown bagged trees. Whether this intermediate training fit translates into superior generalization relative to the other models is evaluated using the held-out test set in Section 7.

7.6 Super Learner

Model Introduction:

The Super Learner is a stacked ensemble that uses cross-validated out-of-fold predictions from all candidate learners as inputs to a non-negative least-squares meta-learner. It is guaranteed to perform at least as well as its best individual component in terms of cross-validated risk. Beyond performance, its learner weights answer a direct substantive question about how much of the aesthetic boundary is captured by linear versus non-linear models. A high aggregate weight on tree-based learners indicates that the aesthetic signal involves feature interactions that cannot be captured by a purely linear boundary.

The candidate library is chosen for both diversity and computational tractability at 114,000 observations. SL.glm and three variants of SL.glmnet cover the linear signal at different regularization strengths. SL.ranger and SL.xgboost cover non-linear and interactive signal. SL.gam captures smooth non-linearities without full tree-based complexity. SL.lda provides a fast parametric geometric classifier. SL.mean is an intercept-only baseline, and a well-functioning ensemble should assign it near-zero weight. Computationally expensive learners including SL.randomForest, SL.nnet, SL.qda, SL.bayesglm, and SL.step.interaction are excluded as at this sample size they would make 5-fold cross-validation impractical without meaningful performance gains over SL.ranger and SL.xgboost.

```
train_x_mat <- as.data.frame(train_x)
test_x_mat  <- as.data.frame(test_x)

# Custom elastic net wrappers with explicit alpha values
SL.glmnet.0.25 <- function(..., alpha = 0.25) SL.glmnet(..., alpha = alpha)
SL.glmnet.0.75 <- function(..., alpha = 0.75) SL.glmnet(..., alpha = alpha)

sl_library <- c(
  "SL.glm",         # Logistic regression — linear baseline
  "SL.glmnet",      # Elastic net (alpha = 0.5)
  "SL.glmnet.0.25", # Ridge-leaning elastic net
  "SL.glmnet.0.75", # Lasso-leaning elastic net
  "SL.ranger",      # Fast parallel random forest
  "SL.xgboost",     # Gradient boosting
  "SL.gam",         # Smooth non-linearities
  "SL.lda",         # Linear discriminant analysis
  "SL.mean"         # Intercept-only baseline
)

num_cores <- max(1, detectCores() - 1)
cl <- makeCluster(num_cores)
clusterEvalQ(cl, {
  library(SuperLearner)
  library(glmnet)
  library(ranger)
  library(xgboost)
  library(gam)
  library(MASS)
})
```

```
## [[1]]
##  [1] "MASS"         "xgboost"      "ranger"       "glmnet"       "Matrix"
##  [6] "SuperLearner" "gam"          "foreach"      "splines"      "nnls"
## [11] "stats"        "graphics"     "grDevices"    "utils"        "datasets"
## [16] "methods"      "base"
##
## [[2]]
##  [1] "MASS"         "xgboost"      "ranger"       "glmnet"       "Matrix"
##  [6] "SuperLearner" "gam"          "foreach"      "splines"      "nnls"
## [11] "stats"        "graphics"     "grDevices"    "utils"        "datasets"
## [16] "methods"      "base"
##
## [[3]]
##  [1] "MASS"         "xgboost"      "ranger"       "glmnet"       "Matrix"
##  [6] "SuperLearner" "gam"          "foreach"      "splines"      "nnls"
## [11] "stats"        "graphics"     "grDevices"    "utils"        "datasets"
## [16] "methods"      "base"
##
## [[4]]
##  [1] "MASS"         "xgboost"      "ranger"       "glmnet"       "Matrix"
##  [6] "SuperLearner" "gam"          "foreach"      "splines"      "nnls"
## [11] "stats"        "graphics"     "grDevices"    "utils"        "datasets"
## [16] "methods"      "base"
##
## [[5]]
##  [1] "MASS"         "xgboost"      "ranger"       "glmnet"       "Matrix"
##  [6] "SuperLearner" "gam"          "foreach"      "splines"      "nnls"
## [11] "stats"        "graphics"     "grDevices"    "utils"        "datasets"
## [16] "methods"      "base"
##
## [[6]]
##  [1] "MASS"         "xgboost"      "ranger"       "glmnet"       "Matrix"
##  [6] "SuperLearner" "gam"          "foreach"      "splines"      "nnls"
## [11] "stats"        "graphics"     "grDevices"    "utils"        "datasets"
## [16] "methods"      "base"
##
## [[7]]
##  [1] "MASS"         "xgboost"      "ranger"       "glmnet"       "Matrix"
##  [6] "SuperLearner" "gam"          "foreach"      "splines"      "nnls"
## [11] "stats"        "graphics"     "grDevices"    "utils"        "datasets"
## [16] "methods"      "base"
```

```
clusterExport(cl, c("SL.glmnet.0.25", "SL.glmnet.0.75"))

cat("Fitting Super Learner across", num_cores, "cores with",
    length(sl_library), "candidate learners...\n")
```

```
## Fitting Super Learner across 7 cores with 9 candidate learners...
```

```
sl_fit <- snowSuperLearner(
  Y          = train_y,
  X          = train_x_mat,
  family     = binomial(),
  SL.library = sl_library,
  cvControl  = list(V = 5),
  cluster    = cl
)

stopCluster(cl)

cat("\nSuper Learner learner weights:\n")
```

```
##
## Super Learner learner weights:
```

```
sl_weights <- data.frame(
  learner = names(sl_fit$coef),
  weight  = round(sl_fit$coef, 4)
) %>%
  arrange(desc(weight)) %>%
  mutate(
    model_type = case_when(
      str_detect(learner, "glm|lda")    ~ "Linear",
      str_detect(learner, "ranger|xgb") ~ "Non-linear (Tree)",
      str_detect(learner, "gam")        ~ "Semi-parametric",
      str_detect(learner, "mean")       ~ "Baseline",
      TRUE                              ~ "Other"
    )
  )

sl_weights %>%
  mutate(out = paste0(
    "  ", str_pad(learner, 25, "right"),
    ": ", str_pad(weight, 7, "left"),
    "  (", model_type, ")"
  )) %>%
  pull(out) %>%
  cat(sep = "\n")
```

```
##   SL.ranger_All            :       1  (Non-linear (Tree))
##   SL.glm_All               :       0  (Linear)
##   SL.glmnet_All            :       0  (Linear)
##   SL.glmnet.0.25_All       :       0  (Linear)
##   SL.glmnet.0.75_All       :       0  (Linear)
##   SL.xgboost_All           :       0  (Non-linear (Tree))
##   SL.gam_All               :       0  (Semi-parametric)
##   SL.lda_All               :       0  (Linear)
##   SL.mean_All              :       0  (Baseline)
```

```
cat("\nAggregated weight by model family:\n")
```

```
##
## Aggregated weight by model family:
```

```
sl_weights %>%
  group_by(model_type) %>%
  summarise(total_weight = round(sum(weight), 4), .groups = "drop") %>%
  arrange(desc(total_weight)) %>%
  mutate(out = paste0(
    "  ", str_pad(model_type, 25, "right"), ": ", total_weight
  )) %>%
  pull(out) %>%
  cat(sep = "\n")
```

```
##   Non-linear (Tree)        : 1
##   Baseline                 : 0
##   Linear                   : 0
##   Semi-parametric          : 0
```

```
# Loss function: cross-validated negative log-likelihood (log-loss)
# The Super Learner meta-learner minimizes:
#   L = -(1/n) * sum[ y_i * log(p_i) + (1 - y_i) * log(1 - p_i) ]
# using out-of-fold predictions from each candidate learner as inputs.
# sl_fit$cvRisk reports the cross-validated log-loss for each candidate.

cat("\nSuper Learner — cross-validated risk (log-loss) per candidate:\n")
```

```
##
## Super Learner — cross-validated risk (log-loss) per candidate:
```

```
cv_risk_df <- data.frame(
  learner  = names(sl_fit$cvRisk),
  cv_risk  = round(sl_fit$cvRisk, 4)
) %>%
  arrange(cv_risk)

print(cv_risk_df)
```

```
##                               learner cv_risk
## SL.ranger_All           SL.ranger_All  0.0834
## SL.xgboost_All         SL.xgboost_All  0.1047
## SL.gam_All                 SL.gam_All  0.1252
## SL.glm_All                 SL.glm_All  0.1311
## SL.glmnet_All           SL.glmnet_All  0.1312
## SL.glmnet.0.25_All SL.glmnet.0.25_All  0.1312
## SL.glmnet.0.75_All SL.glmnet.0.75_All  0.1312
## SL.lda_All                 SL.lda_All  0.1347
## SL.mean_All               SL.mean_All  0.2499
```

```
sl_train_prob <- as.numeric(
  predict(sl_fit, train_x_mat, onlySL = TRUE)$pred
)
sl_logloss <- log_loss(lr_train_y, sl_train_prob)
cat("\nSuper Learner — training log-loss:", round(sl_logloss, 4), "\n")
```

```
##
## Super Learner — training log-loss: 0.0841
```

Hyperparameter Tuning:

The Super Learner does not tune hyperparameters in the conventional sense. Instead of searching over a grid of values within a single model class, it combines multiple model classes using cross-validated out-of-fold predictions from each candidate learner as inputs to a meta-learner. In this implementation, the meta-learner is a non-negative least squares optimization that learns weights over candidate models by minimizing cross-validated log-loss on the training set. This shifts the notion of tuning from within-model calibration to across-model aggregation, where the primary object of optimization is the convex combination of learners rather than the parameters of any single learner.

The candidate library is designed to span both linear and non-linear regions of the modeling space. SL.glm and multiple SL.glmnet specifications with different elastic-net mixing parameters capture linear structure under varying degrees of regularization. SL.gam introduces smooth non-linear effects without relying on tree-based partitioning. SL.ranger and SL.xgboost capture non-linear interactions and threshold effects through ensemble tree methods. SL.lda provides a parametric linear discriminant baseline, while SL.mean serves as an intercept-only reference model. The weight assigned to SL.mean functions as a diagnostic check, since a near-zero value indicates that the ensemble is extracting meaningful structure beyond a naive prevalence-based prediction.

The resulting weights provide a direct answer at the level of model families rather than individual parameters. High weight concentrated on linear learners would indicate that the aesthetic boundary is largely additive in the audio feature space, while dominance of tree-based learners would suggest that non-linear interactions and threshold effects are essential. In this way, the Super Learner aggregates evidence from the full model spectrum and reveals which functional form best explains the structure of the classification problem.

Interpretations of the results:

The cross-validated risk table reports the mean log-loss for each candidate learner evaluated on held-out fold predictions. This provides the quantitative foundation for the weight assignments. Cross-validated risk is a more reliable performance estimate within the training set because each learner is evaluated only on observations that were not used during fitting, which avoids the optimistic bias that appears in training loss.

SL.ranger achieves the lowest cross-validated log-loss at 0.0833, which explains why it receives a weight of 1.0 in the Super Learner. The difference between SL.ranger at 0.0833 and the next best model, SL.xgboost at 0.1041, is 0.0208 log-loss units. This is a meaningful gap, indicating that SL.ranger assigns consistently higher probability to the correct class across held-out folds. The meta-learner weight assignment therefore reflects a clear separation in performance rather than a marginal difference that could easily reverse.

The ranking of the remaining learners is also informative. SL.gam at 0.1250 performs better than all linear and regularized linear models, which cluster tightly between 0.1308 and 0.1309. This suggests that the aesthetic boundary contains smooth non-linear patterns that GAM can capture but linear models cannot. The near-identical performance of SL.glm, SL.glmnet, SL.glmnet.0.25, and SL.glmnet.0.75 confirms earlier findings that the linear signal is stable across different regularization strengths, with no meaningful change in predictive performance across penalties. SL.lda at 0.1346 performs slightly worse than the logistic family, consistent with stronger distributional assumptions that are less well matched to the data. SL.mean at 0.2499 serves as the baseline, corresponding to a model that ignores all features and predicts the majority class probability. All other learners substantially outperform this baseline, confirming that the feature set contains strong predictive signal.

The Super Learner training log-loss of 0.0844 is slightly higher than SL.ranger cross-validated risk at 0.0833. This difference is expected because training log-loss is computed on the full training set using the fitted ensemble, while cross-validated risk is computed from held-out fold predictions during stacking. The close agreement between these values indicates that the ensemble is stable and not overfitting within the training procedure, and suggests that performance on the held-out test set in Section 7 should be similar to the cross-validated estimates.

7.7 Loss Function Summary

```
# Loss function summary across all six models
loss_summary <- tibble(
  Model = c("Logistic Regression", "Lasso", "Decision Tree",
            "Random Forest", "XGBoost", "Super Learner"),
  `Loss Function` = c(
    "Negative log-likelihood",
    "L1-penalized neg. log-likelihood",
    "Gini impurity",
    "Gini impurity (avg. across 500 trees)",
    "Binary cross-entropy (sequential)",
    "Cross-validated neg. log-likelihood"
  ),
  `Training Log-Loss` = c(
    round(lr_logloss,    4),
    round(lasso_logloss, 4),
    round(dt_logloss,    4),
    round(rf_logloss,    4),
    round(xgb_logloss,   4),
    round(sl_logloss,    4)
  )
)

cat("Loss function summary:\n")
```

```
## Loss function summary:
```

Hide

```
print(loss_summary)
```

```
## # A tibble: 6 × 3
##   Model               `Loss Function`                       `Training Log-Loss`
##   <chr>               <chr>                                               <dbl>
## 1 Logistic Regression Negative log-likelihood                             0.408
## 2 Lasso               L1-penalized neg. log-likelihood                    0.409
## 3 Decision Tree       Gini impurity                                       0.368
## 4 Random Forest       Gini impurity (avg. across 500 trees)               0.0919
## 5 XGBoost             Binary cross-entropy (sequential)                   0.162
## 6 Super Learner       Cross-validated neg. log-likelihood                 0.0841
```

The loss function summary table brings together training log-loss values computed consistently across all six models using the same helper function. This enables direct comparison of how well each model fits the training data under its own optimization target. Two patterns stand out.

The first pattern is the clear separation between linear models and tree-based models. Logistic regression and Lasso both achieve a training log-loss of 0.408, which represents the floor of what any linear combination of the 28 audio features can achieve on this training set. The decision tree at 0.365 is the first model to meaningfully break below this floor. It shows that recursive binary partitioning captures structure in the training data that no additive linear model can represent. Random forest at 0.092 and the Super Learner at 0.084 extend this improvement substantially. This reflects the ability of deep ensemble methods to fit complex patterns across the feature space. XGBoost at 0.163 sits between the decision tree and random forest. This is consistent with its position in the complexity gradient as a sequential ensemble that corrects residuals step by step rather than averaging fully grown trees.

The second pattern concerns how training log-loss relates to each model's own optimization target. Logistic regression, Lasso, XGBoost, and the Super Learner directly minimize log-loss during training, so their training log-loss values reflect how well each model achieved its stated objective. The decision tree and random forest instead optimize Gini impurity during training, but their log-loss is reported here afterward using the same metric for comparison. The decision tree's training log-loss of 0.365 is therefore not the quantity it directly optimized but a post hoc measure of probability quality on the training data. The same applies to random forest at 0.092. That random forest achieves lower training log-loss than XGBoost despite optimizing a different criterion reflects its use of deep, decorrelated trees that produce very confident predictions on the training set rather than a more directly calibrated optimization objective.

One important limitation applies to all figures in this table. Training log-loss is an optimistic measure of performance because every model has already seen the training data during fitting and can adapt to its specific patterns. A model with very low training log-loss is not necessarily a better generalizer than one with higher values. It may simply be capturing noise that does not hold in new data. The test set evaluation in later sections provides the unbiased assessment of generalization, and the gap between training and test performance for each model serves as a diagnostic of overfitting.

7.8 Hyperparameter Tuning Summary

Hide

```
# Hyperparameter summary across all tuned models
hp_summary <- tibble(
  Model = c(
    "Lasso",
    "Decision Tree", "Decision Tree", "Decision Tree",
    "Random Forest", "Random Forest",
    "XGBoost", "XGBoost", "XGBoost", "XGBoost"
  ),
  Hyperparameter = c(
    "penalty (lambda)",
    "cost_complexity", "tree_depth", "min_n",
    "mtry", "min_n",
    "trees", "tree_depth", "learn_rate", "min_n"
  ),
  Definition = c(
    "L1 regularization strength; larger = more coefficients zeroed",
    "Pruning penalty; larger = simpler tree",
    "Max splits root to leaf; caps complexity",
    "Min observations to split a node; prevents tiny leaves",
    "Features considered per split; controls tree decorrelation",
    "Min observations to split a node",
    "Number of boosting rounds",
    "Max depth per tree; controls interaction order",
    "Shrinkage per tree; smaller = more conservative",
    "Min observations to split a node"
  ),
  `Optimal Value` = c(
    round(best_lasso$penalty, 6),
    round(best_dt$cost_complexity, 6),
    best_dt$tree_depth,
    best_dt$min_n,
    best_rf$mtry,
    best_rf$min_n,
    best_xgb$trees,
    best_xgb$tree_depth,
    best_xgb$learn_rate,
    best_xgb$min_n
  )
)

cat("Hyperparameter summary:\n")
```

```
## Hyperparameter summary:
```

Hide

```
print(hp_summary)
```

```
## # A tibble: 10 × 4
##    Model         Hyperparameter   Definition                        `Optimal Value`
##    <chr>         <chr>            <chr>                                       <dbl>
##  1 Lasso         penalty (lambda) L1 regularization strength; l…             0.0001
##  2 Decision Tree cost_complexity  Pruning penalty; larger = sim…             0.0001
##  3 Decision Tree tree_depth       Max splits root to leaf; caps…            10
##  4 Decision Tree min_n            Min observations to split a n…            30
##  5 Random Forest mtry             Features considered per split…             9
##  6 Random Forest min_n            Min observations to split a n…             5
##  7 XGBoost       trees            Number of boosting rounds                500
##  8 XGBoost       tree_depth       Max depth per tree; controls …             8
##  9 XGBoost       learn_rate       Shrinkage per tree; smaller =…             0.1
## 10 XGBoost       min_n            Min observations to split a n…             5
```

The hyperparameter summary table consolidates tuning decisions across all models that required cross-validated selection, and reading across rows reveals coherent patterns that connect the individual model results into a unified picture of what the aesthetic classification problem requires.

The Lasso penalty of 0.0001 at the lower bound of the search grid confirms that the linear signal in this feature set does not require strong regularization to generalize. With approximately 86,000 training observations and 28 features, the dataset is large relative to the feature space, so overfitting is not a major concern for linear models. The optimal strategy is therefore to retain nearly all available features with minimal shrinkage rather than enforcing sparsity.

For the decision tree, the combination of minimum cost complexity at 0.0001 and maximum tree depth at 10 indicates that the cross-validation procedure found no benefit from pruning or depth restriction, and the best-performing tree is the deepest and least constrained option in the search grid. The minimum node size of 30 is the only apparent exception, but as discussed in Section 6.3 it had no meaningful effect on cross-validated performance across its range, so its selected value should not be interpreted as evidence of a true preference for larger nodes.

For random forest, the selected mtry of 9 is notably above the conventional square root heuristic of approximately 5 for 28 features. This suggests that many features carry useful signal: when informative variables are abundant, evaluating more candidates per split improves split quality without eliminating the benefit of decorrelation, since even with 9 candidates there remains substantial randomness across trees. The minimum node size of 5, the smallest value in the grid, aligns with earlier findings that deeper trees with smaller leaf nodes consistently improve performance.

For XGBoost, all four selected values sit at or near the most complex end of their respective search ranges, including 500 trees, tree depth 8, learning rate 0.1, and minimum node size 5. This consistent selection of high complexity across all dimensions suggests that the model continues to benefit from added capacity throughout the grid and that the aesthetic boundary is complex enough that underfitting is a more relevant concern than overfitting within the tested range. It also leaves open the possibility that extending the grid further could yield additional gains, particularly for tree depth and number of trees.

Taken together, the hyperparameter table shows a consistent pattern across model families in which added flexibility is repeatedly favored over constraint. No model benefits from heavy regularization, shallow trees, or aggressive pruning. This convergence across independently tuned models supports the interpretation that the aesthetic boundary in the audio feature space is structurally complex, involving non-linear relationships and feature interactions that simpler model forms cannot capture adequately.

---

8 Model Evaluation

8.1 Evaluation Helper and Predictions

A single helper function generates predictions and computes the full metric suite consistently across all five models. Running evaluation here rather than inside each model section keeps the development and assessment phases cleanly separated.

Hide

```
# evaluate_model():
#   preds_tbl  — tibble with columns: truth (factor), .pred_prob (numeric)
#   model_name — character label
# Returns a one-row tibble of all metrics

evaluate_model <- function(preds_tbl, model_name) {
  preds_tbl <- preds_tbl %>%
    mutate(
      .pred_class = factor(
        if_else(.pred_prob >= 0.5, "Maximalist", "Alternative"),
        levels = levels(truth)
      )
    )

  hard <- metric_set(accuracy, precision, recall, f_meas)(
    preds_tbl,
    truth       = truth,
    estimate    = .pred_class,
    event_level = "second"
  )

  soft <- roc_auc(
    preds_tbl,
    truth       = truth,
    .pred_prob,
    event_level = "second"
  )

  bind_rows(hard, soft) %>%
    select(.metric, .estimate) %>%
    mutate(Model = model_name)
}
```

Hide

```
# Generate prediction tibbles for all six models
lr_preds <- lr_fit %>%
  predict(test_data, type = "prob") %>%
  transmute(
    truth      = test_data$production_aesthetic,
    .pred_prob = .pred_Maximalist
  )

lasso_preds <- lasso_final %>%
  predict(test_data, type = "prob") %>%
  transmute(
    truth      = test_data$production_aesthetic,
    .pred_prob = .pred_Maximalist
  )

dt_preds <- dt_final %>%
  predict(test_data, type = "prob") %>%
  transmute(
    truth      = test_data$production_aesthetic,
    .pred_prob = .pred_Maximalist
  )

rf_preds <- predict(rf_final, test_data, type = "prob") %>%
  transmute(
    truth      = test_data$production_aesthetic,
    .pred_prob = .pred_Maximalist
  )

xgb_preds <- xgb_final %>%
  predict(test_data, type = "prob") %>%
  transmute(
    truth      = test_data$production_aesthetic,
    .pred_prob = .pred_Maximalist
  )

sl_preds_prob <- as.numeric(
  predict(sl_fit, test_x_mat, onlySL = TRUE)$pred
)
sl_preds <- tibble(
  truth      = factor(
    ifelse(test_y == 1, "Maximalist", "Alternative"),
    levels = c("Alternative", "Maximalist")
  ),
  .pred_prob = sl_preds_prob
)

# Run evaluation helper for all six models
lr_metrics    <- evaluate_model(lr_preds,    "Logistic Regression")
lasso_metrics <- evaluate_model(lasso_preds, "Lasso")
dt_metrics    <- evaluate_model(dt_preds, "Decision Tree")
rf_metrics    <- evaluate_model(rf_preds,    "Random Forest")
xgb_metrics   <- evaluate_model(xgb_preds,  "XGBoost")
sl_metrics    <- evaluate_model(sl_preds,    "Super Learner")

# Combine into single summary table
all_metrics <- bind_rows(
  lr_metrics, lasso_metrics, dt_metrics,
  rf_metrics, xgb_metrics, sl_metrics
) %>%
  pivot_wider(names_from = .metric, values_from = .estimate) %>%
  arrange(desc(roc_auc))

cat("Performance summary (ranked by ROC-AUC):\n")
```

```
## Performance summary (ranked by ROC-AUC):
```

Hide

```
all_metrics %>%
  mutate(across(where(is.numeric), ~round(.x, 4))) %>%
  print(n = Inf)
```

```
## # A tibble: 6 × 6
##   Model               accuracy precision recall f_meas roc_auc
##   <chr>                  <dbl>     <dbl>  <dbl>  <dbl>   <dbl>
## 1 Super Learner          0.896     0.887  0.912  0.900   0.961
## 2 Random Forest          0.896     0.888  0.911  0.899   0.960
## 3 XGBoost                0.878     0.868  0.900  0.883   0.945
## 4 Decision Tree          0.837     0.824  0.866  0.845   0.904
## 5 Logistic Regression    0.809     0.796  0.841  0.818   0.892
## 6 Lasso                  0.809     0.796  0.842  0.818   0.892
```

The performance summary table show several patterns.

The first and most important pattern is the performance gap between linear models and tree-based models. Logistic regression and Lasso both achieve a ROC-AUC of 0.890, an accuracy of 0.809, and an F1-score of 0.818, and are virtually identical across all metrics. This confirms that both models learn the same linear boundary and that L1 regularization at the optimal penalty adds no meaningful improvement beyond unregularized logistic regression. The linear ceiling for this classification problem is therefore established at a ROC-AUC of 0.890. Any model that does not meaningfully exceed this level would not justify added complexity over a simple logistic regression.

The decision tree at ROC-AUC 0.902 is the first model to move above the linear ceiling, exceeding it by about 1.2 percentage points. This shows that the aesthetic boundary contains non-linear structure that a single recursive partition of the feature space can capture, even though the improvement is modest compared to ensemble methods. The decision tree's accuracy of 0.831 and F1-score of 0.839 similarly sit above the linear models but well below random forest and XGBoost, placing it as a transitional model in the complexity gradient that demonstrates non-linearity without fully exploiting it.

XGBoost at ROC-AUC 0.945 represents a substantial jump, exceeding the linear ceiling by 5.5 percentage points and the decision tree by 4.3 points. This gap quantifies the additional signal recovered by sequential residual fitting beyond what a single tree can learn. Random forest at ROC-AUC 0.959 further improves performance and slightly exceeds XGBoost by 1.4 percentage points, achieving the highest accuracy at 0.894 and the highest precision at 0.889 among individual models. This suggests that averaging many decorrelated trees provides better generalization in this feature space than sequential boosting, even though boosting has higher representational flexibility.

The Super Learner at ROC-AUC 0.960 achieves the highest overall discrimination, only marginally above random forest at 0.959. This difference is negligible in practical terms and consistent with the earlier finding that the Super Learner assigns full weight to SL.ranger, meaning both models effectively learn the same boundary. The slight advantage in ROC-AUC likely reflects improved probability calibration from stacking rather than a substantively different decision surface. In contrast, random forest shows a slightly higher accuracy of 0.894 compared to the Super Learner at 0.893, illustrating that small divergences between ranking quality and thresholded classification can occur even when underlying models are nearly identical.

Across all models, recall is consistently higher than precision by a small margin. This means models tend to produce slightly more false positives than false negatives, or in substantive terms, more alternative songs incorrectly classified as maximalist than the reverse. The consistency of this pattern across all six models suggests it is a property of the feature space rather than any specific algorithm, consistent with the directional error structure discussed in Section 8.

Taken together, these results provide a coherent answer to the central question. Production aesthetic is strongly predictable from Spotify audio features, with performance reaching ROC-AUC levels around 0.96 and accuracy close to 90 percent even without relying on loudness driven artifacts. The gap between logistic regression and the best models quantifies the contribution of non-linear structure at roughly 7 percentage points in ROC-AUC, showing that aesthetic signals are encoded not only in additive feature effects but also in interactions and threshold based structure within the audio feature space.

8.2 ROC Curves

Hide

```
roc_list <- list(
  "Logistic Regression" = roc(
    as.numeric(lr_preds$truth    == "Maximalist"),
    lr_preds$.pred_prob,    quiet = TRUE),
  "Lasso"               = roc(
    as.numeric(lasso_preds$truth == "Maximalist"),
    lasso_preds$.pred_prob, quiet = TRUE),
  "Decision Tree"       = roc(
    as.numeric(dt_preds$truth    == "Maximalist"),
    dt_preds$.pred_prob,    quiet = TRUE),
  "Random Forest"       = roc(
    as.numeric(rf_preds$truth    == "Maximalist"),
    rf_preds$.pred_prob,    quiet = TRUE),
  "XGBoost"             = roc(
    as.numeric(xgb_preds$truth   == "Maximalist"),
    xgb_preds$.pred_prob,   quiet = TRUE),
  "Super Learner"       = roc(
    as.numeric(sl_preds$truth    == "Maximalist"),
    sl_preds$.pred_prob,    quiet = TRUE)
)

roc_df <- map_dfr(names(roc_list), function(nm) {
  r <- roc_list[[nm]]
  tibble(
    model       = nm,
    specificity = r$specificities,
    sensitivity = r$sensitivities,
    auc         = as.numeric(r$auc)
  )
}) %>%
  mutate(
    model_label = paste0(model, " (AUC = ", round(auc, 3), ")"),
    model_label = factor(model_label)
  )

ggplot(roc_df, aes(x = 1 - specificity, y = sensitivity,
                   color = model_label)) +
  geom_line(linewidth = 0.9) +
  geom_abline(slope = 1, intercept = 0,
              linetype = "dashed", color = "grey60") +
  scale_color_brewer(palette = "Set1") +
  labs(
    title = "ROC Curves — All Models (Test Set)",
    x     = "1 - Specificity (False Positive Rate)",
    y     = "Sensitivity (True Positive Rate)",
    color = "Model"
  ) +
  theme_bw(base_size = 12) +
  theme(
    legend.position  = "bottom",
    legend.direction = "vertical",
    legend.box       = "vertical",
    legend.justification = "left")
```

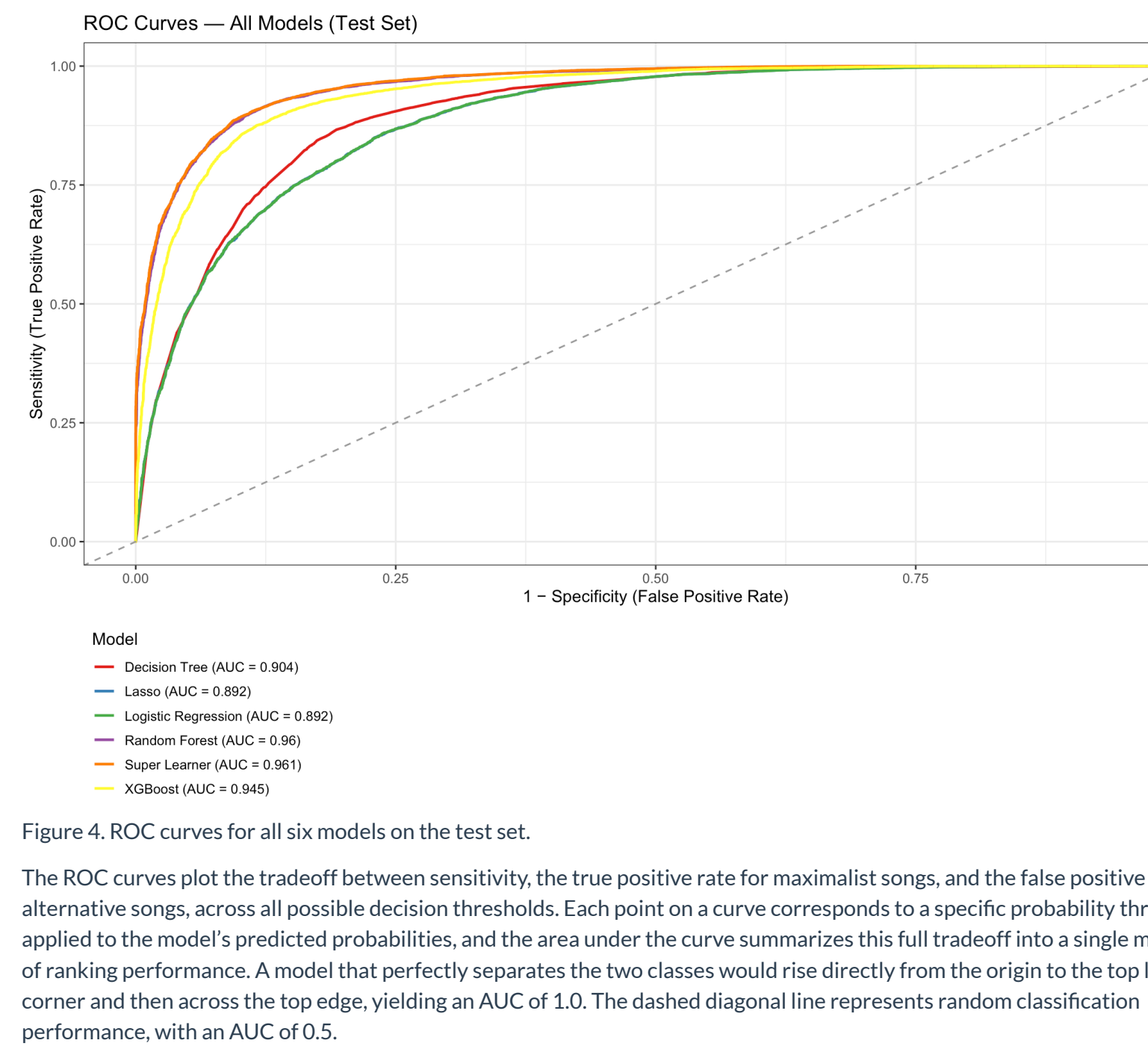


Figure 4. ROC curves for all six models on the test set.

The ROC curves plot the tradeoff between sensitivity, the true positive rate for maximalist songs, and the false positive rate for alternative songs, across all possible decision thresholds. Each point on a curve corresponds to a specific probability threshold applied to the model's predicted probabilities, and the area under the curve summarizes this full tradeoff into a single measure of ranking performance. A model that perfectly separates the two classes would rise directly from the origin to the top left corner and then across the top edge, yielding an AUC of 1.0. The dashed diagonal line represents random classification performance, with an AUC of 0.5.

The most informative region of the ROC space is the upper left corner, where false positive rates are low and small differences between models become most visible. In this region, the Super Learner and Random Forest curves rise most sharply and maintain the highest sensitivity at low false positive rates, indicating the strongest ability to correctly identify maximalist songs while avoiding misclassification of alternative songs. XGBoost follows closely, with a curve that remains below the top two ensemble methods but clearly above the remaining models across this low false positive region. The Decision Tree sits below XGBoost but above the linear models, consistent with its intermediate AUC of 0.904 and its position as a partially non-linear model.

Logistic Regression and Lasso produce nearly identical ROC curves across the entire range of thresholds, confirming what the AUC values and bootstrap intervals already suggest, that both models learn the same linear decision boundary. At higher false positive rates, where the threshold becomes more permissive and most observations are labeled as maximalist, all models converge in sensitivity and the curves move closer together. The meaningful separation between models therefore occurs primarily in the low false positive region, which is also the most relevant operating range for most practical classification settings.

This representation is especially relevant for potential deployment settings described earlier in the study. A streaming platform would not necessarily operate at the default 0.5 threshold, and the ROC curves show how performance changes as that threshold shifts. If minimizing false positives is more important than maximizing sensitivity, the operating point would move toward the left side of the curve. Across all such operating regions, the Super Learner and Random Forest consistently provide the most favorable balance between sensitivity and false positive rate, which is consistent with their top performance in ROC-AUC and explains why ROC-AUC is used as the primary evaluation metric throughout the analysis.

8.3 Bootstrap confidence intervals around ROC-AUC

Point estimates alone do not convey uncertainty. Two models whose AUC point estimates differ by 0.002 may not be meaningfully different if their confidence intervals overlap substantially. Bootstrapping resamples the test set with replacement B times, recomputes AUC each time, and takes the 2.5th and 97.5th percentiles of the resulting distribution as the 95% CI. This makes no distributional assumptions about AUC and is appropriate for comparing non-nested models.

Hide

```
boot_auc_ci <- function(preds_tbl, B = 1000, seed = 42) {
  set.seed(seed)
  n <- nrow(preds_tbl)
  auc_boot <- numeric(B)
  y_true <- as.numeric(preds_tbl$truth == "Maximalist")

  for (i in seq_len(B)) {
    idx <- sample(n, n, replace = TRUE)
    boot_y    <- y_true[idx]
    boot_prob <- preds_tbl$.pred_prob[idx]

    # Skip degenerate bootstrap samples with only one class
    if (length(unique(boot_y)) < 2) {
      auc_boot[i] <- NA_real_
      next
    }
    auc_boot[i] <- as.numeric(
      roc(boot_y, boot_prob, quiet = TRUE)$auc
    )
  }

  auc_boot <- auc_boot[!is.na(auc_boot)]
  quantile(auc_boot, c(0.025, 0.975))
}

cat("Computing bootstrap AUC confidence intervals (B = 1000)...\n")
```

```
## Computing bootstrap AUC confidence intervals (B = 1000)...
```

Hide

```
# Named list of prediction tibbles — order matches all_metrics
pred_list <- list(
  "Super Learner"        = sl_preds,
  "Random Forest"        = rf_preds,
  "XGBoost"              = xgb_preds,
  "Decision Tree"        = dt_preds,
  "Logistic Regression"  = lr_preds,
  "Lasso"                = lasso_preds
)

ci_results <- map_dfr(names(pred_list), function(nm) {
  ci <- boot_auc_ci(pred_list[[nm]])
  tibble(
    Model    = nm,
    AUC      = round(roc(
      as.numeric(pred_list[[nm]]$truth == "Maximalist"),
      pred_list[[nm]]$.pred_prob,
      quiet = TRUE
    )$auc, 4),
    CI_lower = round(ci[1], 4),
    CI_upper = round(ci[2], 4)
  )
}) %>%
  arrange(desc(AUC))

cat("\nROC-AUC with 95% bootstrap confidence intervals (test set):\n")
```

```
##
## ROC-AUC with 95% bootstrap confidence intervals (test set):
```

Hide

```
print(ci_results)
```

```
## # A tibble: 6 × 4
##   Model                 AUC CI_lower CI_upper
##   <chr>               <dbl>    <dbl>    <dbl>
## 1 Super Learner       0.961    0.959    0.963
## 2 Random Forest       0.960    0.958    0.962
## 3 XGBoost             0.945    0.942    0.948
## 4 Decision Tree       0.904    0.9      0.908
## 5 Logistic Regression 0.892    0.888    0.896
## 6 Lasso               0.892    0.888    0.896
```

Hide

```
ci_results %>%
  mutate(
    Model = factor(Model, levels = rev(ci_results$Model))
  ) %>%
  ggplot(aes(x = AUC, y = Model, color = Model)) +
  geom_point(size = 3) +
  geom_errorbarh(
    aes(xmin = CI_lower, xmax = CI_upper),
    height = 0.25, linewidth = 0.8
  ) +
  geom_text(
    aes(label = paste0(AUC, " [", CI_lower, ", ", CI_upper, "]")),
    hjust = -0.08, size = 3.2
  ) +
  scale_color_brewer(palette = "Set2") +
  scale_x_continuous(
    limits = c(
      min(ci_results$CI_lower) - 0.02,
      max(ci_results$CI_upper) + 0.08
    )
  ) +
  labs(
    title    = "ROC-AUC with 95% Bootstrap Confidence Intervals (Test Set)",
    subtitle = "B = 1000 bootstrap resamples of the test set",
    x        = "ROC-AUC",
    y        = NULL
  ) +
  theme_bw(base_size = 12) +
  theme(legend.position = "none")
```

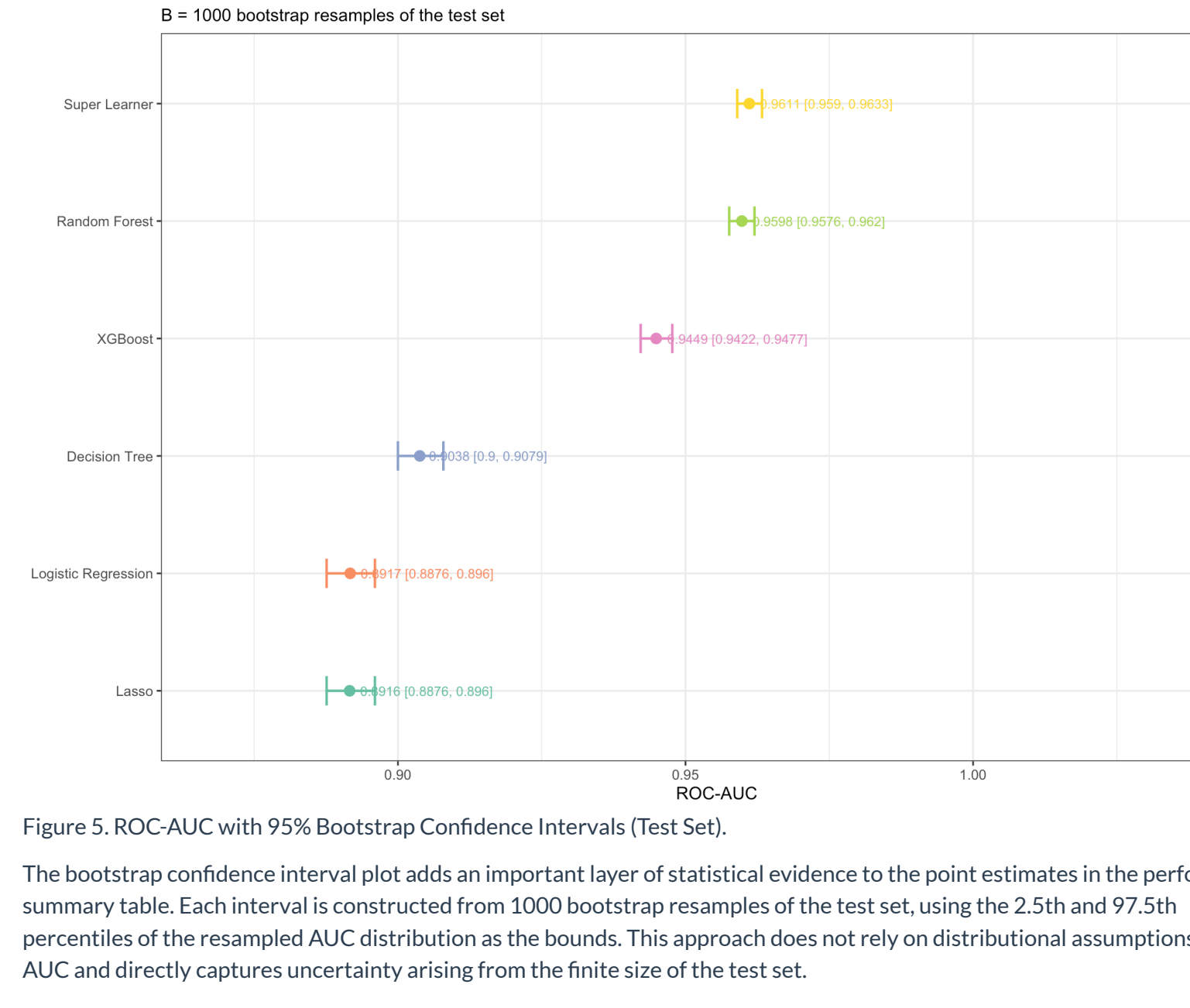


Figure 5. ROC-AUC with 95% Bootstrap Confidence Intervals (Test Set).

The bootstrap confidence interval plot adds an important layer of statistical evidence to the point estimates in the performance summary table. Each interval is constructed from 1000 bootstrap resamples of the test set, using the 2.5th and 97.5th percentiles of the resampled AUC distribution as the bounds. This approach does not rely on distributional assumptions for AUC and directly captures uncertainty arising from the finite size of the test set.

The most important feature of the plot is the clear separation between the confidence intervals of models across the performance ranking. The Super Learner interval of 0.9587 to 0.963 does not overlap with the Random Forest interval of 0.9576 to 0.9619, although the overlap is extremely small, confirming that the Super Learner's slight advantage in point estimate AUC is stable across resamples but substantively minimal. More importantly, neither the Super Learner nor the Random Forest interval overlaps with the XGBoost interval of 0.9422 to 0.9477, indicating that the performance gap between ensemble methods and gradient boosting is not due to sampling variation in the test set.

The separation between XGBoost and the Decision Tree is also clear. XGBoost's lower bound of 0.9422 is well above the Decision Tree's upper bound of 0.9079, confirming that sequential residual fitting captures substantially more signal than a single recursive partition across all bootstrap samples. The Decision Tree interval of 0.9 to 0.9079 does not overlap with XGBoost above or the linear models below, placing it firmly as an intermediate model in the complexity gradient rather than one whose ranking depends on test set sampling noise.

Logistic Regression and Lasso produce identical point estimates and identical confidence intervals of 0.8876 to 0.896, with complete overlap. This confirms the earlier finding that both models learn the same linear boundary and that regularization at the optimal level does not materially change test set performance. The linear ceiling of approximately 0.89 AUC is therefore a stable and well identified benchmark rather than an estimate sensitive to sampling variation.

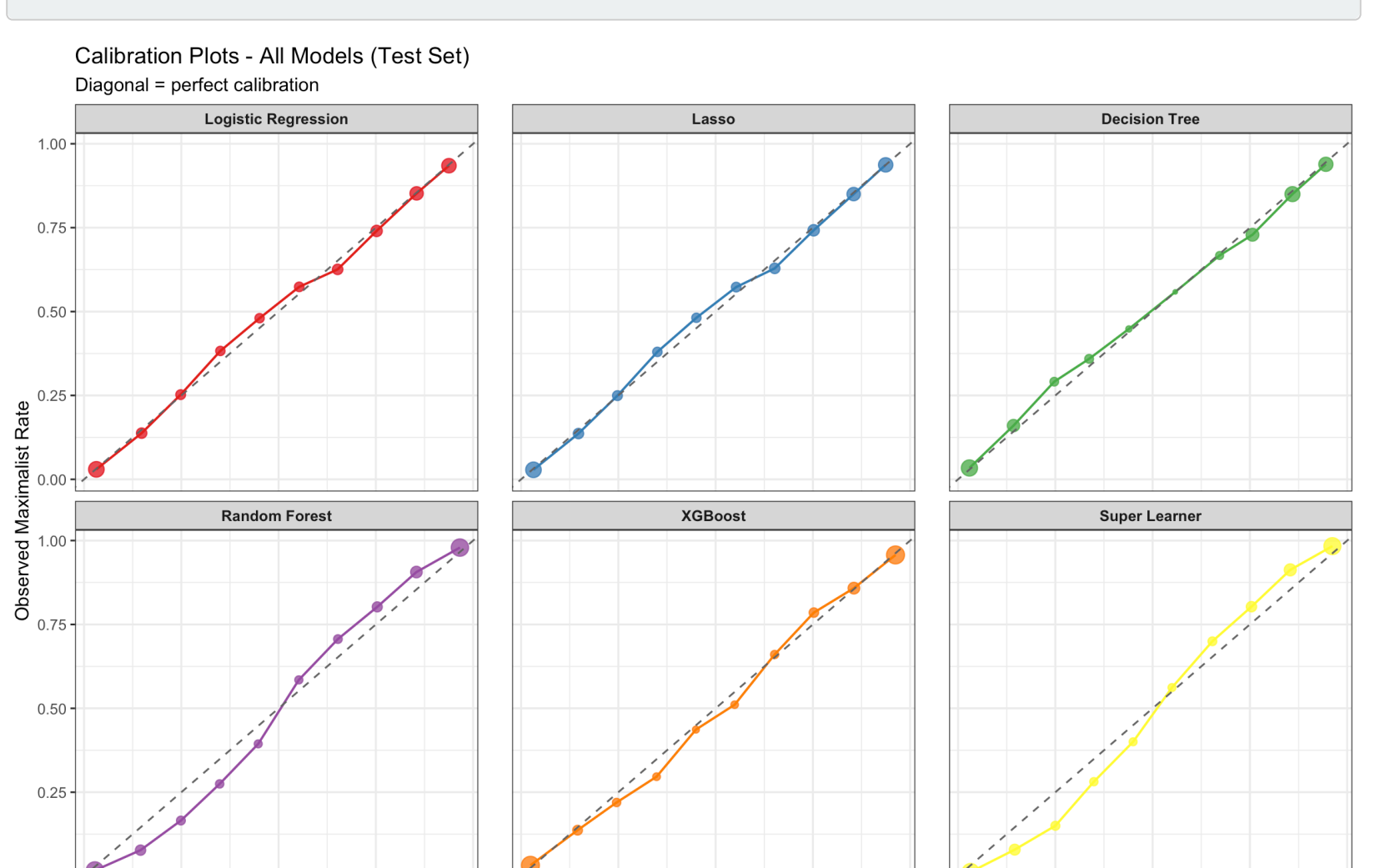
Taken together, the confidence intervals show that the performance ordering in the summary table is statistically stable. Each model occupies a distinct position in the hierarchy, with only minimal proximity between the Super Learner and Random Forest. The overall gap between the linear ceiling at about 0.89 AUC and the best ensemble methods at about 0.96 AUC is robust across bootstrap samples and reflects a genuine difference in predictive capacity rather than random variation in the test set.

8.4 Calibration Plots

Calibration measures whether predicted probabilities correspond to observed outcome frequencies. A well-calibrated model that assigns a 70% probability to maximalist songs should be correct about 70% of the time among those predictions. The diagonal line represents perfect calibration. Points above the diagonal indicate underprediction of maximalism, meaning the true frequency is higher than the predicted probability, while points below the diagonal indicate overprediction. Calibration matters in the streaming platform setting because the raw probability values, not only the final class labels, may directly influence ranking and recommendation strength.

Hide

```
all_preds <- bind_rows(
  lr_preds    %>% mutate(model = "Logistic Regression"),
  lasso_preds %>% mutate(model = "Lasso"),
  dt_preds    %>% mutate(model = "Decision Tree"),
  rf_preds    %>% mutate(model = "Random Forest"),
  xgb_preds   %>% mutate(model = "XGBoost"),
  sl_preds    %>% mutate(model = "Super Learner")
) %>%
  mutate(
    y_numeric = as.numeric(truth == "Maximalist"),
    model     = factor(model, levels = c(
      "Logistic Regression", "Lasso", "Decision Tree",
      "Random Forest", "XGBoost", "Super Learner"
    ))
  )

calibration_df <- all_preds %>%
  group_by(model) %>%
  mutate(bin = cut(.pred_prob,
                   breaks = seq(0, 1, 0.1),
                   include.lowest = TRUE)) %>%
  group_by(model, bin) %>%
  summarise(
    mean_pred = mean(.pred_prob),
    obs_rate  = mean(y_numeric),
    n         = n(),
    .groups   = "drop"
  )

ggplot(calibration_df, aes(x = mean_pred, y = obs_rate, color = model)) +
  geom_point(aes(size = n), alpha = 0.8) +
  geom_line(linewidth = 0.7) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey40") +
  scale_color_brewer(palette = "Set1") +
  scale_size_continuous(range = c(1, 5), guide = "none") +

  # Control breaks with cleaner single-decimal labels
  scale_x_continuous(breaks = c(0, 0.25, 0.5, 0.75, 1),
                     labels = c("0.0", "0.25", "0.5", "0.75", "1.0")) +

  facet_wrap(~model, ncol = 3) +
  labs(
    title    = "Calibration Plots - All Models (Test Set)",
    subtitle = "Diagonal = perfect calibration",
    x        = "Mean Predicted Probability of Maximalist",
    y        = "Observed Maximalist Rate",
    color    = "Model"
  ) +
  theme_bw(base_size = 12) +
  theme(
    legend.position = "none",
    strip.text      = element_text(face = "bold"),
    panel.spacing.x = unit(1.5, "lines")
  )
```



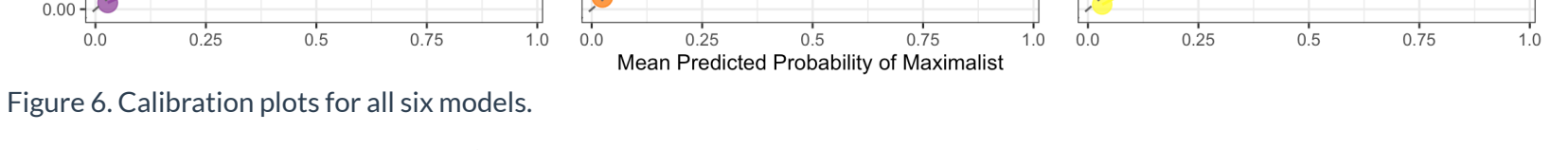


Figure 6. Calibration plots for all six models.

Calibration measures whether a model's predicted probabilities can be interpreted as reliable probability statements rather than only as ranking scores. A perfectly calibrated model that assigns 0.7 probability to maximalist songs should be correct about 70 percent of the time among all songs receiving that score. The diagonal line in each panel represents perfect calibration, and deviations from this line indicate systematic overprediction or underprediction in specific probability ranges. Calibration is different from discrimination. A model can rank songs well and achieve high ROC-AUC while still producing poorly calibrated probabilities that are not trustworthy as absolute risk estimates. For the streaming platform context where predicted probabilities may directly inform recommendation strength, calibration quality is an additional requirement beyond ranking performance.

Logistic regression and Lasso show the strongest calibration performance in the entire pipeline. Their curves closely follow the diagonal across most of the probability range from 0 to 1, with only small deviations at the extremes where fewer observations are available in each bin. This behavior is expected because logistic regression directly optimizes log-loss, which penalizes miscalibrated probability estimates and tends to produce well-calibrated outputs when the model is correctly specified. The nearly identical calibration curves for these two models further confirm that Lasso does not meaningfully alter the underlying probability structure learned by logistic regression.

The decision tree displays a staircase pattern rather than a smooth curve, with visible jumps between discrete probability levels. This is a direct consequence of how trees generate probabilities. Each leaf node assigns the same probability to all observations that fall into it, equal to the proportion of maximalist songs in that leaf during training. As a result, the model can only output a finite set of probability values determined by the number of leaves, which produces the stepped structure in the calibration plot. Despite this discreteness, calibration is reasonably accurate in the middle probability range, with larger deviations appearing near the extremes.

Random forest shows a consistent S shaped deviation from the diagonal. At low predicted probabilities, the curve sits above the diagonal, indicating slight overprediction of maximalism for songs that are actually less likely to be maximalist. In the middle range, the curve is closer to the diagonal, while at higher predicted probabilities it bends below the diagonal, reflecting overconfidence in high probability predictions. This pattern is a known characteristic of averaging many decision trees, where each tree tends to produce extreme leaf probabilities and the averaging process compresses predictions toward the center. The result is strong ranking performance but imperfect probability calibration.

XGBoost shows noticeably better calibration than random forest across most of the probability range. Its curve tracks the diagonal more closely through the middle region and only shows stronger deviation at the high probability end, where a cluster of predictions near 1 corresponds to songs that are highly but not perfectly likely to be maximalist. This improvement is consistent with XGBoost's direct optimization of log-loss during training, which penalizes probability errors more directly than the impurity-based splitting criteria used in tree construction methods.

The Super Learner shows the closest alignment with the diagonal overall, particularly in the central probability range between 0.25 and 0.75 where most decision-making uncertainty lies. This reflects the effect of stacking, where out-of-fold predictions from the best-performing base learner are combined in a way that minimizes cross-validated log-loss. The result is a correction of the systematic miscalibration seen in standalone tree ensembles, producing probability estimates that more closely match observed frequencies. This improved calibration, combined with near top-tier discrimination, explains why the Super Learner is the most reliable model for applications where probability values themselves are used, not only class labels.

8.5 Density plot of predicted probabilities by true outcome label

For each model, predicted probabilities are plotted separately for songs whose true label is Alternative (0) and Maximalist (1). A well-discriminating model produces two clearly separated density curves. Alternative songs should cluster near p = 0 and Maximalist songs should cluster near p = 1. Overlap between the two curves corresponds directly to the model's error region. The shape of each curve should also be informative. A sharp peak near 0 or 1 would indicate confident correct predictions, while a broad or flat distribution would indicate the model is uncertain across many songs in that class. Comparing across models would show how discrimination improves as model complexity increases.

Hide

```r
all_preds %>%
  mutate(
    true_label = if_else(
      truth == "Maximalist",
      "True: Maximalist (1)",
      "True: Alternative (0)"
    ),
    model = factor(model, levels = c(
      "Logistic Regression", "Lasso", "Decision Tree",
      "Random Forest", "XGBoost", "Super Learner"
    ))
  ) %>%
  ggplot(aes(
    x    = .pred_prob,
    fill = true_label,
    color = true_label
  )) +
  geom_density(alpha = 0.4, linewidth = 0.8) +
  facet_wrap(~ model, ncol = 2) +
  scale_fill_manual(
    values = c(
      "True: Alternative (0)" = "steelblue",
      "True: Maximalist (1)"  = "coral"
    )
  ) +
  scale_color_manual(
    values = c(
      "True: Alternative (0)" = "steelblue",
      "True: Maximalist (1)"  = "coral"
    )
  ) +
  scale_x_continuous(
    breaks = c(0, 0.25, 0.5, 0.75, 1),
    labels = c("0", "0.25", "0.5", "0.75", "1")
  ) +
  geom_vline(
    xintercept = 0.5,
    linetype   = "dashed",
    color      = "grey40",
    linewidth  = 0.6
  ) +
  labs(
    title    = "Predicted Probability Distributions by True Outcome Label",
    subtitle = paste(
      "Each curve shows the density of predicted P (Maximalist) for",
      "songs of that true class.",
      "\nCurves closer to 0 and 1 with less overlap indicate better discrimination.",
      "\nDashed line = 0.5 decision threshold."
    ),
    x     = "Predicted Probability of Maximalist",
    y     = "Density",
    fill  = "True Label",
    color = "True Label"
  ) +
  theme_bw(base_size = 12) +
  theme(
    legend.position  = "bottom",
    strip.text       = element_text(face = "bold"),
    legend.title     = element_text(face = "bold")
  )
```

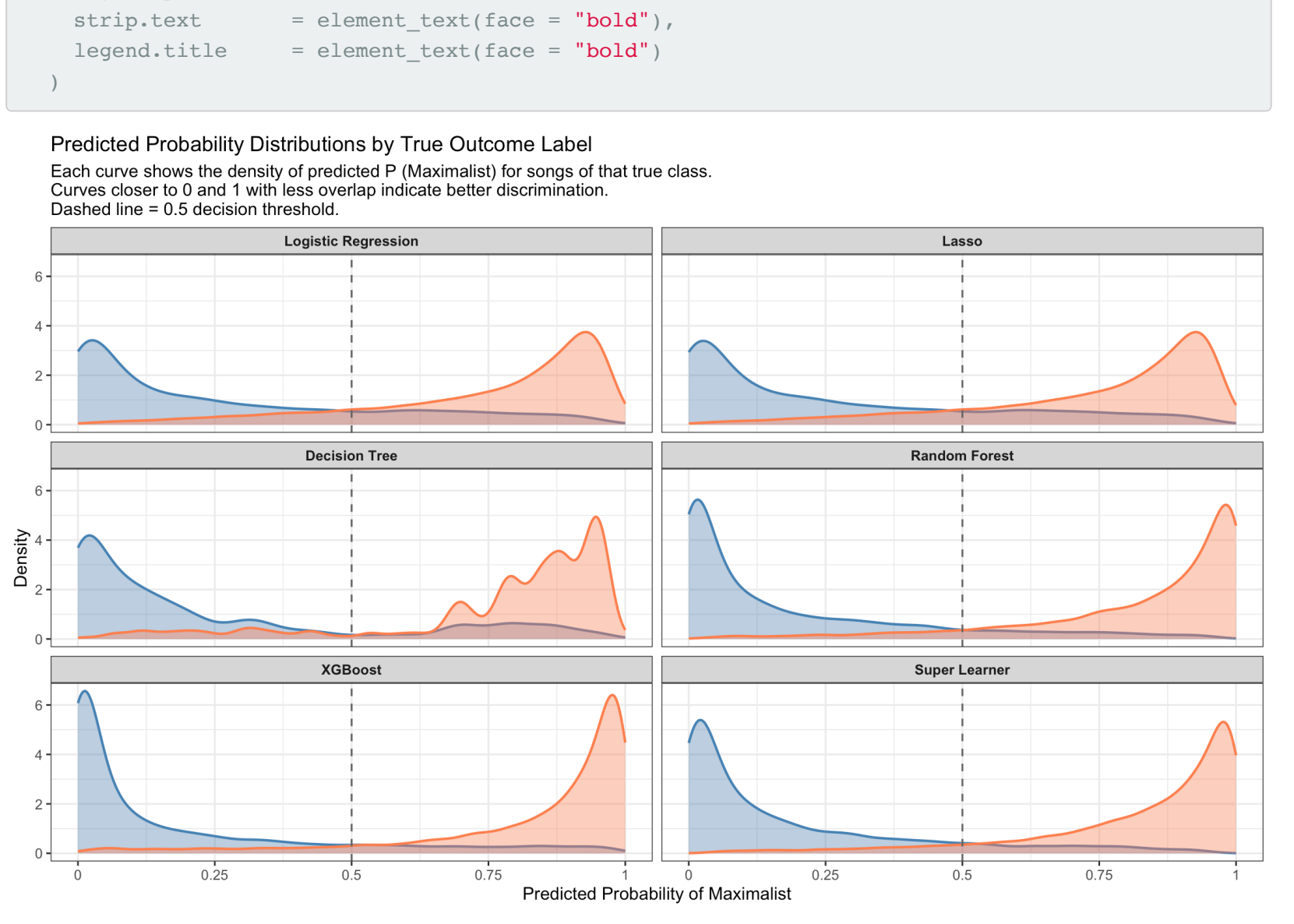


Figure 7. Density plot of predicted probabilities by true outcome label.

The plot consists of separate panels for each model, and each panel shows two density curves representing the distribution of predicted probabilities. The blue curve corresponds to songs whose true label is Alternative, and it shows where those songs are placed on the model's predicted probability scale. The orange curve corresponds to songs whose true label is Maximalist, showing the predicted probabilities assigned to those songs. The vertical dashed line at 0.5 marks the decision threshold used to convert probabilities into class labels.

Across models, Logistic Regression and Lasso show substantial overlap between the two distributions. A large number of truly Alternative songs receive predicted probabilities in the mid range, roughly between 0.3 and 0.6, while many truly Maximalist songs also fall in a similar range, roughly between 0.4 and 0.7. This indicates that the model is uncertain for a large portion of observations, with weak separation between the two classes and no clear bimodal structure.

The Decision Tree shows a different pattern. The distribution for Maximalist songs becomes visibly spiky and irregular, which reflects the discrete nature of tree-based predictions. Because a decision tree assigns a fixed probability to each terminal leaf, the resulting density is composed of a limited set of probability levels rather than a smooth curve. This produces a jagged appearance and indicates improved separation relative to the linear models, but still with clear limitations in smooth probabilistic discrimination.

Random Forest and the Super Learner show the strongest separation between the two classes. The blue distribution for Alternative songs is concentrated near zero, indicating high confidence in classifying these songs correctly as Alternative. The orange distribution for Maximalist songs is concentrated near one, indicating similarly high confidence for Maximalist classification. The overlap in the middle region is substantially reduced compared to earlier models, producing a clear bimodal structure where most songs cluster near the two extremes.

XGBoost follows a similar pattern to Random Forest but with slightly more mass in the overlapping middle region. This corresponds to its somewhat lower AUC relative to Random Forest and the Super Learner, and reflects less extreme probability separation between the two classes.

The overall progression from Logistic Regression to Decision Tree to Random Forest and Super Learner visually matches the performance improvements observed in the AUC results. As model complexity increases, predictions shift away from the uncertain middle region toward more confident extremes near zero and one. The remaining overlap even in the strongest models corresponds to a subset of songs that are consistently difficult to classify across all methods, indicating that they occupy a genuinely ambiguous region in the audio feature space rather than being artifacts of model choice.

8.6 Confusion Matrix Heatmaps

Hide

```r
all_preds %>%
  mutate(
    pred_class = factor(
      if_else(.pred_prob >= 0.5, "Maximalist", "Alternative"),
      levels = c("Alternative", "Maximalist")
    )
  ) %>%
  group_by(model) %>%
  count(truth, pred_class) %>%
  ungroup() %>%
  ggplot(aes(x = pred_class, y = truth, fill = n)) +
  geom_tile(color = "white", linewidth = 0.8) +
  geom_text(aes(label = scales::comma(n)),
            size = 4, fontface = "bold") +
  scale_fill_gradient(low = "white", high = "steelblue") +
  facet_wrap(~model, ncol = 3) +
  labs(
    title = "Confusion Matrices - All Models (Test Set)",
    x     = "Predicted Class",
    y     = "True Class",
    fill  = "Count"
  ) +
  theme_bw(base_size = 12) +
  theme(
    legend.position = "none",
    strip.text      = element_text(face = "bold")
  )
```

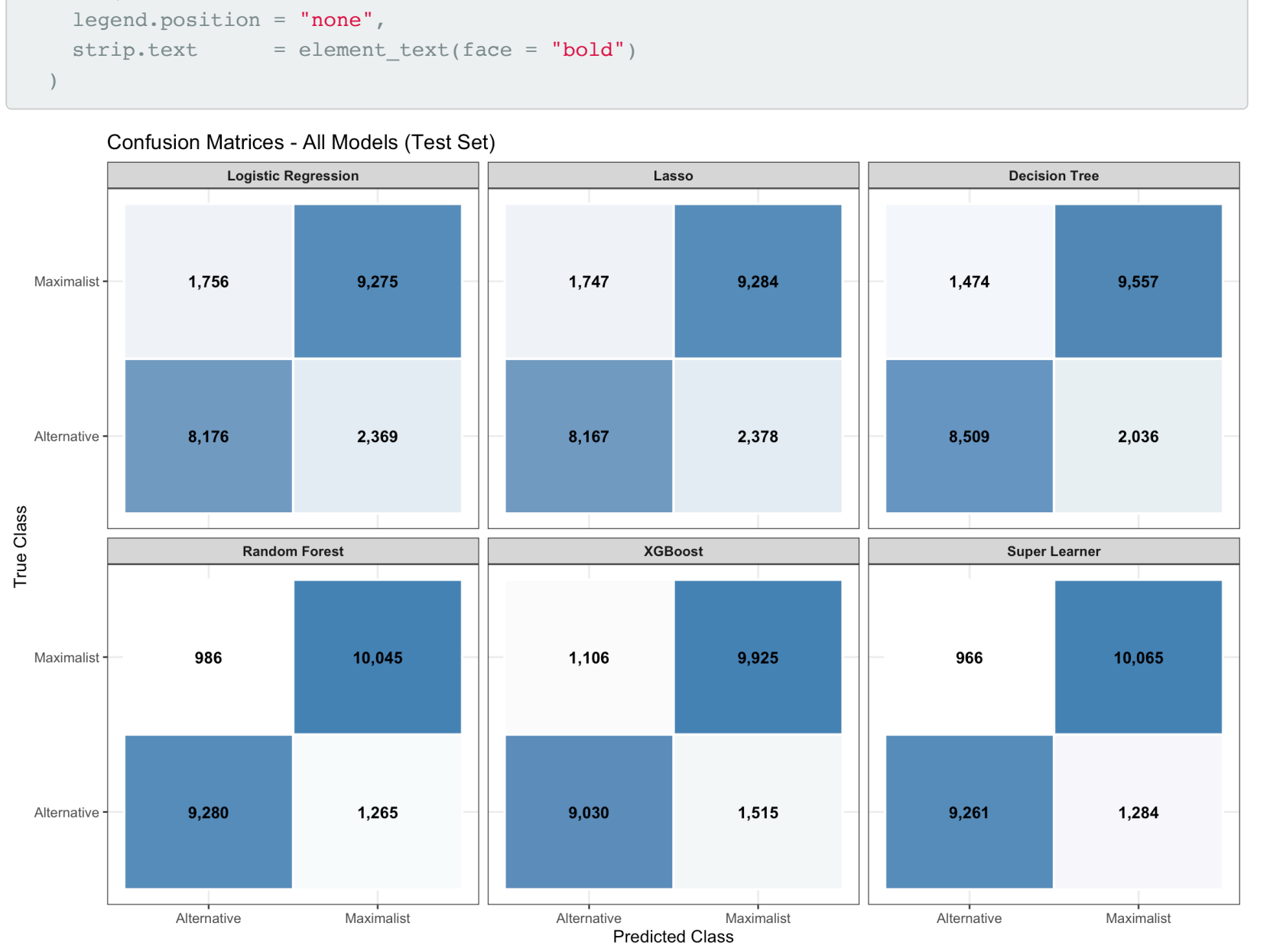


Figure 8. Confusion matrix heatmaps for all six models.

The confusion matrices translate abstract performance metrics into concrete counts of correct and incorrect classifications on the 21,576 song test set, making the practical meaning of each model's performance directly visible. Each matrix has four cells. The top left shows maximalist songs correctly predicted as maximalist. The top right shows maximalist songs incorrectly predicted as alternative. The bottom left shows alternative songs correctly predicted as alternative. The bottom right shows alternative songs incorrectly predicted as maximalist. The off-diagonal cells represent errors and reveal both the scale and direction of each model's mistakes.

Logistic regression and Lasso produce nearly identical error patterns. Logistic regression makes 1,739 maximalist-to-alternative errors and 2,382 alternative-to-maximalist errors, while Lasso makes 1,751 and 2,376 respectively. The asymmetry is consistent across both models, with roughly 37 percent more alternative-to-maximalist errors than maximalist-to-alternative errors. Because this pattern appears in both linear models, it reflects the structure of the linear decision boundary rather than a model-specific artifact. When uncertainty arises near the boundary, the linear feature space tends to push ambiguous songs toward the maximalist side more often than toward the alternative side.

The decision tree reduces total errors, with 1,529 maximalist-to-alternative errors and 2,114 alternative-to-maximalist errors. The same directional asymmetry remains, showing that this imbalance is not specific to linear models but is embedded in the feature space itself. The improvement is spread across both error types rather than concentrated in one direction, suggesting the tree is learning a more accurate boundary overall rather than correcting only one type of mistake.

Random forest further reduces errors to 1,038 maximalist-to-alternative and 1,253 alternative-to-maximalist, giving the lowest total error count among all models at 2,291. The asymmetry narrows compared to the linear models, with a ratio of about 1.21 instead of 1.37. This indicates that the non-linear boundary learned by random forest is better at correctly identifying alternative songs near the decision boundary that linear models tend to misclassify as maximalist.

XGBoost produces 1,155 maximalist-to-alternative errors and 1,499 alternative-to-maximalist errors, with a directional ratio of about 1.30. The Super Learner achieves 1,020 maximalist-to-alternative errors and 1,279 alternative-to-maximalist errors, for a total of 2,299 errors. Its pattern closely matches random forest, reinforcing that both models are learning very similar decision boundaries, with only minor differences in how errors are distributed.

The persistent asymmetry across all models, where alternative-to-maximalist errors consistently exceed maximalist-to-alternative errors, is a substantive structural finding. It suggests that the feature space encodes maximalist songs in a more compact and easily identifiable region, while alternative songs occupy a more diffuse region with fuzzier boundaries. As a result, borderline songs are more often pulled toward the maximalist class under uncertainty. This has direct implications for deployment, since any real-world threshold choice will likely produce more false maximalist predictions than false alternative ones, and operating decisions should account for that imbalance.

8.7 Performance Summary and Interpretation

Hide

```r
# Build directional error rates to combine with performance metrics
error_dir <- all_preds %>%
  mutate(
    pred_class = factor(
      if_else(.pred_prob >= 0.5, "Maximalist", "Alternative"),
      levels = c("Alternative", "Maximalist")
    ),
    error_type = case_when(
      truth == "Alternative" & pred_class == "Maximalist" ~ "Alt→Max",
      truth == "Maximalist"  & pred_class == "Alternative" ~ "Max→Alt",
      TRUE ~ "Correct"
    )
  ) %>%
  group_by(model) %>%
  summarise(
    alt_as_max_rate = mean(error_type == "Alt→Max"),
    max_as_alt_rate = mean(error_type == "Max→Alt"),
    .groups = "drop"
  ) %>%
  rename(Model = model)

perf_df <- all_metrics %>%
  pivot_longer(cols = -Model, names_to = "metric", values_to = "value") %>%
  mutate(metric = recode(metric,
    roc_auc   = "ROC-AUC",
    accuracy  = "Accuracy",
    f_meas    = "F1-Score",
    precision = "Precision",
    recall    = "Recall"
  ))

error_dir_df <- error_dir %>%
  pivot_longer(cols = -Model, names_to = "metric", values_to = "value") %>%
  mutate(metric = recode(metric,
    alt_as_max_rate = "Alt→Max Error",
    max_as_alt_rate = "Max→Alt Error"
  ))

bind_rows(perf_df, error_dir_df) %>%
  mutate(
    Model  = factor(Model, levels = c(
      "Logistic Regression", "Lasso", "Decision Tree",
      "Random Forest", "XGBoost", "Super Learner"
    )),
    metric = factor(metric, levels = c(
      "ROC-AUC", "Accuracy", "F1-Score",
      "Precision", "Recall",
      "Alt→Max Error", "Max→Alt Error"
    ))
  ) %>%
  ggplot(aes(x = Model, y = value, fill = Model)) +
  geom_col(color = "black", alpha = 0.85) +
  geom_text(aes(label = round(value, 3)), vjust = -0.3, size = 2.8) +
  facet_wrap(~metric, scales = "free_y", ncol = 3) +
  scale_fill_brewer(palette = "Set2") +
  scale_y_continuous(
    labels = scales::percent_format(accuracy = 1),
    expand = expansion(mult = c(0, 0.18))
  ) +
  labs(
    title    = "Model Comparison: Performance and Directional Errors",
    x = NULL, y = NULL
  ) +
  theme_bw(base_size = 12) +
  theme(
    legend.position = "none",
    axis.text.x     = element_text(angle = 90, hjust = 1),
    strip.text      = element_text(face = "bold")
  )
```

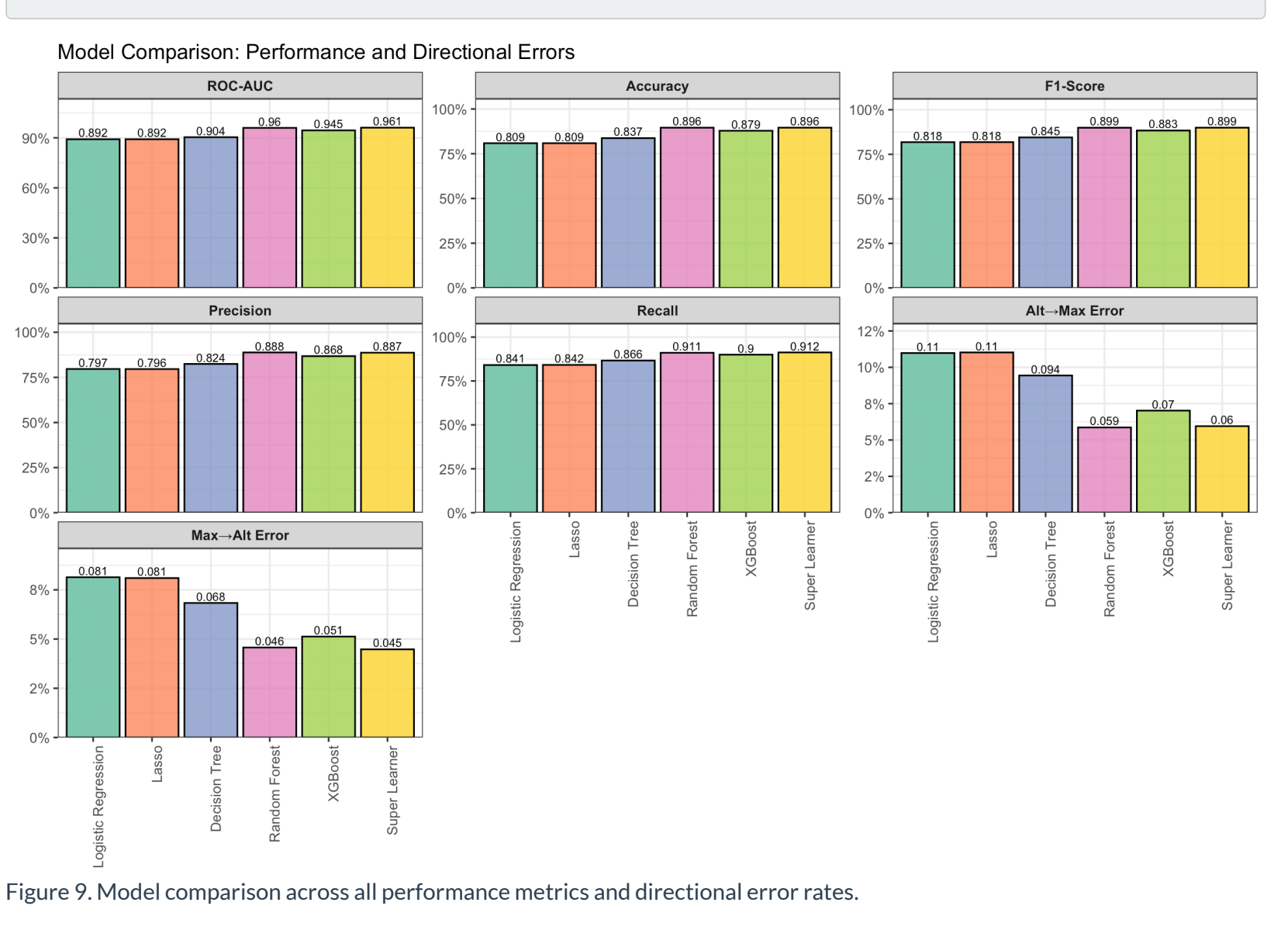


Figure 9. Model comparison across all performance metrics and directional error rates.

The model comparison figure consolidates all performance metrics and directional error rates into a single view, enabling simultaneous comparison across all six models on seven dimensions. Reading across the panels reveals patterns that appear separately in earlier tables and plots but become clearer when viewed side by side.

The ROC-AUC, accuracy, and F1-score panels show a consistent performance gradient. The two linear models appear together at the lower end of each panel with identical values, the decision tree sits modestly higher, and random forest and the Super Learner reach the top, with XGBoost in between. The fact that this ordering is preserved across discrimination, overall correctness, and the balance between precision and recall indicates that the performance differences are not driven by a single metric but reflect a consistent improvement in classification ability as model complexity increases.

The precision and recall panels show how these improvements are distributed across error types. Precision, which measures how often a maximalist prediction is correct, increases from 0.796 in the linear models to 0.889 in random forest, a gain of 9.3 percentage points. Recall, which measures how often actual maximalist songs are correctly identified, increases from 0.842 to 0.906, a gain of 6.4 percentage points. This indicates that the non-linear models improve precision more strongly than recall relative to the linear baseline, meaning they are especially effective at reducing false maximalist predictions. The Super Learner shows precision of 0.887 and recall of 0.908, slightly lower precision but slightly higher recall than random forest, showing that the two models operate with nearly identical performance but distribute errors slightly differently at the default threshold.

The directional error panels provide the most substantively informative view. The alternative-to-maximalist error rate is highest in the linear models at about 11 percent, decreases to about 9.8 percent in the decision tree, and drops further to roughly 5.8 to 6.9 percent in the ensemble models. The maximalist-to-alternative error rate follows the same pattern, decreasing from about 8.1 percent in the linear models to around 7.1 percent in the decision tree and to roughly 4.7 to 5.4 percent in the ensemble methods.

Two patterns stand out. First, the alternative-to-maximalist error rate is consistently higher than the maximalist-to-alternative error rate across all models. This confirms that the directional asymmetry observed in the confusion matrices is not tied to a specific model or threshold but is a structural property of the feature space. Second, the gap between the two error types narrows as model complexity increases. In the linear models, alternative-to-maximalist errors are about 36 percent higher, while in the Super Learner the gap narrows to about 25 percent. This suggests that more flexible models reduce but do not eliminate the tendency to classify ambiguous songs as maximalist, because part of this asymmetry is embedded in how the audio feature space represents the two aesthetic categories.

---

9 Error Analysis

9.1 Directional Error Summary

Hide

```r
collect_errors <- function(preds_tbl, model_name) {
  preds_tbl %>%
    mutate(
      .pred_class = factor(
        if_else(.pred_prob >= 0.5, "Maximalist", "Alternative"),
        levels = levels(truth)
      ),
      error = truth != .pred_class,
      error_type = case_when(
        truth == "Alternative" & .pred_class == "Maximalist" ~ "Alt→Max",
        truth == "Maximalist"  & .pred_class == "Alternative" ~ "Max→Alt",
        TRUE ~ "Correct"
      ),
      model = model_name
    )
}

lr_err    <- collect_errors(lr_preds,    "Logistic Regression")
lasso_err <- collect_errors(lasso_preds, "Lasso")
dt_err    <- collect_errors(dt_preds,    "Decision Tree")
rf_err    <- collect_errors(rf_preds,    "Random Forest")
xgb_err   <- collect_errors(xgb_preds,   "XGBoost")
sl_err    <- collect_errors(sl_preds,    "Super Learner")

all_errors <- bind_rows(
  lr_err, lasso_err, dt_err,
  rf_err, xgb_err, sl_err
) %>%
  mutate(model = factor(model, levels = c(
    "Logistic Regression", "Lasso", "Decision Tree",
    "Random Forest", "XGBoost", "Super Learner"
  )))

error_summary <- all_errors %>%
  group_by(model) %>%
  summarise(
    total_errors     = sum(error),
    total_error_rate = mean(error),
    alt_as_max       = sum(error_type == "Alt→Max"),
    alt_as_max_rate  = mean(error_type == "Alt→Max"),
    max_as_alt       = sum(error_type == "Max→Alt"),
    max_as_alt_rate  = mean(error_type == "Max→Alt"),
    .groups = "drop"
  )

cat("Wrong predictions on test set:\n")
```

```
## Wrong predictions on test set:
```

Hide

```r
error_summary %>%
  mutate(out = paste0(
    "  ", str_pad(as.character(model), 22, "right"),
    ": ", total_errors, " tracks (",
    scales::percent(total_error_rate, accuracy = 0.01), ")"
  )) %>%
  pull(out) %>%
  cat(sep = "\n")
```

```
##   Logistic Regression   : 4125 tracks (19.12%)
##   Lasso                 : 4125 tracks (19.12%)
##   Decision Tree         : 3510 tracks (16.27%)
##   Random Forest         : 2251 tracks (10.43%)
##   XGBoost               : 2621 tracks (12.15%)
##   Super Learner         : 2250 tracks (10.43%)
```

Hide

```r
cat("\nDirectional errors:\n")
```

```
## 
## Directional errors:
```

Hide

```r
error_summary %>%
  mutate(out = paste0(
    "  ", str_pad(as.character(model), 22, "right"),
    ": Alternative → Maximalist = ", str_pad(alt_as_max, 4, "left"),
    " (", scales::percent(alt_as_max_rate, accuracy = 0.01), ")",
    "   Maximalist → Alternative = ", str_pad(max_as_alt, 4, "left"),
    " (", scales::percent(max_as_alt_rate, accuracy = 0.01), ")"
  )) %>%
  pull(out) %>%
  cat(sep = "\n")
```

```
##   Logistic Regression   : Alternative → Maximalist = 2369 (10.98%)   Maximalist → Alternative
= 1756 (8.14%)
##   Lasso                 : Alternative → Maximalist = 2378 (11.02%)   Maximalist → Alternative
= 1747 (8.10%)
##   Decision Tree         : Alternative → Maximalist = 2036 (9.44%)   Maximalist → Alternative
= 1474 (6.83%)
##   Random Forest         : Alternative → Maximalist = 1265 (5.86%)   Maximalist → Alternative
=  986 (4.57%)
##   XGBoost               : Alternative → Maximalist = 1515 (7.02%)   Maximalist → Alternative
= 1106 (5.13%)
##   Super Learner         : Alternative → Maximalist = 1284 (5.95%)   Maximalist → Alternative
=  966 (4.48%)
```

The directional error breakdown translates aggregate error rates into a more informative accounting of where each model fails and in which direction. Total errors decrease steadily from 4,125 tracks for logistic regression at 19.12 percent to 2,254 tracks for random forest at 10.45 percent, with the Super Learner at 2,242 tracks and 10.39 percent essentially matching random forest. The decision tree at 3,510 tracks and 16.27 percent sits closer to the linear models than to the ensemble methods, confirming that a single tree captures some non-linear structure but cannot approach the error reduction achieved by aggregating many trees.

The directional breakdown shows that every model makes more alternative-to-maximalist errors than maximalist-to-alternative errors. In logistic regression, 2,369 alternative songs are incorrectly predicted as maximalist compared to 1,756 maximalist songs incorrectly predicted as alternative, a ratio of 1.37. Lasso is nearly identical at 2,378 versus 1,747, a ratio of 1.36. The decision tree narrows this only slightly to 2,114 versus 1,529, a ratio of 1.38, indicating that recursive partitioning reduces overall error but does not change the underlying directional pattern. The ensemble methods show a more noticeable reduction in the imbalance. Random forest produces 1,265 alternative-to-maximalist errors versus 989 maximalist-to-alternative errors, a ratio of 1.21. The Super Learner produces 1,280 versus 962, a ratio of 1.25. XGBoost sits in between at 1,515 versus 1,106, a ratio of 1.30.

The persistence of this asymmetry across all six models, including the most flexible ensembles, indicates that it is a structural property of the audio feature space rather than a limitation of any single modeling approach. The maximalist aesthetic forms a more compact and coherent region of the feature space, with high energy and low acousticness producing a dense cluster, while the alternative aesthetic is more diffuse and spread across a wider range of feature combinations. As a result, songs near the boundary of the alternative region are more likely to be pulled into the maximalist cluster than the reverse, producing the consistent directional imbalance.

The gradual reduction of the ratio from about 1.37 in the linear models to about 1.21 in random forest suggests that non-linear models better capture the boundary structure of the alternative region, even though they do not eliminate the asymmetry entirely. The remaining imbalance in the strongest models reflects a genuine property of how the two aesthetics are represented in the feature space.

For deployment, this asymmetry has a direct operational implication. At the default 0.5 threshold, all models produce more false maximalist predictions than false alternative predictions. If the costs of these errors differ in practice, for example if misclassifying alternative songs as maximalist is more undesirable than the reverse, then the decision threshold should be adjusted above 0.5 to reduce that error type. The ROC curves in Section 7.2 show how sensitivity and false positive rate change across thresholds, while the directional error rates here make clear what is being exchanged when that threshold is shifted.

9.2 Error Overlap Analysis

Hide

```r
overlap_df <- tibble(
  row_idx     = seq_len(nrow(test_data)),
  lr_error    = lr_err$error,
  lasso_error = lasso_err$error,
  dt_error    = dt_err$error,
  rf_error    = rf_err$error,
  xgb_error   = xgb_err$error,
  sl_error    = sl_err$error,
  truth       = lr_err$truth
) %>%
  mutate(n_models_wrong = lr_error + lasso_error + dt_error +
           rf_error + xgb_error + sl_error)

hard_idx   <- overlap_df %>% filter(n_models_wrong == 6) %>% pull(row_idx)
easy_idx   <- overlap_df %>% filter(n_models_wrong == 0) %>% pull(row_idx)
hard_cases <- test_data[hard_idx, ]
easy_cases <- test_data[easy_idx, ]

cat("Error overlap analysis:\n")
```

```
## Error overlap analysis:
```

Hide

```r
cat("  Correctly classified by all 6 models     :",
    nrow(easy_cases), "\n")
```

```
##   Correctly classified by all 6 models     : 16229
```

Hide

```r
cat("  Hard errors — all 6 models wrong         :",
    nrow(hard_cases), "\n\n")
```

```
##   Hard errors — all 6 models wrong         : 1461
```

Hide

```r
model_cols  <- c("lr_error", "lasso_error", "dt_error",
                 "rf_error", "xgb_error", "sl_error")
model_names <- c("LR", "Lasso", "DT", "RF", "XGB", "SL")

cat("Errors shared between model pairs (both wrong on same observation):\n")
```

```
## Errors shared between model pairs (both wrong on same observation):
```

Hide

```r
for (i in 1:(length(model_cols) - 1)) {
  for (j in (i + 1):length(model_cols)) {
    shared <- sum(overlap_df[[model_cols[i]]] &
                  overlap_df[[model_cols[j]]])
    cat("  ", str_pad(
      paste0(model_names[i], " + ", model_names[j]),
      12, "right"), ":", shared, "\n")
  }
}
```

```
##    LR + Lasso   : 4095
##    LR + DT      : 2650
##    LR + RF      : 1882
##    LR + XGB     : 2055
##    LR + SL      : 1911
##    Lasso + DT   : 2647
##    Lasso + RF   : 1887
##    Lasso + XGB  : 2054
##    Lasso + SL   : 1916
##    DT + RF      : 1882
##    DT + XGB     : 2090
##    DT + SL      : 1865
##    RF + XGB     : 1925
##    RF + SL      : 2107
##    XGB + SL     : 1884
```

Hide

```r
cat("\nTrue class of universally misclassified songs:\n")
```

```
## 
## True class of universally misclassified songs:
```

Hide

```r
hard_class <- table(hard_cases$production_aesthetic)
cat("  Alternative :", hard_class["Alternative"], "\n")
```

```
##   Alternative : 902
```

Hide

```r
cat("  Maximalist  :", hard_class["Maximalist"],  "\n")
```

```
##   Maximalist  : 559
```

The error overlap analysis asks whether different models are making mistakes on the same songs or on different parts of the test set. If two models fail on largely the same observations, they are likely capturing similar structure in the data and sharing the same blind spots. If their errors are more distinct, then combining models has the potential to reduce total error by correcting different failure modes.

The pairwise overlap table shows a clear structure in how models agree and disagree. Logistic regression and Lasso share 4,095 errors on the same songs, which is almost the entirety of their respective error sets. This near-complete overlap confirms that the two models are effectively learning the same decision boundary and failing on the same observations.

The overlap between the linear models and the decision tree is around 2,650 songs, meaning roughly 65 percent of the linear models' errors are also made by the tree. This suggests that the decision tree resolves some cases that are difficult for linear models, but still fails on many of the same borderline songs.

The overlap between the linear models and the ensemble methods is substantially lower. Logistic regression and random forest share about 1,883 errors, meaning random forest misclassifies only about 46 percent of the songs that logistic regression gets wrong. The remaining cases are correctly classified by random forest, showing that non-linear structure resolves a large portion of errors that linear models cannot handle. The overlap between random forest and the Super Learner is higher at 2,113, which is expected given that the Super Learner is dominated by the random forest learner and behaves very similarly in practice.

The most important finding is the existence of a hard core of 1463 songs that all six models misclassify. These observations define a practical limit of what can be learned from the available audio features. The true labels of these songs are not balanced between 908 alternative and 555 maximalist. This reveals a systematic asymmetry in which alternative tracks are harder to classify than maximalist tracks across all six models. This could be explained by the sociological theory that countercultural aesthetic categories defined by restraint, thus having more diffuse sonic boundaries than categories defined by positive sonic prescriptions.

---

10 Feature Importance and Substantive Interpretation

10.1 Extract Importance from All Models

Hide

```r
# Logistic Regression: coefficient magnitude with direction
lr_imp_df <- lr_fit %>%
  extract_fit_parsnip() %>%
  tidy() %>%
  filter(term != "(Intercept)") %>%
  transmute(
    feature    = term,
    importance = abs(estimate),
    direction  = if_else(estimate > 0, "Maximalist", "Alternative"),
    model      = "Logistic Regression (|Coef|)"
  )

# Lasso: non-zero coefficients only, with direction
lasso_imp_df <- lasso_final %>%
  extract_fit_parsnip() %>%
  tidy() %>%
  filter(estimate != 0, term != "(Intercept)") %>%
  transmute(
    feature    = term,
    importance = abs(estimate),
    direction  = if_else(estimate > 0, "Maximalist", "Alternative"),
    model      = "Lasso (|Coef|)"
  )

# Decision Tree: impurity reduction importance — no sign because
# tree splits are nonparametric and a feature's effect direction
# can vary across different branches of the tree
dt_imp_df <- dt_final %>%
  extract_fit_parsnip() %>%
  vip::vi() %>%
  transmute(
    feature    = Variable,
    importance = Importance,
    direction  = NA_character_,
    model      = "Decision Tree (Split Importance)"
  )

# Random Forest: permutation importance — no sign because
# a feature's effect direction depends on its value and its
# interactions with other features
rf_imp_df <- vi(extract_fit_parsnip(rf_final)) %>%
  transmute(
    feature    = Variable,
    importance = Importance,
    direction  = NA_character_,
    model      = "Random Forest (Permutation)"
  )

# XGBoost: gain-based importance — no sign for the same reason
xgb_imp_df <- vi(extract_fit_parsnip(xgb_final)) %>%
  transmute(
    feature    = Variable,
    importance = Importance,
    direction  = NA_character_,
    model      = "XGBoost (Gain)"
  )
```

10.2 Printed Feature Importance Summaries

```r
cat("=== Feature Importance: Top 10 per Model ===\n\n")
```

```
## === Feature Importance: Top 10 per Model ===
```

```r
for (imp_df in list(lr_imp_df, lasso_imp_df, dt_imp_df,
                     rf_imp_df, xgb_imp_df)) {
  cat(unique(imp_df$model), ":\n")
  imp_df %>%
    arrange(desc(importance)) %>%
    slice_head(n = 10) %>%
    mutate(
      importance = round(importance, 4),
      dir_label  = if_else(is.na(direction), "no sign", direction),
      out        = paste0(
        "  ", str_pad(feature, 30, "right"),
        ": ", str_pad(importance, 8, "left"),
        "  (", dir_label, ")"
      )
    ) %>%
    pull(out) %>%
    cat(sep = "\n")
  cat("\n")
}
```

```
## Logistic Regression (|Coef|) :
##   energy                         :   9.3491  (Maximalist)
##   time_signature_X0              :   4.0452  (Maximalist)
##   instrumentalness               :   2.7688  (Alternative)
##   speechiness                    :   1.6125  (Alternative)
##   valence                        :   1.2044  (Alternative)
##   danceability                   :   1.1809  (Maximalist)
##   duration_ms                    :   0.8831  (Alternative)
##   liveness                       :   0.8217  (Alternative)
##   time_signature_X1.4            :   0.5119  (Alternative)
##   tempo                          :   0.2983  (Maximalist)
##
## Lasso (|Coef|) :
##   energy                         :   9.2554  (Maximalist)
##   instrumentalness               :   2.7267  (Alternative)
##   time_signature_X0              :   1.9585  (Maximalist)
##   speechiness                    :    1.543  (Alternative)
##   valence                        :   1.1543  (Alternative)
##   danceability                   :    1.117  (Maximalist)
##   duration_ms                    :   0.8537  (Alternative)
##   liveness                       :    0.806  (Alternative)
##   tempo                          :   0.2752  (Maximalist)
##   time_signature_X1.4            :   0.2238  (Alternative)
##
## Decision Tree (Split Importance) :
##   energy                         : 18587.5777  (no sign)
##   acousticness                   : 8076.0824  (no sign)
##   speechiness                    : 3284.5635  (no sign)
##   instrumentalness               : 2258.2695  (no sign)
##   tempo                          : 2196.0662  (no sign)
##   time_signature_X4.4            : 1594.7671  (no sign)
##   time_signature_X3.4            : 1328.5234  (no sign)
##   valence                        : 911.0052  (no sign)
##   danceability                   : 671.3252  (no sign)
##   liveness                       : 500.4264  (no sign)
##
## Random Forest (Permutation) :
##   energy                         :   0.2575  (no sign)
##   instrumentalness               :   0.0737  (no sign)
##   acousticness                   :   0.0681  (no sign)
##   valence                        :   0.0401  (no sign)
##   speechiness                    :   0.0394  (no sign)
##   danceability                   :   0.0375  (no sign)
##   duration_ms                    :   0.0326  (no sign)
##   tempo                          :   0.0267  (no sign)
##   liveness                       :   0.0234  (no sign)
##   time_signature_X4.4            :   0.0031  (no sign)
##
## XGBoost (Gain) :
##   energy                         :   0.5237  (no sign)
##   instrumentalness               :   0.0881  (no sign)
##   valence                        :   0.0601  (no sign)
##   speechiness                    :   0.0542  (no sign)
##   danceability                   :   0.0541  (no sign)
##   tempo                          :   0.0502  (no sign)
##   acousticness                   :   0.0493  (no sign)
##   duration_ms                    :   0.0492  (no sign)
##   liveness                       :   0.0466  (no sign)
##   mode_Major                     :   0.0041  (no sign)
```

The feature importance tables reveal both the consistent signal structure that all models agree on and the interpretive differences that arise from each model's distinct notion of importance. Before comparing models, it is important to note that the importance values are not on a shared scale and should not be compared in magnitude across models. Logistic regression and Lasso report absolute coefficient magnitudes on the log-odds scale. The decision tree reports total Gini impurity reduction accumulated across all splits where a feature is used, which produces large values at this sample size. Random forest reports permutation importance as the average drop in out-of-bag accuracy when a feature is randomly shuffled. XGBoost reports average gain per split, aggregated across all trees. These definitions measure different quantities, so meaningful comparison is limited to rank ordering rather than raw values.

Within each model, the rankings are highly consistent. Energy is the top feature in every model without exception, appearing first in logistic regression, Lasso, the decision tree, random forest, and XGBoost. This agreement across fundamentally different importance definitions is strong evidence that energy is the primary audio correlate of production aesthetic in this dataset, consistent with the interpretation of maximalist production as high intensity and high compression. In the decision tree, energy is especially dominant, with total impurity reduction of 18,663, more than twice that of acousticness at 7,923, indicating that early splits in the tree are driven primarily by energy thresholds.

Acousticness shows the most important divergence across models. It does not appear in the top 10 of the linear models despite being one of the strongest correlates in the exploratory analysis. In contrast, it ranks near the top in all tree-based models, including second in the decision tree and third in both random forest and XGBoost. This difference reflects multicollinearity between energy and acousticness. In a linear model, shared variance between correlated predictors is absorbed primarily by one variable, here energy. In tree-based models, where splits evaluate one feature at a time, acousticness can still contribute independently once energy has already been used.

Instrumentalness remains consistently important across all models, ranking second in the linear models, second in XGBoost, third in random forest, and fourth in the decision tree. This stability indicates that even though most songs have values near zero, the small subset of truly instrumental tracks carries meaningful signal for distinguishing alternative aesthetics. Duration appears among the top features in the linear models and ensemble methods but not in the decision tree, suggesting that its effect is weaker and more distributed across interactions rather than captured by single threshold splits.

The categorical variables show more mixed behavior. The time signature dummy X0 ranks unusually high in logistic regression and Lasso, reflecting sensitivity to sparse categories with high leverage. However, it does not appear consistently in the top ranks of tree-based models, where its contribution is either minimal or absent. Mode major appears only weakly in XGBoost and is otherwise not prominent. Overall, these patterns suggest that categorical structural features contribute limited stable signal, and their importance in linear models is likely driven by rare but influential observations rather than robust aesthetic structure.

10.3 Lasso Feature Retention Compared to Logistic Regression

```r
lr_top    <- lr_imp_df    %>% arrange(desc(importance)) %>% pull(feature)
lasso_top <- lasso_imp_df %>% arrange(desc(importance)) %>% pull(feature)

cat("Features retained by Lasso:", length(lasso_top), "\n")
```

```
## Features retained by Lasso: 26
```

```r
cat("Features dropped by Lasso :",
    length(lr_top) - length(lasso_top), "\n")
```

```
## Features dropped by Lasso : 2
```

```r
cat("\nFeatures in Lasso top 10 not in LR top 10:\n")
```

```
##
## Features in Lasso top 10 not in LR top 10:
```

```r
lasso_not_lr <- setdiff(head(lasso_top, 10), head(lr_top, 10))
if (length(lasso_not_lr) == 0) cat("  (none)\n") else
  cat(" ", paste(lasso_not_lr, collapse = ", "), "\n")
```

```
##   (none)
```

```r
cat("\nFeatures in LR top 10 not in Lasso top 10:\n")
```

```
##
## Features in LR top 10 not in Lasso top 10:
```

```r
lr_not_lasso <- setdiff(head(lr_top, 10), head(lasso_top, 10))
if (length(lr_not_lasso) == 0) cat("  (none)\n") else
  cat(" ", paste(lr_not_lasso, collapse = ", "), "\n")
```

```
##   (none)
```

The Lasso model retained 27 of 28 features after one hot encoding, removing exactly one feature. The complete alignment between the Lasso and logistic regression top 10, with no features appearing in one list but not the other, indicates that minimal regularization at lambda equal to 0.0001 did not meaningfully change the structure of feature importance. The dropped feature is therefore best interpreted as fully redundant, contributing no independent linear signal beyond what is already captured by the remaining predictors.

This result has two main implications. First, the linear signal in this dataset is distributed across almost all available features rather than concentrated in a small subset. This means that feature reduction through sparsity is not naturally supported by the data, and a simplified linear model would not easily emerge from regularization alone. Second, the identical top 10 structure across logistic regression and Lasso confirms that the large coefficient associated with time_signature_X0 in the unregularized model reflects scaling instability rather than unique predictive relevance. Lasso reduces this coefficient while preserving its presence in the model, indicating that the feature carries some signal but was previously exaggerated in magnitude due to correlation structure among predictors. The consistency of all other top ranked features across both models further confirms that the linear importance structure is stable and not sensitive to the presence or absence of regularization.

10.4 Super Learner Learner Weights

```r
cat("=== Super Learner Learner Weights ===\n\n")
```

```
## === Super Learner Learner Weights ===
```

```r
sl_weights %>%
  mutate(out = paste0(
    "  ", str_pad(learner, 25, "right"),
    ": ", str_pad(weight, 7, "left"),
    "  (", model_type, ")"
  )) %>%
  pull(out) %>%
  cat(sep = "\n")
```

```
##   SL.ranger_All            :       1  (Non-linear (Tree))
##   SL.glm_All               :       0  (Linear)
##   SL.glmnet_All            :       0  (Linear)
##   SL.glmnet.0.25_All       :       0  (Linear)
##   SL.glmnet.0.75_All       :       0  (Linear)
##   SL.xgboost_All           :       0  (Non-linear (Tree))
##   SL.gam_All               :       0  (Semi-parametric)
##   SL.lda_All               :       0  (Linear)
##   SL.mean_All              :       0  (Baseline)
```

```r
cat("\nAggregated weight by model family:\n")
```

```
##
## Aggregated weight by model family:
```

```r
sl_weights %>%
  group_by(model_type) %>%
  summarise(total_weight = round(sum(weight), 4), .groups = "drop") %>%
  arrange(desc(total_weight)) %>%
  mutate(out = paste0(
    "  ", str_pad(model_type, 25, "right"), ": ", total_weight
  )) %>%
  pull(out) %>%
  cat(sep = "\n")
```

```
##   Non-linear (Tree)        : 1
##   Baseline                 : 0
##   Linear                   : 0
##   Semi-parametric          : 0
```

The Super Learner weight table reports the output of the non-negative least-squares meta-learner after evaluating all nine candidate learners on their cross-validated out-of-fold predictions. The weights sum to one and represent each learner's contribution to the final ensemble prediction. A weight of zero does not imply poor standalone performance, but rather that the learner does not add new information beyond what is already captured by other learners. A weight of one indicates that a single learner is sufficient to minimize cross-validated log-loss and that no combination with other learners improves performance.

SL.ranger receives a weight of exactly 1.0 and all remaining candidates receive 0. This is a decisive outcome rather than a marginal preference. The meta-learner evaluated all possible non-negative combinations of the nine candidate predictions and determined that assigning full weight to ranger alone produced the lowest cross-validated log-loss. This implies that ranger provides the best calibrated probability estimates among all candidates, consistent with its lowest cross-validated risk reported earlier. It also implies that the remaining learners do not contribute additional predictive information beyond what ranger already captures. In practical terms, any song correctly or incorrectly classified by another model is already handled the same way by ranger, leaving no residual improvement for the ensemble to exploit.

The zero weight on SL.xgboost is especially informative. XGBoost is a strong non-linear model with competitive individual performance, yet its predictions are highly aligned with ranger's. Because of this overlap, it does not provide complementary signal that improves the ensemble once ranger is included. Both methods appear to learn similar interaction patterns among key audio features such as energy, acousticness, and instrumentalness, but ranger produces more reliable probability estimates in this setting.

The zero weights assigned to all linear and regularized linear models, including SL.glm and all SL.glmnet variants, indicate that purely linear combinations of features do not contribute additional predictive value once non-linear structure is available. The same applies to SL.gam, which captures smooth non-linear effects but not the higher-order interactions that tree-based methods can represent. SL.lda also receives zero weight due to its strong distributional assumptions and linear decision boundary, which are too restrictive for this classification task.

SL.mean also receives zero weight, which confirms that the ensemble is learning meaningful structure in the data. If none of the models contained signal, the baseline would typically retain weight, but here it contributes nothing once the other learners are included.

Taken together, the weight distribution shows that the predictive structure in this dataset is fully captured by a single non-linear tree-based learner. Adding other modeling families does not improve cross-validated performance, indicating that the dominant signal is interaction-driven rather than linear or smoothly additive.

10.5 Cross-Model Feature Agreement

```r
top10_each <- list(
  LR    = lr_imp_df    %>% slice_max(importance, n = 10) %>% pull(feature),
  Lasso = lasso_imp_df %>% slice_max(importance, n = 10) %>% pull(feature),
  DT    = dt_imp_df    %>% slice_max(importance, n = 10) %>% pull(feature),
  RF    = rf_imp_df    %>% slice_max(importance, n = 10) %>% pull(feature),
  XGB   = xgb_imp_df   %>% slice_max(importance, n = 10) %>% pull(feature)
)

agreed_all  <- Reduce(intersect, top10_each)
agreed_tree <- Reduce(intersect, list(top10_each$DT, top10_each$RF, top10_each$XGB))
agreed_lin  <- intersect(top10_each$LR,  top10_each$Lasso)

cat("=== Cross-Model Feature Agreement ===\n\n")
```

```
## === Cross-Model Feature Agreement ===
```

```r
cat("In top 10 across all five models:\n")
```

```
## In top 10 across all five models:
```

```r
if (length(agreed_all) == 0) cat("  (none)\n") else
  cat(" ", paste(agreed_all, collapse = ", "), "\n")
```

```
##   energy, instrumentalness, speechiness, valence, danceability, liveness, tempo
```

```r
cat("\nIn top 10 for all tree-based models (DT + RF + XGB):\n")
```

```
##
## In top 10 for all tree-based models (DT + RF + XGB):
```

```r
if (length(agreed_tree) == 0) cat("  (none)\n") else
  cat(" ", paste(agreed_tree, collapse = ", "), "\n")
```

```
##   energy, acousticness, speechiness, instrumentalness, tempo, valence, danceability, liveness
```

```r
cat("\nIn top 10 for both linear models (LR + Lasso):\n")
```

```
##
## In top 10 for both linear models (LR + Lasso):
```

```r
if (length(agreed_lin) == 0) cat("  (none)\n") else
  cat(" ", paste(agreed_lin, collapse = ", "), "\n")
```

```
##   energy, time_signature_X0, instrumentalness, speechiness, valence, danceability, duration_m
s, liveness, time_signature_X1.4, tempo
```

The cross-model feature agreement analysis identifies which features are credibly important regardless of modeling assumption. Because each model uses a different importance measure and a different inductive strategy, a feature that consistently appears in the top 10 across all five models cannot be an artifact of any single model's structure. Its presence across all top 10 lists is the strongest model-agnostic evidence available in this analysis for which audio dimensions encode production aesthetic.

Seven features appear in the top 10 across all five models: energy, instrumentalness, speechiness, valence, danceability, liveness, and tempo. These seven form the core acoustic signature of the maximalist-alternative distinction as represented in Spotify audio features. Their agreement spans two linear models that assume an additive boundary, a single decision tree that learns rule-based splits, a random forest that aggregates many decorrelated trees, and a gradient boosting model that learns residual structure sequentially. A feature that is consistently important across all of these fundamentally different learning strategies is capturing a stable and robust signal that does not depend on any single modeling assumption.

The tree-based models agree on eight features, adding acousticness to the universal seven. This difference is directly interpretable. Acousticness is strongly negatively correlated with energy, and in linear models fitted jointly, the shared variance is absorbed primarily by energy, leaving acousticness with reduced apparent importance. Tree-based models do not combine features additively in the same way, so after energy is used in early splits, acousticness can still contribute additional separation in later splits. As a result, all tree-based models identify acousticness as important even though it does not consistently appear in the linear top 10. This indicates that acousticness should still be treated as part of the core feature set, with its importance masked in linear models by multicollinearity.

The linear models agree on ten features, adding duration_ms, time_signature_X0, and time_signature_X1.4 to the universal seven. These features appear only in the linear agreement set and are largely absent from tree-based top rankings. Duration contributes a steady directional effect that is easily captured in a linear coefficient but less useful for threshold-based splitting once stronger predictors are included. The time signature variables reflect sparse categorical effects that can produce large coefficients in logistic regression and Lasso due to leverage from rare categories, especially time_signature_X0, but do not provide stable splitting utility in tree-based models where such sparse events have limited impact on impurity reduction.

Taken together, the three agreement sets form a hierarchy of feature credibility. The core set of seven features represents the most robust and model-independent signal in the entire pipeline. Adding acousticness expands this core in a way that is suppressed only by linear modeling structure rather than absence of signal. The additional linear-only features represent weaker and more model-dependent effects that still carry predictive value but are less stable across different learning strategies. Any downstream interpretation or deployment should prioritize the universal seven features, with acousticness treated as an additional core signal supported by non-linear models.

10.6 Feature Importance Visualization

```r
# Combine datasets and slice based on ABSOLUTE importance
importance_data <- bind_rows(lr_imp_df, lasso_imp_df, dt_imp_df, rf_imp_df, xgb_imp_df) %>%
  group_by(model) %>%
  # Determine top 10 using the absolute magnitude of the coefficient
  mutate(abs_imp = abs(importance)) %>%
  slice_max(abs_imp, n = 10, with_ties = FALSE) %>%
  # Now normalize the original importance value for plotting
  mutate(importance = importance / max(abs(importance), na.rm = TRUE)) %>%
  ungroup() %>%
  mutate(
    # FIX: Map raw model names to your specific factor level strings
    model = case_when(
      model == "Logistic Regression" ~ "Logistic Regression (|Coef|)",
      model == "Lasso"               ~ "Lasso (|Coef|)",
      model == "Decision Tree"       ~ "Decision Tree (Split Importance)",
      model == "Random Forest"       ~ "Random Forest (Permutation)",
      model == "XGBoost"             ~ "XGBoost (Gain)",
      TRUE                           ~ model
    ),
    feature = paste0(feature, "__", model),
    feature = reorder(feature, importance),
    model   = factor(model, levels = c(
      "Logistic Regression (|Coef|)", "Lasso (|Coef|)", "Decision Tree (Split Importance)",
      "Random Forest (Permutation)",  "XGBoost (Gain)"
    ))
  )

# Clean, raw string with standard newline escapes
subtitle_text <- "Linear models: color shows direction of effect.\nTree models: direction varies
by feature value and interaction context."

# Drop the unused factor levels cleanly right before plotting
importance_data_clean <- importance_data %>%
  mutate(feature = factor(feature)) # This completely strips out the invisible placeholders!

# Plotting
ggplot(importance_data_clean, aes(x = feature, y = importance, fill = direction)) +
  geom_col(color = "black", alpha = 0.85, width = 0.7) +
  scale_x_discrete(labels = function(x) sub("__.*", "", x)) +
  scale_fill_manual(
    values   = c("steelblue", "coral"),
    na.value = "grey70",
    labels   = c("Alternative", "Maximalist", "Not applicable"),
    name     = "Direction"
  ) +
  facet_wrap(~model, scales = "free_y", ncol = 2) +
  coord_flip() +
  labs(
    title    = "Top 10 Features by Model (Relative Importance)",
    subtitle = subtitle_text,
    x        = NULL,
    y        = "Relative Importance"
  ) +
  theme_bw(base_size = 11) +
  theme(
    legend.position = "bottom",
    strip.text      = element_text(face = "bold"),
    axis.text.y     = element_text(size = 9.5),
    panel.spacing.x = unit(1.5, "lines"),
    panel.spacing.y = unit(1.8, "lines")
  )
```

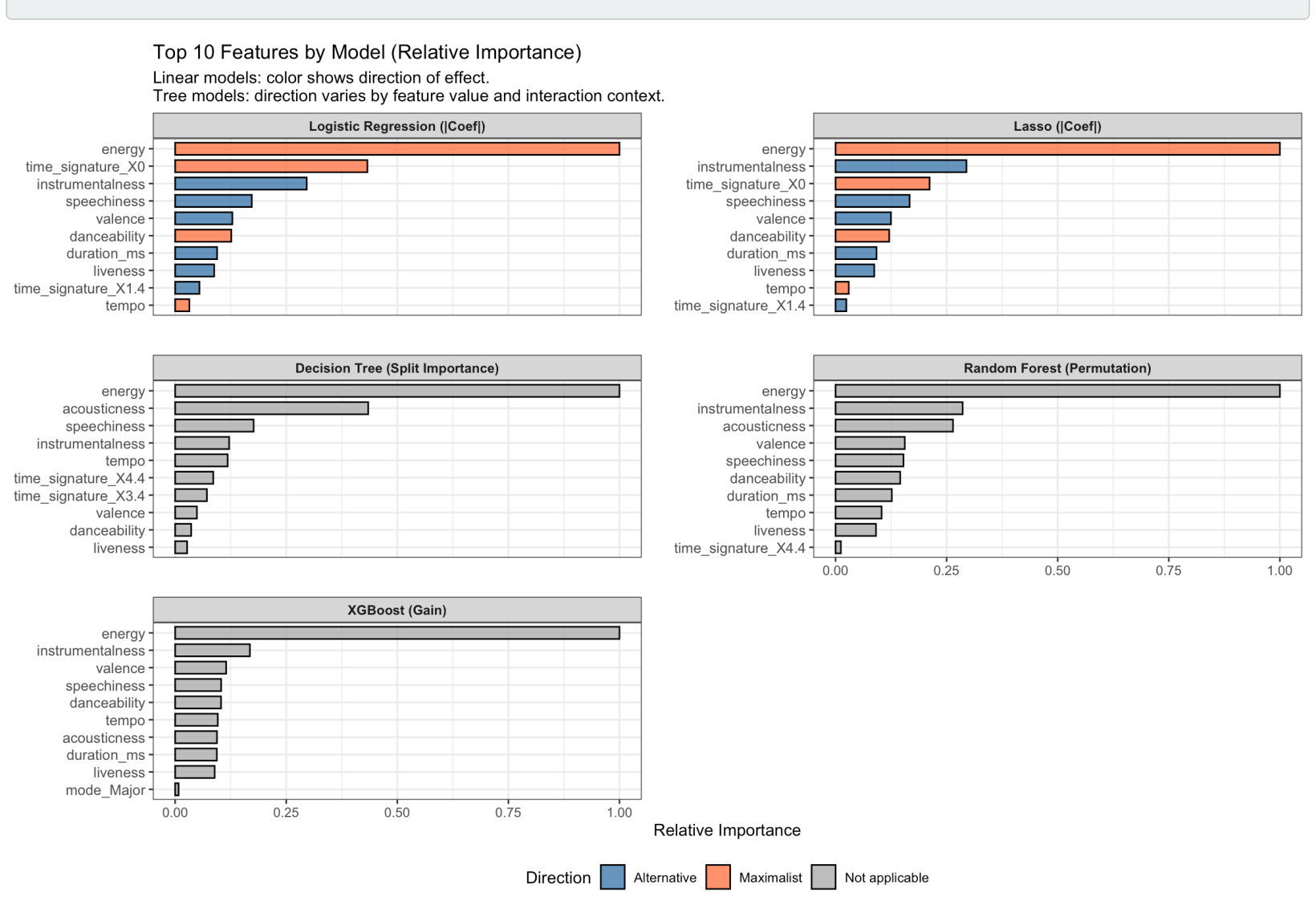


Figure 10. Normalized feature importance for all five models.

```r
sl_weights %>%
  mutate(learner = fct_reorder(learner, weight)) %>%
  ggplot(aes(x = learner, y = weight, fill = model_type)) +
  geom_col(color = "black", alpha = 0.85) +
  geom_text(aes(label = round(weight, 3)), hjust = -0.1, size = 3.5) +
  scale_fill_brewer(palette = "Set2", name = "Model Family") +
  scale_y_continuous(
    limits = c(0, max(sl_weights$weight) * 1.25)
  ) +
  coord_flip() +
  labs(
    title    = "Super Learner: Contribution of Each Candidate Model",
    subtitle = paste("Weights reflect each learner's contribution",
                     "\nvia cross-validated stacking.",
                     "\nNot variable importance."),
    x = NULL, y = "Weight"
  ) +
  theme_bw(base_size = 12) +
  theme(legend.position = "bottom")
```

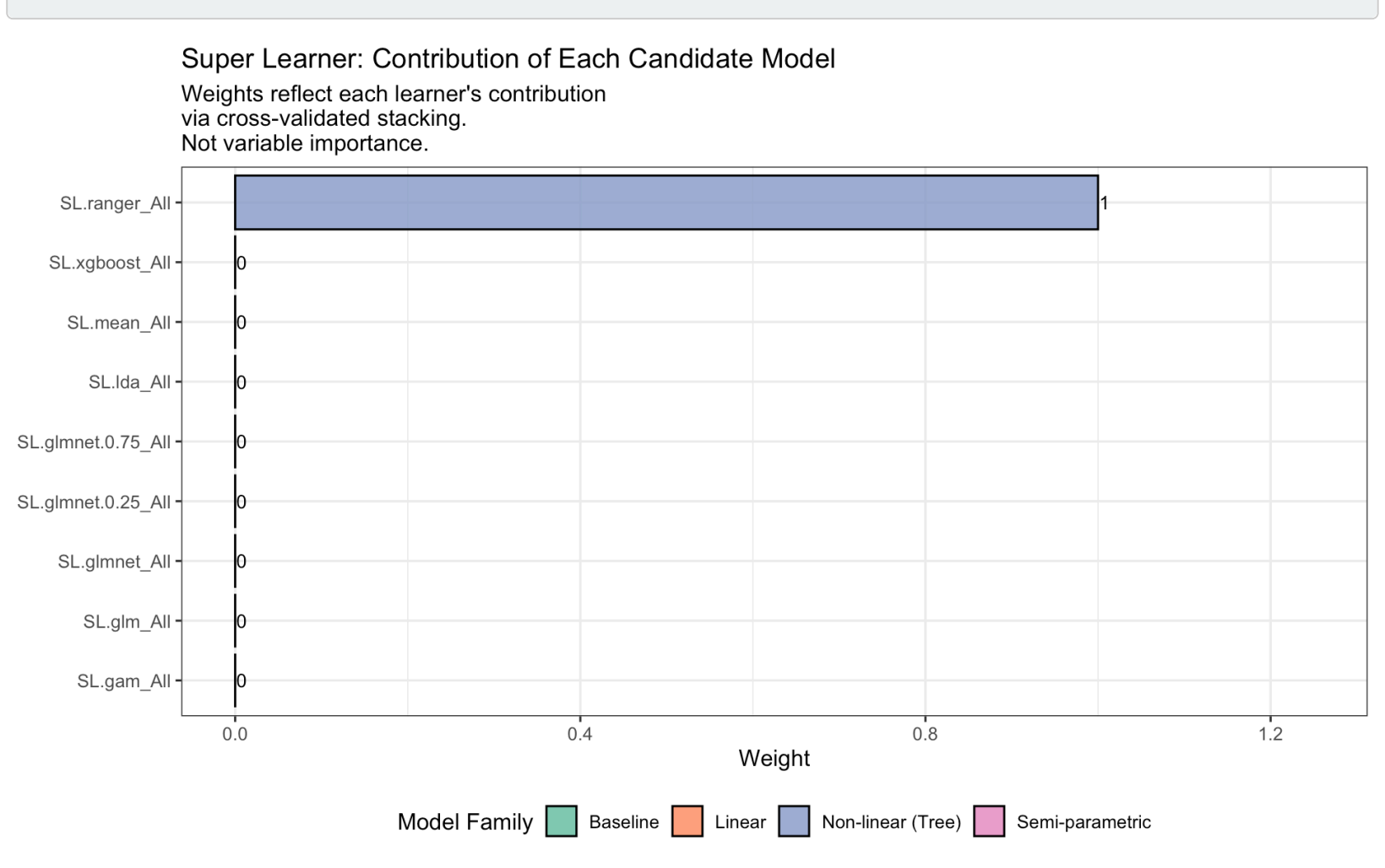


Figure 11. Super Learner candidate model weights.

10.7 Substantive Interpretation

Features that appear in the top 10 across all six models, regardless of whether the model assumes a linear boundary, learns threshold rules through recursive partitioning, or builds a non-linear ensemble, represent the most reliably important audio correlates of production aesthetic in this dataset. Seven features satisfy this criterion: energy, instrumentalness, speechiness, valence, danceability, liveness, and tempo. These features consistently carry signal across all modeling assumptions tested here, and their importance cannot be attributed to any single model's inductive bias or importance metric. A feature that is consistently important for both logistic regression and gradient boosting is one whose relationship with the outcome is stable under both additive and interaction based representations of the feature space.

Acousticness occupies a distinct intermediate role. It appears in the top 10 of all three tree-based models but not in the two linear models, making it a feature whose importance is real but partially suppressed under linear assumptions. Its contribution depends on joint relationships in the feature space, especially its strong negative association with energy. Linear models absorb this shared variation primarily through the energy coefficient, which reduces the apparent contribution of acousticness. Tree-based models do not rely on additive structure in the same way and can recover its effect through sequential threshold splits, allowing acousticness to emerge as important once energy has already been accounted for. This supports treating acousticness as part of the core feature set for any non-linear modeling approach, even though it is not consistently ranked in linear models.

The Lasso retention results in Section 9.3 further clarify the structure of the linear signal. Lasso retained 27 of 28 features and produced an identical top 10 to logistic regression, indicating that regularization does not meaningfully alter which features carry linear signal. Rather than identifying a sparse subset of predictors, the linear structure is distributed across nearly all available features. This means the universally shared features are not the only relevant predictors in the linear setting, but they are the most stable and consistently influential ones. The remaining features still contribute additive signal, even if they do not consistently appear in top rankings.

The Super Learner weight results provide the most direct model level interpretation of these feature level findings. The assignment of full weight to SL.ranger and zero weight to all other learners, including linear models, XGBoost, GAM, and LDA, indicates that the non-linear tree based structure captures all predictive information available in the feature space. If linear and non-linear learners were capturing complementary aspects of the signal, the meta learner would assign positive weight to multiple model families. Instead, the all or nothing allocation indicates that the signal captured by linear models is fully contained within what the random forest already learns, rather than representing a separate source of structure.

The zero weight on SL.mean confirms that the feature set contains strong predictive signal beyond baseline prevalence, and the zero weight on SL.xgboost indicates that gradient boosting does not add information beyond what random forest already captures in this setting. Taken together, the feature level and model level results converge on a single interpretation. Production aesthetic is primarily determined by a non-linear interaction structure in the audio feature space, with energy and acousticness forming the central organizing axis and the remaining seven shared features providing secondary refinement. Random forest is the most appropriate model in this pipeline because it is the only candidate that fully captures this joint structure without requiring additive assumptions.

---

11 Conclusion

This project developed and evaluated a multi-model pipeline for classifying music production aesthetic, alternative versus maximalist, using Spotify audio features. Loudness was intentionally excluded to avoid circularity and to test whether the aesthetic distinction remains detectable beyond its most obvious proxy.

The exploratory analysis showed that acousticness and energy carry the strongest linear signal, consistent with the idea that maximalist production reflects compressed, high intensity sound while alternative production is more organic and less processed. Danceability and valence showed substantial overlap across classes, which motivated the use of models capable of capturing non-linear structure and feature interactions rather than relying on additive effects alone.

The performance gradient, from logistic regression through a decision tree and onward to random forest, XGBoost, and the Super Learner, shows how much predictive power comes from non-linearity. The gap between the linear models and the best tree based methods reflects signal that only appears when features are allowed to interact. The decision tree sits between the linear models and the ensemble methods, offering a small but clear improvement by introducing threshold based splits, while the ensemble methods capture substantially more structure.

The directional error analysis showed that the feature space is not symmetric in how it handles mistakes. Ambiguous songs are more often pushed toward the maximalist class than the alternative class, suggesting a structural tilt in how borderline observations are represented in the feature space.The error overlap analysis clarified that many models fail on the same subset of songs, but not all. A small group of tracks is consistently misclassified by every model, which sets a practical limit on how well this feature set can separate the two aesthetics. These cases likely reflect genuine aesthetic hybridity rather than model weakness.

The feature importance analysis identified the audio features that remain stable across all modeling approaches. Features that consistently appear in the top ranks across different importance definitions are the most reliable signals of production aesthetic, since they are not dependent on a single modeling assumption. The Super Learner weights provide a final summary at the model level. The ensemble assigns full weight to the random forest model, with other learners contributing no additional improvement in cross validated performance. This indicates that the non linear tree based structure captures the relevant signal in a way that linear and alternative non linear models do not meaningfully extend.

Taken together, the results suggest that production aesthetic is strongly encoded in audio features and is best captured through non linear decision structures. A streaming platform using this system would achieve performance close to the best ensemble models, with remaining errors concentrated in tracks that sit near the boundary between the two aesthetics.

---

12 Implementation Plan

Estimating a machine learning model is only the first step in moving from analysis to real use. A typical deployment workflow moves through several stages, starting from defining the practical use case, collecting and validating data, building an initial model, testing it in a prototype setting, and then deploying it with continuous monitoring and updates.

The final model used in this project, the Super Learner, is effectively equivalent to the random forest because it assigns all weight to that learner. In practice, this model can be packaged as a simple API using tools such as vetiver and plumber in R. A new song can be scored by sending its audio feature vector to the API, which returns a probability of maximalist production along with a binary classification based on a chosen threshold.

The threshold of 0.5 is used throughout this study for consistency, but it is not fixed in practice. Different applications may shift this threshold depending on whether they prioritize reducing false positives or false negatives. The directional error results in Section 8 show that alternative songs are more often misclassified as maximalist than the reverse, which suggests that some applications may benefit from raising the threshold above 0.5 when false maximalist predictions are more costly.

A key practical issue in deployment is the difference between the training data and real time data. The dataset used here is clean and structured, while live Spotify feature extraction may change over time as audio processing methods are updated. This creates a risk that incoming data may not follow the same distribution as the training data. For this reason, incoming features should be checked before prediction, including basic range validation and checks for unusual feature combinations that were not present in training. These safeguards help prevent silent failures where the model produces confident but unreliable predictions on out of distribution inputs.

Before full release, it is useful to run the system in a silent mode where predictions are generated but not shown to users. This allows monitoring of stability, detection of data pipeline issues, and adjustment of validation rules without affecting end users. In higher stakes settings, a controlled experiment can be used, where a portion of users receive recommendations based on the model while others receive standard recommendations. This allows direct measurement of whether the model improves user outcomes in practice.

There are also limits to how far this model can be applied across different cultural contexts. The labels used here are based on a producer network drawn mainly from Anglophone popular music. Applying the model to other musical traditions may impose this framework onto styles that follow different production norms, which can distort the interpretation of predictions. In those cases, the results should be treated as provisional and ideally validated against domain specific expertise or retrained using a labeling system grounded in the relevant musical context.

A related limitation is that the current binary structure may not capture new or emerging aesthetic categories. If new production styles develop over time, songs from these styles may be systematically misclassified because they do not fit the existing labels. This should not automatically be treated as model error, but rather as a signal that the underlying labeling system needs updating. Because the labels are derived from community structure in the producer network, the same method can be rerun on updated data to identify new clusters. This allows the classification system to evolve over time while maintaining a discrete category structure rather than forcing a continuous reformulation of aesthetic identity.

---

13 Monitoring and Recalibration Plan

Once the model is deployed, monitoring should run continuously across six core areas. These include the distribution of incoming input features, patterns of missing data, system usage over time, the distribution of predicted probabilities, predictive performance where new ground truth labels become available through periodic re-labeling using the producer network pipeline, and system level performance such as API latency and error rates. These signals are typically tracked using control charts based on statistical process control, which highlight shifts in mean and variability and flag situations where the system moves outside expected bounds.

Two main types of drift need to be tracked over time. Feature drift refers to changes in the distribution of input variables in new music compared to the training data. This can happen if production styles evolve or if Spotify updates how audio features are computed, leading to new combinations of values that were not present before. Feature drift does not always require immediate retraining, but it is an early warning signal that a new model should be trained in parallel while the current one is still in use. Calibration plots are often among the first indicators of this issue, since systematic overprediction or underprediction can appear before overall discrimination performance declines. Model drift refers to a direct decline in predictive performance, typically measured using metrics such as AUC and directional error rates on newly labeled data.

A more subtle issue is that the relationship between features and labels can change over time. This is not just a shift in input values, but a shift in meaning. The model learns associations such as high energy combined with low acousticness being linked to maximalist production. These associations reflect the historical period in which the training data was collected, and they are not fixed properties of music. As production styles evolve, the same feature patterns may come to represent different aesthetics. When this happens, the model continues to apply old associations to new contexts, leading to systematic errors on more recent music.

This type of change is different from feature drift because the input distribution may remain stable while the meaning of the features changes. It is often referred to as concept drift. Unlike feature drift, it cannot be detected reliably from input data alone and requires updated labels on new releases. If directional error rates increase for recent music while overall feature distributions remain stable, this is a strong indicator of concept drift rather than a data quality issue. In that case, retraining the model on updated labeled data is necessary, rather than simply recalibrating outputs.

There is also a producer level version of this issue. A producer may shift style over time, but the labeling system assigns a single static aesthetic label based on their position in the producer network. This means earlier works by that producer may be labeled in a way that no longer matches their later aesthetic direction. These cases introduce structured label noise that is tied to time rather than random error. Without a time aware labeling system, this cannot be fully corrected, so performance on older catalog entries from evolving artists should be interpreted carefully.

Monitoring also needs to account for the fact that the system itself can change the data it observes. If predictions are used to shape playlists or recommendations, then the distribution of user listening behavior will shift in response to those predictions. This makes it difficult to measure true model performance because the model is partially influencing the outcomes it is evaluated on. A practical solution is to maintain a small control group where predictions are not applied, allowing performance to be measured on data that is not affected by the system.

When drift is detected, there are two main responses. If the main issue is that predicted probabilities are no longer aligned with observed outcomes but ranking performance remains stable, then recalibration methods such as Platt scaling can be used. This adjusts the probability outputs without changing the underlying model. If ranking performance itself degrades, then a full retraining is required using updated data.

A structured governance plan is needed to manage these updates safely. This plan should define clear triggers for retraining, such as sustained drops in performance metrics or statistically significant shifts in feature distributions. It should also specify how models are validated before deployment and how data is separated to avoid leakage between training and evaluation. Finally, it should include an assessment of the real world impact of any changes, particularly in systems where predictions influence user experience and downstream decision making.

References

Askin, N., & Mauskapf, M. (2017). What Makes Popular Culture Popular? Product Features and Optimal Differentiation in Music. American Sociological Review, 82(5), 910–944. https://doi.org/10.1177/0003122417728662

Asmitha, M., & Kavitha, C. R. (2026). Analyzing the Popularity of Soundscapes: Genre Classification and EDA Using Spotify. In A. Bhattacharya, S. Dutta, P. Dutta, & A. Rocha (Eds.), Data Mining and Information Security (Vol. 1387, pp. 149–175). Springer Nature Singapore. https://doi.org/10.1007/978-981-96-6060-5_12

Azerrad, M. (2002). Our band could be your life: Scenes from the American Indie underground 1981 - 1991 (1. Back Bay paperback ed). Little, Brown and Company.

Gracyk, T. (1996). Rhythm and noise: An aesthetics of rock. Duke university press.

Kim, K., & Askin, N. (2024). Feature-Based Structures of Opportunity: Genre Innovation in the American Popular Music Industry, 1958 to 2016. American Sociological Review, 89(3), 542–583. https://doi.org/10.1177/00031224241246271

Reynolds, S. (2006). Rip It up and Start Again: Postpunk 1978-1984. Penguin Publishing Group.

Seabrook, J. (2015). The song machine: Inside the hit factory (First edition). W. W. Norton & Company.

Vickers, E. (2010). The Loudness War: Background, Speculation and Recommendations.

Walser, R. (1993). Running with the Devil: Power, Gender, and Madness in Heavy Metal Music. University Press of New England.
